

UNDERGRADUATE FINAL YEAR PROJECT REPORT
Department of Computer and Information Systems Engineering
NED University of Engineering and Technology



In-Line Tablet Quality Control using Computer Vision

Real-Time Automated Inspection for Pharmaceutical Precision

Group Number: 16
Group Member Names:

Batch: 2022 - 2026

Manahil Ejaz

CS-22011

Mahnoor Zia

CS-22101

Anoosha Khalid

CS-22104

Laiba Iqar

CS-22112

.....
Dr. Syed Zaffar Qasim

Assistant Professor

Project
Advisor

THIS PAGE IS INTENTIONALLY LEFT BLANK



Author's Declarations

We declare that we are the sole authors of this project. It is the actual copy of the project that was accepted by our advisor(s) including any necessary revisions. We also grant NED University of Engineering and Technology permission to reproduce and distribute electronic or paper copies of this project.

Signature and Date	Signature and Date	Signature and Date	Signature and Date
.....			
.	.	.	.
Manahil Ejaz	Mahnoor Zia	Anoosha Khalid	Laiba Iqrar
CS-22011	CS-22101	CS-22104	CS-22112
manahilejaz16@gmail.com	mahnoorzia205@gmail.com	anooshakhalid999@gmail.com	laibaiqarahmedkhan@gmail.com

Statement of Contributions

This Final Year Project was carried out collaboratively by our team of four members, with each member contributing to specific aspects of the project based on their expertise and assigned responsibilities. The contributions are summarized as follows:

Manahil Ejaz

Manahil was responsible for data collection from online sources and conveyor footage, as well as dataset cleaning and filtering. She carried out iterative annotation work covering classification, defect-only detection, and segmentation masks, and managed the preprocessing and augmentation pipeline. Her model development involved progressive training across YOLOv8n classification, YOLOv5n detection, and YOLOv11n-seg, along with the implementation of a two-stage pipeline and the final segmentation model training and evaluation.

Mahnoor Zia

Mahnoor was involved in the industry visit. She contributed to data annotation, designed the Human-Machine Interface (HMI) initial prototype and was responsible for managing and procuring the hardware components required for the inspection system. She assisted in the physical assembly and integration of all hardware modules. She also contributed to system testing & validation.

Anoosha Khalid

Anoosha participated in the industry visit and dataset creation of a chipped tablets. She was responsible for designing and developing the complete inference pipeline of the proposed inline tablet quality control system and designing the overall hardware architecture. She integrated various system components to enable real-time inspection, data processing, logging and monitoring. Additionally, she performed final system integration and conducted testing and validation of the complete system in multiple operating environments to evaluate its functionality, performance, and reliability.

Laiba Iqrar

Laiba contributed to the industry visit and defected tablet sample collection. She was responsible for designing the architecture of two-stage pipeline and entire segmentation strategy. She developed and tested multiple mathematical CV techniques. She implemented the hybrid approach for classical and Deep learning architecture, participated in data annotation, preprocessing, dataset creation and EDA.

All team members actively participated in project discussions, design reviews, and validation of results. The writing of the final report, presentation slides, and research paper was a joint effort, with each member contributing sections aligned to their work.

Executive Summary

Conventional pharmaceutical tablet quality control relies largely on off-line testing performed on limited samples, which is time-consuming, labor-intensive, and prone to missing critical defects. This limitation highlights the need for an automated in-line inspection approach capable of ensuring consistent product quality while minimizing waste. This project presents an in-line tablet quality control system based on machine vision and deep learning for real-time inspection of tablet cores.

The proposed system employs a conveyor-mounted imaging setup integrated with controlled illumination and a high-resolution RGB camera to capture detailed surface images of tablets under motion. A custom dataset was developed using images collected across multiple orientations and lighting conditions to ensure robustness. Five common surface defect categories, namely surface defects, blur or smear, edge chipping, spots, and scratches, were annotated and used for model training and evaluation. A YOLO-family deep learning model was trained and optimized to perform real-time defect detection, with emphasis on accuracy, low latency, and high throughput suitable for industrial environments.

The trained model was deployed within an end-to-end inference pipeline comprising image acquisition, preprocessing, defect detection, post-processing, and result logging. A web-based HMI was developed to provide live inspection statistics, per-class defect counts, batch summaries, and quality control reports, supporting traceability and audit requirements. Performance evaluation on held-out in-house data demonstrated that the system achieves high defect detection accuracy, sensitivity, and specificity, while meeting real-time throughput constraints required for in-line inspection.

The results indicate that the proposed system enables 100 percent inspection of tablet cores and significantly reduces dependence on conventional off-line testing. By facilitating early defect identification and timely intervention, the system enhances patient safety, improves manufacturing efficiency, and reduces material wastage. Overall, this project demonstrates the feasibility and practicality of deploying a deep learning-based machine vision system as a scalable and industry-aligned solution for pharmaceutical tablet quality control.

Acknowledgments

We would like to express our sincere gratitude to **NutraSource International** for their valuable support throughout this project. The company arranged an informative industrial visit for our team, during which we were introduced to the complete tablet manufacturing process. We were guided on how different tablet defects appear, their possible causes, and preventive measures. In addition, we were shown the existing manual quality assurance procedures used for defect inspection prior to packaging and were informed how the proposed system can help reduce manual labor and improve inspection efficiency. This industrial exposure significantly enhanced our understanding of practical pharmaceutical quality control practices.

We would also like to acknowledge our project advisor, **Dr. Syed Zaffar Qasim**, for his continuous guidance and support throughout the project. Through his professional contacts, he facilitated access to a relevant dataset provided by pharmaceutical experts, which played an important role in model development and evaluation. His technical guidance, constructive feedback, and encouragement were instrumental in the successful completion of this project.

We are also thankful to the Department of Computer and information Systems Engineering, NED, for providing the environment, laboratories and resources for the initial research and development for this project. We would also like to acknowledge the use of publicly available datasets, research literature and open-source tools that have supported our progress. We are grateful to our families and classmates for their encouragement and motivation throughout this phase of the project.

With gratitude,

Inline Tablet Quality Control Team,

Manahil Ejaz

Mahnoor Zia

Anoosha Khalid

Laiba Iqrar

Table of Contents

Author's Declaration	iii
Statement of Contributions	iv
Executive Summary.....	vi
Acknowledgments	vii
Table of Contents	viii
List of Figures.....	xiii
List of Tables.....	xvi
United Nations Sustainable Development Goals	xvii
Similarity Index Report	xviii
Chapter 1 Introduction.....	1
1.1 Background Information.....	1
1.2 Significance and Motivation.....	1
1.2.1 Importance of In-Line Quality Inspection	1
1.2.2 Reduction of Manual labor and human error	1
1.2.3 Industrial Relevance and Patient safety.....	1
1.3 Aims and Objectives.....	2
1.4 Methodology.....	2
1.5 Report Outline.....	2
Chapter 2 Literature Review	3
2.1 Introduction.....	3
2.2 The Limitations of Conventional Quality Control (QC)	3
2.2.1 Destructive testing & sampling risk	3
2.2.2 Process Variability & Defect type.....	3
2.3 Review of Commercial High-Speed Inspection Systems	4
2.3.1 Proprietary solutions from Industry leaders	4
2.3.2 Commercial Limitations	6
2.4 Novelty and Proposed Contribution	7

2.4.1 Transition to Deep Learning systems	7
2.4.2 Architectural Advantages of the Proposed solution	7
2.5 Summary.....	7
Chapter 3 System Design and Methodology	9
3.1 Introduction.....	9
3.2 Hardware Setup.....	9
3.2.1 Tablet Image Acquisition Setup	9
3.2.2 Raw Data Collection	10
3.3 Software System Design and Methodology	10
3.3.1 Initial Data Preparation	10
3.3.2 Data Annotation and Dataset Management	10
3.3.3 Dataset Export and Model Training	10
3.3.4 Model Evaluation	11
3.3.5 Model Inference and User Interface	11
3.1 Conclusion	11
Chapter 4 Model Development and Training	12
4.1 Introduction.....	12
4.2 Progressive Model Training Phase	12
4.3 Model Development Using Mathematical Vision-Based Defect Detection.....	27
4.3 Summary.....	39
Chapter 5 Hybrid Approach for Tablet Localization & Defect Segmentation.....	40
5.1 Introduction.....	40
5.2 Core Rationales for a Decoupled Architecture.....	40
5.2.1 Eliminating Background Noise and Target Isolation	40
5.2.2 Drastic Reduction in Computational Complexity and Inference Latency.....	40
5.2.3 Data Scarcity and Accelerating Model Convergence.....	40
5.3 Hybrid System Architecture.....	41
6.7 Summary.....	41
Chapter 6 Hardware Architecture & Deployment	49
6.1 Introduction....	49

6.2 Overall Hardware System Architecture.....	49
6.3 Tablet Transport & Conveyor System.....	51
6.3.1 Functional Role.....	51
6.3.2 Mechanical Design Consideration	51
6.3.3 Physical Construction.....	51
6.3.4 Speed Regulation.....	53
6.3.5 Engineering Considerations.....	54
6.3.6 Limitations of Prototype Conveyor.....	55
6.4 Imaging System (Camera Module)	55
6.4.1 Role of the Imaging System.....	55
6.4.2 Camera Selection & Specification	55
6.4.3 Camera Placement Strategy & Lighting Setup.....	56
6.4.4 Image Acquisition Process.....	57
6.5 Edge Inference & Processing Hardware	58
6.5.1 Raspberry Pi 4 Hardware Module.....	58
6.5.2 Separation of Edge and Monitoring Infrastructure	59
6.6 Development Strategy & Operational Environment of the Inspection System.....	60
6.6.1 Objectives of System Deployment.....	60
6.6.2 Transition from Development Environment to Operational Environment	60
6.6.3 Overview of the final Deployment Architecture.....	61
6.6.4 Data & System Integration Workflow.....	63
6.6.5 System Constraints & Limitations.....	64
6.7 Summary.....	65
Chapter 7 System Testing, Deployment Constraints & Performance Evaluation.....	66
7.1 Introduction....	66
7.2 Configuration & Remote Development Environment.....	66
7.2.1 Raspberry Pi OS Configuration.....	66
7.2.2 SSH Based Remote Administration	67

7.3 Evolution of a Deployed Architecture	68
7.3.1 OpenCV Socket based Streaming.....	68
7.3.2 Challenges Encountered during Socket based Streaming	69
7.3.3 Transition towards RTSP based Streaming.....	69
7.3.4 Challenges Encountered during RTSP based Streaming.....	70
7.3.5 Adoption of RealVNC for Edge Device.....	70
7.3.6 Comparative Analysis of Monitoring Approaches.....	72
7.4 InfluxDB Integration & Data Persistence.....	72
7.4.1 Motivation for Database Integration.....	72
7.4.2 Selection of InfluxDB.....	73
7.4.3 Installation & Configuration of InfluxDB.....	74
7.4.4 Inspection Data Structure.....	75
7.4.5 Database Communication Validation.....	75
7.4.6 Benefits of Continous Inspection logging	77
7.4.7 InfluxDB Dashboard.....	77
7.5 Dashboard Development & Visualization	78
7.5.1 Grafana Installation & Configuration.....	78
7.5.2 Configuration of Data Source	79
7.5.3 Dashboard Design Objectives.....	80
7.5.4 Development of Production Statistics Panel.....	81
7.5.5 Real-time Tablet Inspection Dashboard.....	83
7.6 System Testing & Experimental Validation	83
7.1.1 Testing Methology.....	83
7.6.2 Testing in Laboratory Environment.....	84
7.6.3 Testing in Home Environment.....	84
7.6.4 Real-time Defect Validation.....	85
7.6.5 Conveyor based Inspection Testing.....	85
7.5.6 Throughput Estimation & Production Capacity Analysis.....	87
7.6.7 End-to-End System Validation.....	87
7.7 Summary.....	89
Chapter 8 Conclusions	9

8.1 Summary.....	90
References	91

List of Figures

Figure 2.1: Viswill Automated Visual Inspection System	4
Figure 2.2: Type of Defects Covered by the Viswill's System	5
Figure 2.3: Enclony Automated Visual Inspection System	5
Figure 2.4: Tablet Inspection Process	6
Figure 2.5: Inspection Criteria of Cognex System	6
Figure 3.1: Hardware Setup	10
Figure 3.2: Visualization of Preprocessed Dataset	12
Figure 3.3: Two-way Annotation Hierarchy	13
Figure 3.4: CVAT Annotation Snapshot	15
Figure 4.1: Two-Stage Pipeline for Segmentation	20
Figure 4.2: Mask PR Curve of Rapid Edge (Yolo-seg 1st Stage)	25
Figure 4.3: Confusion Matrix of Rapid Edge (Yolo-seg 1st Stage)	25
Figure 4.4: Mask PR Curve of 1.1 Augmented (Yolo-seg 2nd Stage)	26
Figure 4.5: Confusion Matrix of 1.1 Augmented (Yolo-seg 2nd Stage)	26
Figure 4.6: Confusion Matrix of Run3_SGDCosine (Mask RCNN) Best Model	29
Figure 4.7: Model Comparison of MaskRCNN Architectures	30
Figure 4.8: Pipeline for Classical Approach to Tablet Contour Localization	31
Figure 4.9: Tablet Localization Using Adaptive Segmentation and Morphological Refinement	33

Figure 4.10: Geometric Deviation and Global Gradient Profiling	33
Figure 4.11: Real-time Results of Variation-1	36
Figure 4.12: Real-time Results of Variation-2	39
Figure 4.13: Colorimetric Profiling and Morphological Feature Filtering Flow	40
Figure 4.14: Real-time Results of Variation-3	43
Figure 5.1: Real-time Result of Hybrid Approach on a Conveyor Belt	45
Figure 5.2: Mathematical and Deep Learning Hybrid Architecture	47
Figure 6.1: Hardware System Architecture	50
Figure 6.2: Conveyor Belt After Phase I.....	53
Figure 6.3: Conveyor Belt during Repairing Process.....	53
Figure 6.4: Conveyor Belt in use.....	53
Figure 6.5: Auto Focus HD Webcam.....	56
Figure 6.8: Setup of a Camera.....	57
Figure 6.9: Setup of a Ring Light.....	57
Figure 6.10: Raspberry Pi 4 Model B.....	58
Figure 6.9: Hardware-to-Software Data Folow Pipeline.....	62
Figure 6.10: Hardware-to-Software Setup	62
Figure 6.11: System Integration WorkFlow	63
Figure 7.1: PuTTY Interface.....	68
Figure 7.2: Socket Based Streaming	69
Figure 7.3: RTSP Based Streaming	70

Figure 7.4: RealVNC Communication	71
Figure 7.5: VNC Terminal	71
Figure 7.6: InfluxDB Deployment WorkFlow.....	74
Figure 7.7: InfluxDB Login Interface.....	75
Figure 7.8: Data Bucket of a Tablet.....	76
Figure 7.9: Real-time Data loggings.....	77
Figure 7.10: InfluxDB Dashboard.....	77
Figure 7.11: Grafana Deployment WorkFlow.....	79
Figure 7.12: Grafana Login Interface.....	79
Figure 7.13: Successful Configuration of the InfluxDB Source within Grafana.....	80
Figure 7.14: Data Source login within Grafana	80
Figure 7.15: Panel Development WorkFlow	82
Figure 7.16: Panel Selection & Designing Process.....	82
Figure 7.17: Tablet Quality Control Cumulative Dashboard.....	83
Figure 7.17: Tablet Quality Control Real-time Dashboard.....	83
Figure 7.18: Testing in laboratory Environment.....	84
Figure 7.19: Testing in laboratory Environment	84
Figure 7.20: Real time Defect Detection	85
Figure 7.21: Conveyor Based Inspection Testing	87
Figure 7.22: End-to-End System	88

List of Tables

Table 3.1 : Finalized dataset partition and composition matrix.....	16
Table 3.2 : Semantic class instance distribution within the training partition.....	17
Table 4.1: Model Comparison	13
Table 4.2: Hyperparameter and Augmentation Mapping for Stage-1 Initializations	22
Table 4.3 : Comparative Analysis of Stage-1 Model Performance and Inference Latency.....	22
Table 4.4 : Architectural and Hyperparameter Comparison of Stage-2 YOLO11n-seg	24
Table 4.5 : Comparative Analysis of Stage-2 Model Performance and Inference Latency	24
Table 4.6 : Architectural and Hyperparameter Comparison of Mask RCNN	28
Table 4.7 : Comparative Analysis of Mask RCNN Performance and Inference Latency	28
Table 4.8 : Cross-Architectural Comparison Matrix (YOLO11n-seg vs. Mask R-CNN)	31
Table 6.1 : Development & Refinement Process of the Conveyor System.....	52
Table 6.2 : Specification of a Camera	56
Table 6.3 : Raspberry Pi Specifications	59
Table 7.1 : OS Development Iteration	67
Table 7.2 : Comparative Analysis of Monitoring Approaches.....	72
Table 7.3 : Inspection Data Fields stored in InfluxDB	75
Table 7.4 : Database Validation Results	76

United Nation's Sustainable Development Goals

The Sustainable Development Goals (SDGs) are the blueprint to achieve a better and more sustainable future for all. They address the global challenges we face, including poverty, inequality, climate change, environmental degradation, peace and justice. There is a total of 17 SDGs as mentioned below. Check the appropriate SDGs related to the project.

- ☐ No Poverty
- ☐ Zero Hunger
- Good Health and Wellbeing
- ☐ Quality Education
- ☐ Gender Equality
- ☐ Clean Water and Sanitation
- ☐ Affordable and Clean Energy
- ☐ Decent Work and Economic Growth
- Industry, Innovation and Infrastructure
- ☐ Reduced Inequalities
- ☐ Sustainable Cities and Communities
- Responsible Consumption and Production
- ☐ Climate Action
- ☐ Life Below Water
- ☐ Life on Land
- ☐ Peace and Justice and Strong Institutions
- ☐ Partnerships to Achieve the Goals

Similarity Index Report

Following students have compiled the final year report on the topic given below for partial fulfillment of the requirement for Bachelor's degree in Computer and Information Systems Engineering.

Project Title: In-Line Tablet Quality Control Using Computer Vision

S. No.	Student Name	Seat Number
1.	Manahil Ejaz	CS-22011
2.	Mahnoor Zia	CS-22101
3.	Anoosha Khalid	CS-22104
4.	Laiba Iqrar	CS-22112

This is to certify that Plagiarism test was conducted on complete report, and overall similarity index was found to be less than 20%, with maximum 5% from single source, as required.

Signature and Date

.....

Dr. Syed Zaffar Qasim

Chapter 1

Introduction

1.1 Background Information

Quality control is a critical component of pharmaceutical manufacturing, as it directly impacts product safety, efficacy, and regulatory compliance. Tablets must meet strict quality standards related to their physical appearance, surface integrity, and structural consistency before they can be released for packaging and distribution. Traditionally, tablet quality inspection is carried out using off-line quality control procedures, where a limited number of tablets are manually inspected or tested from each production batch. While these methods are widely used, they are time-consuming, labor-intensive, and unable to guarantee complete inspection coverage.

In modern high-speed pharmaceutical production lines, thousands of tablets are manufactured per hour, making manual inspection inefficient and prone to human error. Minor surface defects such as scratches, spots, edge chipping, or smear marks may go unnoticed, potentially affecting product quality and patient safety.

Advancements in computer vision have enabled automated visual inspection systems capable of analyzing products in real time. Deep learning-based object detection models, particularly those from the YOLO family, have demonstrated strong performance in fast and accurate defect detection. This project focuses on developing an in-line tablet quality control system using computer vision to automatically detect common surface defects on tablet cores while they move on a conveyor system.

1.2 Significance and Motivation

The significance of this project lies in its potential to modernize pharmaceutical quality control by introducing automation, accuracy, and scalability.

1.2.1 Importance of In-Line Quality Inspection

In-line inspection allows continuous monitoring of tablets during production, enabling early detection of defects. This reduces the risk of defective products reaching the packaging stage and ensures consistent quality throughout the manufacturing process.

1.2.2 Reduction of Manual Labor and Human Error

Manual inspection is subjective and depends heavily on operator skill and attention. Automating defect detection minimizes human involvement, reduces fatigue-related errors, and improves repeatability and reliability of quality checks.

1.2.3 Industrial Relevance and Patient Safety

By detecting defects early and accurately, the proposed system contributes to improved patient safety and regulatory compliance. It also aligns with modern pharmaceutical manufacturing practices such as Process Analytical Technology and Quality by Design.

1.3 Aims and Objectives

The main aim of this project is to design and develop an automated in-line tablet quality control system using deep learning and machine vision.

The specific objectives of the project are as follows:

- To design and implement a conveyor-based imaging setup with controlled illumination.
- To collect and curate a dataset of tablet images representing common surface defects.
- To annotate the dataset for supervised deep learning model training.
- To train and evaluate a YOLO-based model for real-time tablet defect detection.
- To develop an end-to-end inference pipeline integrated with a user-friendly dashboard.
- To assess the system's performance in terms of accuracy, sensitivity, specificity, and throughput.

1.4 Methodology

To achieve the project objectives, we adopted a structured methodology comprising system design, data preparation, model development, and deployment. Initially, a conveyor-based imaging system was designed using a high-resolution camera and controlled lighting to capture clear images of tablets under motion. Images were collected under multiple orientations and lighting conditions to enhance dataset diversity.

The collected images were annotated to label five defect categories: surface defect, blur or smear, edge chipping, spot, and scratch. A YOLO-family deep learning model was then trained using the annotated dataset, with appropriate data augmentation and hyperparameter tuning to

improve robustness and generalization. The trained model was deployed within a real-time inference pipeline that processes incoming images, detects defects, logs results, and displays inspection statistics through a web-based human–machine interface.

1.5 Report Outline

This report is organized into multiple chapters to clearly present the project work. Chapter 1 introduces the background, significance, objectives, methodology, and scope of the project. Chapter 2 presents a review of related work and existing approaches in pharmaceutical quality control and deep learning-based inspection systems. Chapter 3 describes the system architecture, hardware setup, and data preparation process. Chapter 4 details the model development, training process, and experimental results. Finally, Chapter 6 concludes the report and provides recommendations for future work and system enhancements.

Chapter 2

Literature Review

2.1 Introduction

The pharmaceutical industry requires unwavering precision in quality assurance to ensure patient safety. Historically, Quality Control (QC) for solid oral dosage forms (like tablets) has relied on outdated methodologies. As stated in the project brief, conventional practices involve off-line, manual and destructive tests applied on small random samples. This approach is inherently slow, labor-intensive, and, most critically, carries the risk of defective products escaping detection and reaching the consumer.

This chapter performs a critical review of the current state of tablet inspection technology. It first outlines the limitations of existing manual and semi-automated QC protocols. Subsequently, it details the high-speed, commercial machine vision systems currently deployed by major international manufacturers. Finally, it explores the emerging role of Deep Learning and Computer Vision in overcoming the cost and flexibility barriers posed by commercial systems, thereby establishing the necessary technical and commercial justification for our proposed solution In-Line Tablet Quality Control using Computer Vision.

2.2 The Limitations of Conventional Quality Control (QC)

Traditional tablet inspection systems fail to meet modern standards for manufacturing excellence and process analytical technology (PAT).

2.2.1 Destructive Testing and Sampling Risk

The industry standard often requires physical checks for attributes like dissolution rate, hardness, and disintegration time. While necessary, these are destructive tests and cannot be performed on every tablet. Defect checks such as for weight variation, spots, chipping, pinholes or capping are often relegated to human visual inspection or simple sampling systems. The central problem is that any quality system not providing **100%** inspection leaves a manufacturer vulnerable to product recalls and breaking regulations.

2.2.2 Process Variability and Defect Types

Manufacturing processes are subject to variability, leading to a range of defects that must be

instantly identifiable:

- **Geometric Defects:** Capping, lamination, chipping, and cracks, which compromise the tablet's structural integrity.
- **Surface Defects:** Spots, foreign particles, coating blemishes, or color variation (mottling).
- **Print Defects:** Incomplete, smudged, or misaligned product identification marks.

The detection of these defects, especially subtle flaws like pinholes, spots, capping, and chipping, requires a high-resolution, and real-time methodology.

2.3 Review of Commercial High-Speed Inspection Systems

International Original Equipment Manufacturers (OEMs) have developed sophisticated, high-throughput systems to address the need for automated inspection. These systems leverage complex optics, specialized lighting, and high-speed mechanics.

2.3.1 Proprietary Solutions from Industry Leaders

Major players in the high-speed vision inspection market include Viswill, Enclony, OPTEL, and Cognex that sell high-speed tablet inspection machines or components.

(i) Viswill (T/CVIS-NSR Series): Viswill is a European (Japan) company specializing in high-performance systems for the pharmaceutical industry. Viswill's T/CVIS-NSR is a high-speed automated tablet and capsule visual inspection system that performs 360° surface inspection using multi-camera vision. The system detects surface defects, colour deviations, shape irregularities, and embossing/printing errors while handling products through a vacuum-based conveying mechanism to ensure reproducible positioning. It operates at speeds of up to 300,000 tablets/hour and 125,000 capsules/hour, features a user-friendly graphical interface, and employs an active reject system with confirmation for fail-safe defect removal.

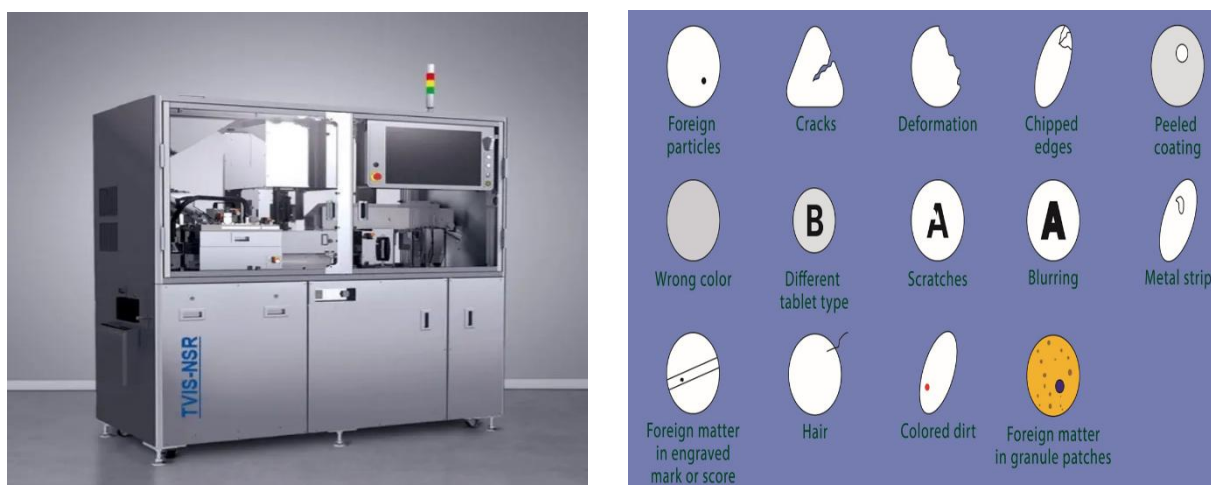


Figure 2.1 Viswill Automated Visual Inspection System

(ii) Enclony (PLANET Series):

Originating from Asia (Korea), Enclony stands as a leader in providing Automated Visual Inspection (AVI) solutions for solid oral dosage forms (SODFs). Their flagship PLANET Series is engineered for high-throughput pharmaceutical environments, specializing in the high-precision detection of surface, shape, and critical visual defects in tablets and capsules. A distinguishing feature of their technology is the achievement of "Zero Blind Spot" inspection,

accomplished through the utilization of a sophisticated multi-sensor fusion setup, which combines multiple high-resolution 2D and 3D cameras to inspect the entire surface area of the product. This robust system can sustain an output exceeding 350,000 tablets per hour, effectively identifying imperfections such as structural damage, color variations, coating blemishes, and printing or embossing defects. Enclony's systems are built with a strong emphasis on regulatory compliance and robust engineering, ensuring stable, reliable automated reject handling for efficient in-line quality control.

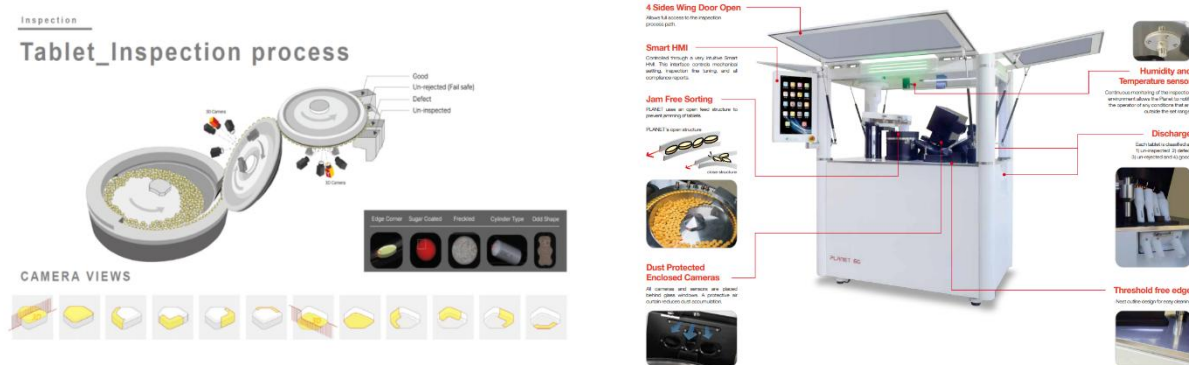


Figure 2.2: Type of Defects Covers by the Viswill's system

(iii) OPTEL & Cognex:

These companies specialize in providing scalable platform technologies rather than single-purpose machines. Their systems, which are often retrofitted onto existing packaging or counting lines, utilize high-resolution cameras and advanced pattern recognition algorithms.

- **Cognex** functions as a global supplier of industrial machine vision, providing high-resolution cameras (e.g., In-Sight series) and powerful software platforms, notably those incorporating Deep Learning (like In-Sight ViDi), to solve complex, subjective inspection tasks that traditional rule-based systems often fail to resolve. Their systems are frequently used to verify complex label placements, inspect variable cosmetic defects, and ensure package closure integrity.



Figure 2.3: Enclon Automated Visual Inspection System

- **OPTEL** also leverages Artificial Intelligence to provide extensive inspection features, often over 60 customizable options for both product and packaging integrity. Their

solutions, such as the TabletProof series and CountSafe, offer specific capabilities for detecting defects like wrong color, missing, broken, chipped, or undersized products during the counting and filling processes.

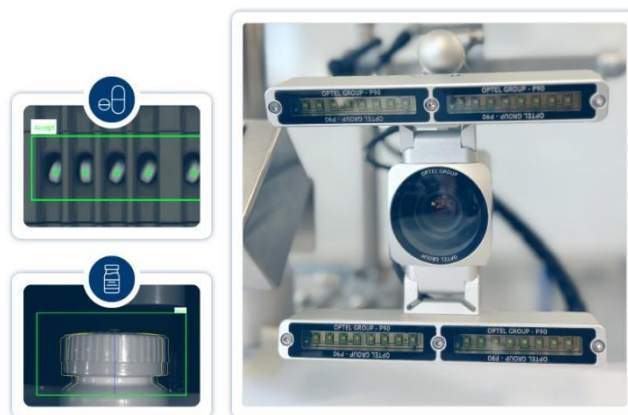


Figure 2.4: Tablet Inspection Process

2.3.2 Commercial Limitations and the Need for a Low-Cost Solution

While the high-throughput, multi-sensor Automated Visual Inspection (AVI) systems developed by global leaders and the integrated AI platforms set the technical performance standard, their deployment presents significant, often prohibitive barriers to smaller- and mid-sized pharmaceutical manufacturers, particularly within the local industrial landscape. The constraints faced by local manufacturers are multifaceted, encompassing financial, technical, and operational risks inherent in adopting foreign, specialized technology:

1. **Significant Capital Expenditure:** The most substantial hurdle is the direct cost. The initial investment required for these imported AVI systems typically runs into hundreds of thousands of US dollars for a single unit. This scale of capital expenditure is unsustainable for Small and Medium-sized Enterprises (SMEs) that constitute a large segment of the local market. Consequently, many manufacturers are economically compelled to continue relying on manual QC or outdated sampling methods, directly undermining the industry's mandate for 100% product safety and inspection.
2. **Proprietary Technology and Technical Rigidity:** Commercial systems are built upon proprietary "black box" software and algorithms. This model creates critical technical dependencies:

- **Obscured Logic:** The core defect classification logic is hidden, making error analysis, troubleshooting, and validation against local quality standards highly dependent on the OEM.
 - **Integration Friction:** Customization and seamless integration with existing local enterprise infrastructure, such as Manufacturing Execution Systems (MES), is technically challenging, requiring costly bespoke integration layers.
 - **Inflexibility:** The fixed nature of the software makes it rigid. Adapting the system to inspect a new tablet dimension, coating type, or a recently identified process-specific defect requires expensive, non-local vendor updates, hindering manufacturing agility.
3. **Maintenance Dependency and Operational Risk:** Reliance on foreign Original Equipment Manufacturers (OEMs) for specialized maintenance and spare parts creates unacceptable operational vulnerabilities. These machines use special parts which make them expensive to own and can cause long production stoppages when something breaks.

2.4 Novelty and Proposed Contribution

Our In-Line Tablet Quality Control using Computer Vision solution is inspired by the technical performance benchmarks set by these commercial systems, the goal of non-destructive, high-speed, 100% inspection yet strategically differentiates itself by overcoming the accessibility and flexibility barriers through technological innovation. The limitations of previous work are therefore identified not as insufficient accuracy (the commercial systems are highly accurate), but as insufficient technical and financial accessibility. The project's technical foundation is rooted in the advancements of modern Computer Vision and Deep Learning.

2.4.1 Transition from Rule-Based to Deep Learning Segmentation Systems

The core limitation of previous generations of automated inspection including pre-AI industrial systems was their dependency on rule-based machine vision. These algorithms required manual feature engineering e.g., detecting a defect based on specific pixel threshold or edge detection parameters, making them brittle and unable to handle complex, variable, or subtle cosmetic

defects precisely the types of defects e.g., pinholes, spots, capping, chipping

During the initial development stage of this project, a CNN based object detection pipeline built was selected due to its low inference latency and proven suitability for high-throughput inline inspection applications. At this stage, the primary objective was to establish reliable real-time defect localization maintaining compatibility with production-line constraints. YOLO-based detection enabled rapid identification of defective tablets by predicting bounding regions around anomalies, providing an effective baseline for automated quality assessment.

As the inspection requirements evolved, it became evident that conventional object detection introduced an important limitation: localization was restricted for determining defect presence and approximate position; this representation lacks the spatial precision required for pharmaceutical quality evaluation. In practice, tablet acceptance criteria are influenced not only

by whether a defect exists, but also by quantitative characteristics such as exact defect geometry, affected surface area, proximity to tablet edges, distribution across the surface, and overall defect severity. Bounding box outputs cannot capture these parameters and may overestimate or underestimate the actual defect extent.

To address these limitations, the inspection framework transitioned from conventional object detection to an **instance segmentation architecture**. This approach builds upon standard detection by employing pixel-level classification to delineate individual defects, replacing coarse bounding boxes with precise segmentation masks. Frameworks such as Detectron2, maskRCNN, SAM(Segment Anything Model), YOLO_seg achieves instance-level enables direct extraction of quantitative quality indicators from the segmented defect region, including exact defect area, defect-to-tablet surface ratio, shape descriptors, edge proximity measurements, clustering behaviour, and severity estimation. Consequently, the inspection system evolves from a defect localization tool into a quantitative quality assessment platform capable of supporting more explainable, reproducible, and manufacturing relevant decisions.

2.4.2 Architectural Advantages of the Proposed Solution

The technological pivot to a Deep Learning architecture yields two strategic advantages that eliminate the commercial barriers:

- Using significant augmentations, CNNs can be trained to concentrate solely on important flaws, a feature previously kept as a trade secret by proprietary systems, therefore ignoring variations like subtle color changes or illumination changes.
- Unlike object detection techniques that provide estimated bounding regions, instance segmentation lets one directly compute flaw area, geometric form, boundary proximity, defect density, and severity indicators, hence enabling objective quality evaluation instead of binary pass/fail decisions.
- Modern segmentation designs keep close real-time inference capacity and greatly increase localization accuracy, hence they are appropriate for ongoing manufacturing line deployment free from throughput loss.
- The project makes sure the software is clear, re-trainable, and configurable by building the detection core on the YOLO-family baseline and open-source frameworks (Python/PyTorch; hence removing the black box reliance on other OEMs).

2.5 Summary

This chapter established the critical need for our solution by performing a rigorous review of pharmaceutical Quality Control (QC) methods. The existing practice of random sampling is insufficient, necessitating a shift to 100% inspection. While commercial systems provide the technical benchmark for accuracy, their cost and proprietary nature create a market gap, leaving local manufacturers without accessible high-performance QC. Our project addresses this gap

by adopting Deep Learning on an open-source, cost-optimized platform. This methodological pivot allows us to achieve comparable commercial performance targeting $\geq 95\%$ accuracy in detecting critical defects i.e. pinholes, capping, chipping while ensuring technical transparency, local maintainability, and low Total Cost of Ownership (TCO). Thus, literature validates the strategic necessity of our final year design project as a localized, high-impact solution that democratizes access to advanced, safe, and sustainable manufacturing practices.

Chapter 3

System Design and Methodology

3.1 Introduction

This chapter presents the detailed design and methodology of the proposed in-line tablet quality control system. The system integrates a hardware-based image acquisition setup with a software-driven computer vision pipeline for automated tablet defect inspection. The complete workflow begins with raw data collection and progresses through data preparation, dataset management, model training, evaluation, and real-time inference using a user interface.

The pipeline is designed to support continuous inspection of tablet cores during manufacturing and to ensure accurate detection and localization of surface defects. The system flow closely follows industrial quality control practices and emphasizes reliability, traceability, and real-time performance.

3.2 Hardware System Design and Methodology

3.2.1 Tablet Image Acquisition Setup

The hardware system consists of a conveyor-based tablet transport mechanism integrated with an imaging unit. Tablets are placed on the conveyor with sufficient spacing to avoid overlap and ensure individual visibility. The conveyor speed is adjusted to maintain stable tablet movement while allowing clear image capture.

A high-resolution color camera is mounted above the conveyor to capture top-view images of tablet surfaces. Controlled illumination is provided using a diffuser lighting arrangement to reduce shadows, glare, and reflections. This lighting setup enhances surface texture details and tablet edges, which is essential for identifying defects such as scratches, spots, blur or smear, and edge chipping. You can see example in Figure 3.1.

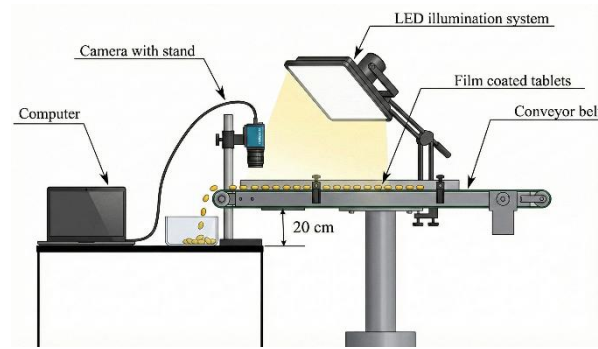


Figure 3.1 Hardware Setup

Camera exposure and focus parameters are configured to prevent motion blur and to ensure uniform image quality across the inspection zone.

3.2.2 Raw Data Collection Flow

Raw image data are collected through two sources. The first source includes images captured by the project team using the mobile phone and hardware setup under varying lighting conditions, tablet types, and orientations. The second source includes tablet images provided by a pharmaceutical expert to increase dataset diversity and realism.

All captured images are stored as raw images without initial filtering. This raw data collection stage ensures that the dataset reflects real manufacturing variations and defect appearances.

The hardware flow can be summarized as follows:

1. Tablets move along the conveyor.
2. Tablets enter the imaging zone.
3. The camera captures high-resolution images.
4. Images are transferred to the software pipeline for further processing.

3.3 Software System Design and Methodology

3.3.1 Initial Data Preparation & Preprocessing

The collected raw images undergo an initial data preparation stage. This includes systematic image renaming, optional resizing, and organization into class-wise folders representing defective and non-defective tablet categories. This step ensures consistency and ease of dataset management before annotation.

The raw dataset consisted of images of pharmaceutical tablets categorized into three classes: normal tablets, chipping defects, and capping defects. To standardize the dataset, all images were renamed using consistent prefixes such as “normal”, “chip”, and “cap,” followed by indexed numbering.

To improve the overall quality of the dataset, two major preprocessing techniques were applied. First, noise reduction was performed using bilateral filtering, which removes sensor noise while preserving important edge details. This is particularly important because defects such as chipping and capping are edge-based features that must remain intact.

```
def denoise(img):  
  
    return cv2.bilateralFilter(img, 9, 75, 75)
```

Second, lighting normalization was applied using CLAHE (Contrast Limited Adaptive Histogram Equalization), which corrects uneven illumination across images. CLAHE was applied on the L-channel of LAB color space, allowing brightness enhancement without significantly altering original color information.

```
def normalize_lighting(img):  
    lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)  
    L, A, B = cv2.split(lab)  
  
    clahe = cv2.createCLAHE(  
        clipLimit=1.2,  
        tileGridSize=(8,8))  
    L = clahe.apply(L)  
    lab = cv2.merge((L, A, B))  
  
    return cv2.cvtColor(lab, cv2.COLOR_LAB2BGR)
```

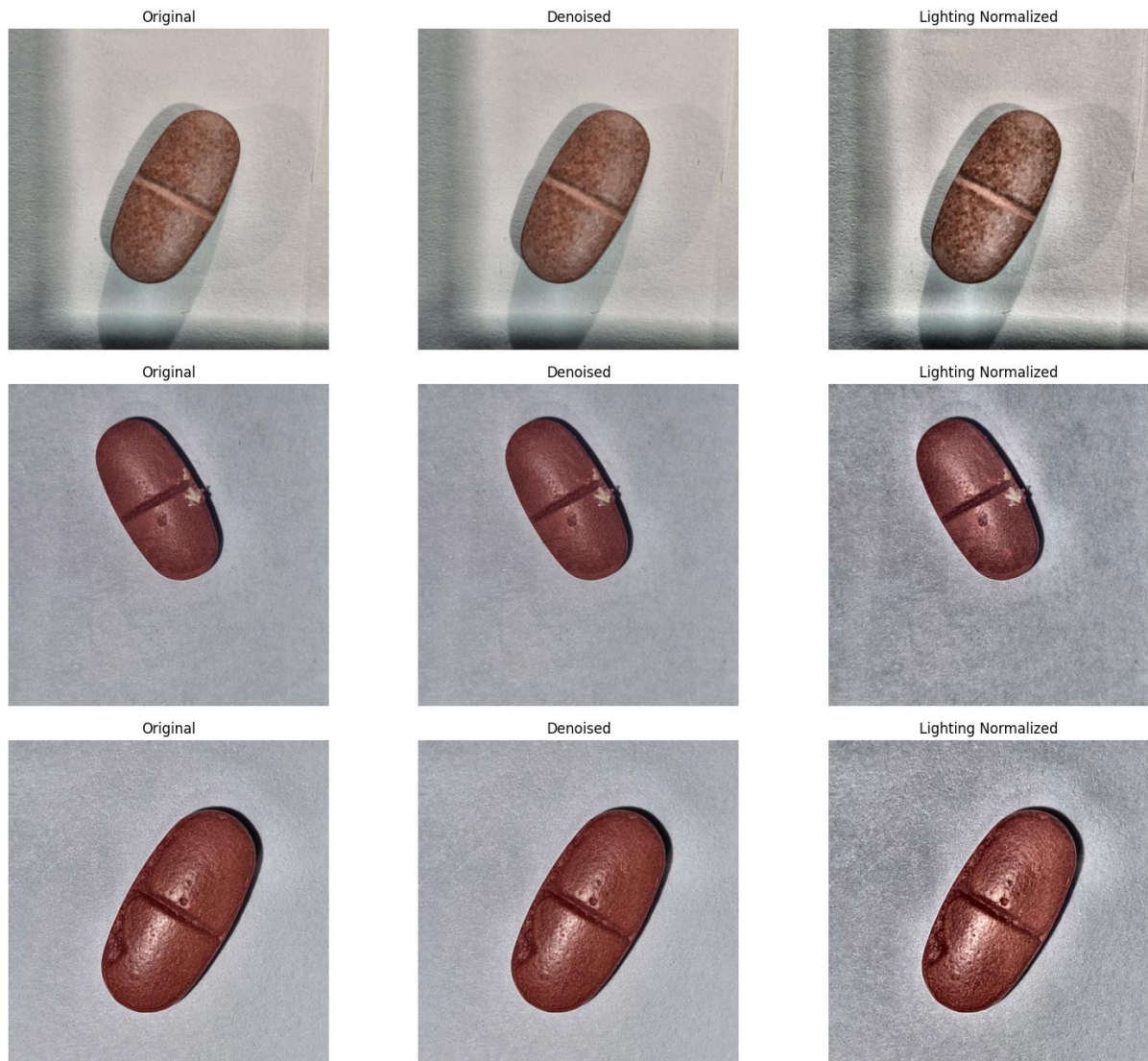



Figure 3.2: Visualization of preprocessed dataset

Image resizing was intentionally not performed during preprocessing. In many computer vision pipelines, images are resized to a fixed dimension for computational efficiency. However, for this study, maintaining the original resolution was considered more important than reducing image size.

3.3.2 Data Annotation and Dataset Management

To facilitate precise geometric and semantic learning, the dataset was annotated using a hierarchical, two-level instance segmentation strategy. This approach explicitly decouples the macro-geometry of the target object from its micro-surface anomalies. Annotations were formalized utilizing polygon-based masks to preserve high-fidelity boundary definitions, which are critical for accurate instance-level segmentation.

3.3.2.1 Two-Way Annotation Hierarchy

The annotation pipeline was executed in a structured, dual-stage sequence to establish a clear spatial hierarchy within the ground-truth data:

1. **Stage I: Primary Object Masking (Tablet Localization):** Every discrete tablet within the image frame was annotated as an individual polygon instance (T_i). This primary mask explicitly defines the spatial boundaries, morphology, and coordinates of the tablet, enabling the network to differentiate the primary objects from complex background noise and adjacent instances.
2. **Stage II: Secondary Defect Masking (Anomaly Segmentation):** For each localized tablet, localized surface anomalies (D_j) including structural chips, micro-cracks, fractures, and surface abrasions were annotated as distinct sub-polygon instances. These secondary masks are spatially bounded by the primary object mask ($D_j \subseteq T_i$). For structurally sound (normal) tablets, the secondary annotation stage was bypassed, yielding zero defect masks for that specific instance.

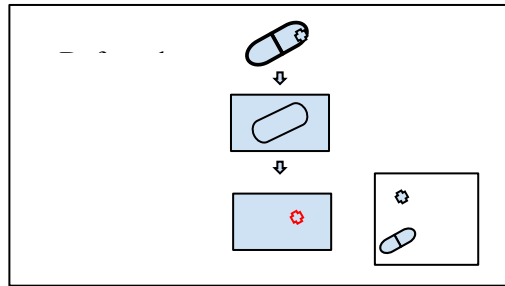


Figure 3.3: Two way Annotation Hierarchy

3.3.2.2 Annotation Environment and Implementation Details

The practical execution of the annotation strategy leveraged a hybrid workflow combining semi-automated foundation models with rigorous manual verification:

- **Annotation Platform:** The Computer Vision Annotation Tool (CVAT) was utilized as the primary integrated development environment for ground-truth generation.
- **Annotation Modality:** Polygon-based instance segmentation was strictly enforced to prevent the spatial quantization errors inherent to standard bounding-box object

detection.

- **Primary Mask Generation (Acceleration):** To optimize throughput and ensure boundary consistency, primary tablet masks were initialized using the Segment Anything Model 3 (SAM3). The model's zero-shot transfer capabilities provided rapid, high-fidelity boundary estimations.
- **Defect Mask Generation (Precision):** Due to the high high-frequency variance and minute scale of surface anomalies, defect masks were curated entirely via manual expert annotation. This ensured pixel-level precision where automated zero-shot models typically exhibit poor edge adherence.

To evaluate model generalizability across diverse operational environments, imagery was aggregated from three distinct source domains, each introducing unique statistical variations and training challenges:

- **Controlled Static Imagery (n=545):** This subset consists of high-resolution, individually framed tablets captured under standardized laboratory lighting conditions. Filenames were systematically structured by class (*chip_XXXX.jpg*, *cap_XXXX.jpg*, etc.). This domain represents a high-fidelity, low-noise baseline for feature extraction.
- **Dynamic Conveyor Video Frames (n=263):** Extracted from operational production lines, these frames contain multi-instance scenes (ranging from 4 to 7 tablets per frame). This subset introduces real-world deployment artifacts, including anisotropic motion blur, non-uniform illuminance, and severe geometric occlusions.

3.3.2.3 Video Frame Extraction and Field Annotation Protocol

3.3.2.3.1 Domain Adaptation Rationale

Initial iterations trained exclusively on the static dataset (n=545) achieved high accuracy on offline validation partitions but exhibited poor generalization when deployed onto live, dynamic conveyor systems. This performance degradation is attributed to a severe domain gap.

To mitigate this covariate shift, operational conveyor footage was acquired and integrated into the training pipeline, injecting critical environmental variances such as multi-instance spatial distributions, localized motion blur, luminance transitions, and peripheral truncation directly into the model's target distribution.

3.3.2.3.2 Video Acquisition and Temporal Subsampling

Continuous digital video streams were captured natively on the production line. Streams were acquired at a spatial resolution of 1920×1080 pixels at a temporal frequency of ≈ 30 frames per second (fps).

To minimize high temporal correlation and redundant gradient information between consecutive frames, a uniform down sampling stride of $\Delta f=10$ was enforced, yielding an effective temporal sampling rate of one frame per 0.33 seconds. This combinatorial pipeline initially yielded 532 candidate frames. Subsequent manual screening eliminated highly degraded or deeply occluded fragments, establishing a finalized core set of 263 conveyor frames.

3.3.2.4 Annotation Specification and Boundary Mapping

The curated conveyor frames were ingested into CVAT for semantic mapping under a strict, standardized annotation protocol:

1. **Sufficient Visibility Threshold:** Instance-level polygon masks were generated only for tablets exhibiting sufficient surface area to confidently diagnose structural integrity.
2. **Peripheral Edge Handling:** For tablets truncated by the image boundary, masks were fitted tightly along the visible perimeter. The class assignment was dictated exclusively by the observable morphology within the frame.
3. **Background Subsumption:** Micro-fragments or extreme edge occlusions incapable of deterministic classification were excluded from annotation, thereby designating them as background noise.

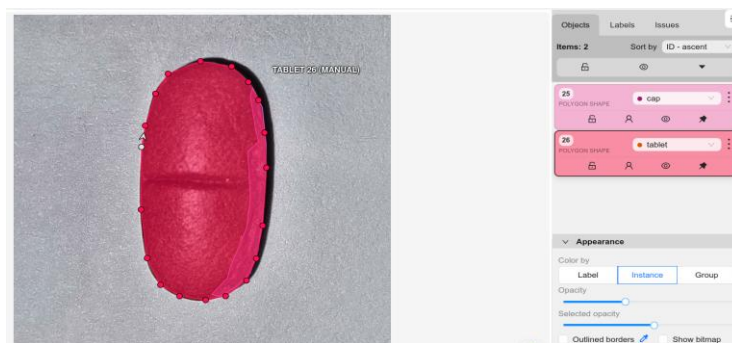


Figure 3.4: CVAT Annotation snapshot

3.3.3 Data Engineering and Curation Pipeline

A programmatic sanitization pipeline was implemented via a custom Python script designed to enforce strict parity between the image and ground-truth annotation directories.

Initial iterations utilized a shallow directory search that proved insufficient, failing to parse assets nested within sub-folder architectures. This limitation was resolved by replacing standard shallow path matching with an iterative, recursive file-system traversal (`Path.rglob`).

This mechanism successfully scanned all nested sub-directories, verifying file existence and isolating orphan assets. Post-execution, unmapped images lacking corresponding label text files were purged, yielding a finalized, verified corpus of 606 fully matched image-annotation pairs.

3.3.4 Dataset Partitioning and Stratification Strategy

3.3.4.1 Initial Partitioning Design

A targeted data-splitting topology was established based on filename pattern-matching semantics. The high-fidelity, baseline laboratory cohort (n=459) was partitioned using a stratified split allocation of 70% training, 20% validation, and 10% testing. Class stratification was strictly enforced during this step to maintain proportional representation of all four morphological categories across each sub-partition.

Initially, to preserve the statistical purity of the evaluation sets, all potentially noisy or out-of-domain assets of the operational conveyor video frames (n=263) were isolated exclusively within the training pool. This choice prevented sub-optimal annotation heuristics or domain-specific anomalies from contaminating the validation and testing baselines.

3.3.4.2 Evaluation Domain Adaptation

In a subsequent optimization step, the partitioning strategy was refined to align validation metrics more closely with the operational realities of the deployment environment. A subset of 44 frames captured from the live conveyor line was extracted from the training partition and transferred to the testing partition.

By inserting dynamic production assets directly into the validation and testing loops, the

evaluation framework was forced to account for active real-world variables, such as variable motion blur and multi-instance occlusion. This alteration transformed the validation metrics into accurate predictors of field performance on the active production line. The quantitative distribution of the finalized dataset splits is codified in Table

Table3.1 : Finalized dataset partition and composition matrix

Partition	Image Count	Structural Composition
Training	565	70% Stratified laboratory baseline and remaining operational video frames
Validation	162	20% Stratified laboratory baseline and duplicated conveyor frame testing subset
Testing	81	10% Stratified laboratory baseline and 44 unique operational conveyor frames

3.3.5 Class Instance Distribution and Imbalance Analysis

Because instance segmentation frameworks optimize on an object-by-object basis, the training pool's evaluation metrics depend on individual instance density rather than raw frame count. Due to the multi-instance nature of the production conveyor frames (which exhibit local spatial densities of 4 to 7 tablets per frame), the aggregate instance tally significantly exceeds the baseline image count. The specific instance distribution across the four semantic classes within the training partition is structured in Table 3.4.

Class Label	Class Identifier	Total Training Instances
Tablet	0	484
Chip	1	261
Cap	2	245
Normal	3	270

Table 3.2: Semantic class instance distribution within the training partition.

3.3.6 Dataset Export and Model Training

After annotation and preprocessing, the dataset is exported in a format suitable for model training, including images, labels, and a configuration file defining dataset structure.

Model training is performed using a computer vision instance segmentation architecture from the COCO and YOLO-SEG family. Training is conducted using a cloud-based computational environment to support efficient processing. Model parameters such as input image size, batch size, and number of training iterations are tuned to optimize detection performance.

3.3.5 Model Evaluation

The trained model is evaluated using standard performance metrics, including precision, recall, and per-class analysis. Evaluation results are analyzed to assess the model's ability to correctly detect and localize different types of tablet defects. This step ensures that the model meets the accuracy and reliability requirements for real-time quality control.

3.3.6 Model Inference and User Interface

The final trained model is integrated into a real-time inference pipeline. Test images are processed through the system to perform defect detection and localization. Detected defects are visualized with bounding boxes and class labels.

A graphical user interface is developed to display inspection results, including detected defects, and visual feedback for operators. This interface supports quality monitoring and provides insights into tablet condition during inspection.

3.4 Summary

The complete system workflow follows a structured sequence: raw data collection, data preparation, annotation and dataset management, dataset export, model training, model

evaluation, and real-time inference with visualization. The integration of hardware and software components enables continuous inspection of tablets, early defect identification, and reduction of manual quality control efforts.

Chapter 4

Model Development and Training

4.1 Introduction

This chapter describes the development and training process of computer vision-based models for tablet defect detection. The objective is to construct a reliable real-time inspection system capable of detecting and localizing surface defects on tablet cores with high accuracy and robustness. Model development is carried out in a structured and progressive manner, where performance improvements are analyzed across multiple training phases.

In the context of model design, the problem can be addressed using three general methodological approaches. The first approach is based on classical computer vision techniques, which rely on mathematical and rule-based formulations for feature extraction and defect identification. The second approach utilizes deep learning-based models, which learn hierarchical feature representations directly from data for automated defect detection and localization. The third approach is a hybrid strategy, which combines classical image processing techniques with deep learning models to leverage the strengths of both paradigms.

In this study, multiple deep learning architectures are explored for performance evaluation. Specifically, three different model families are considered, namely Mask R-CNN, YOLO-SEG, and Segment Anything Model (SAM)-based approaches, resulting in a total of nine model configurations. A detailed discussion of the individual models and their comparative performance is provided in the subsequent sections. Similarly, classical and hybrid approaches are also considered for comparative analysis, which will be elaborated later in this chapter.

4.1.1 Progressive Model Training Phases

Model training was conducted in four distinct phases. In the initial phase, a very small and highly imbalanced dataset was used to validate the feasibility of the training pipeline. Tablet-level bounding boxes were applied to all images. Although high mean average precision values were observed for the normal class, the model showed poor performance in detecting defect classes and exhibited strong bias toward normal tablets. This phase demonstrated that high numerical accuracy does not guarantee meaningful defect learning.

In the second phase, the dataset size was increased and image quality was improved through automatic orientation correction, resizing, and contrast enhancement. Tablet-level annotations were retained to isolate the effect of preprocessing. Training stability improved significantly, and mean average precision values increased; however, defect localization remained weak. The results indicated that preprocessing improves stability but does not resolve limitations caused by inappropriate annotation strategies.

The third phase focused on evaluating annotation strategies using a controlled experimental setup. Mixed, tablet-only, and defect-only annotation approaches were compared while keeping the dataset size constant. The defect-only annotation strategy, where bounding boxes were drawn only around visible defects and normal tablets contained no labels, produced the most reliable localization results. The model achieved accurate defect detection with minimal false positives, highlighting the importance of annotation consistency and semantic clarity.

In the final phase, multiple tablet datasets were combined to increase diversity in tablet shape, color, and surface texture. The defect-only annotation strategy was applied consistently across all data. The final model demonstrated improved generalization and balanced performance across defect classes, achieving satisfactory mean average precision values for both edge chipping and surface defects.

You can view the comparison in Figure 4.1.

Table 4.1 Model Comparison Table

Phase	Dataset Characteristics	Annotation Strategy	Main Objective	Key Results	Key Learning
Phase 1: Initial Models	Very small dataset (tens of images); severe class imbalance (normal $\gg$ chip, cap)	Tablet-level bounding boxes for all classes (Strategy 2)	Validate YOLO pipeline and feasibility of defect detection	High mAP@0.5 for normal class; poor chip detection; strong	High mAP does not imply true defect learning; small datasets

				normal-class bias	encourage biased predictions
Phase 2: Dataset Growth & Preprocessing	Increased dataset size (normal_chip v2–v4); improved image quality via auto-orientation, resizing (512×512), contrast enhancement	Tablet-level bounding boxes maintained (Strategy 2)	Improve training stability and reduce noise	Stable training; mAP@0.5 ≈ 0.95–0.99; defect localization still weak	Preprocessing improves stability, but tablet-level labels hide defect-level behavior
Phase 3: Annotation Strategy Study	Same dataset size and preprocessing as Phase 2 (controlled experiment)	Compared mixed boxes, tablet-only boxes, and defect-only boxes with empty labels (Strategy 3)	Identify annotation strategy suitable for defect localization	Defect-only strategy achieved mAP@0.5 ≈ 0.85 with correct localization and no false positives	Annotation consistency is critical; localization-focused labels yield practical detection

Phase 4: Final Integrated Model	Combined multiple tablet datasets; increased diversity in shape, color, and tablet type	Defect-only bounding boxes with empty labels for normal tablets (Strategy 3)	Improve generalization and robustness	Chip mAP@0.5 $\approx$ 0.7; Cap mAP@0.5 $\approx$ 0.9; Overall mAP@0.5 $\approx$ 0.8	Dataset diversity improves generalization only when annotation semantics remain consistent
--	---	--	---------------------------------------	--	--

4.1.2 Two Stage Pipeline

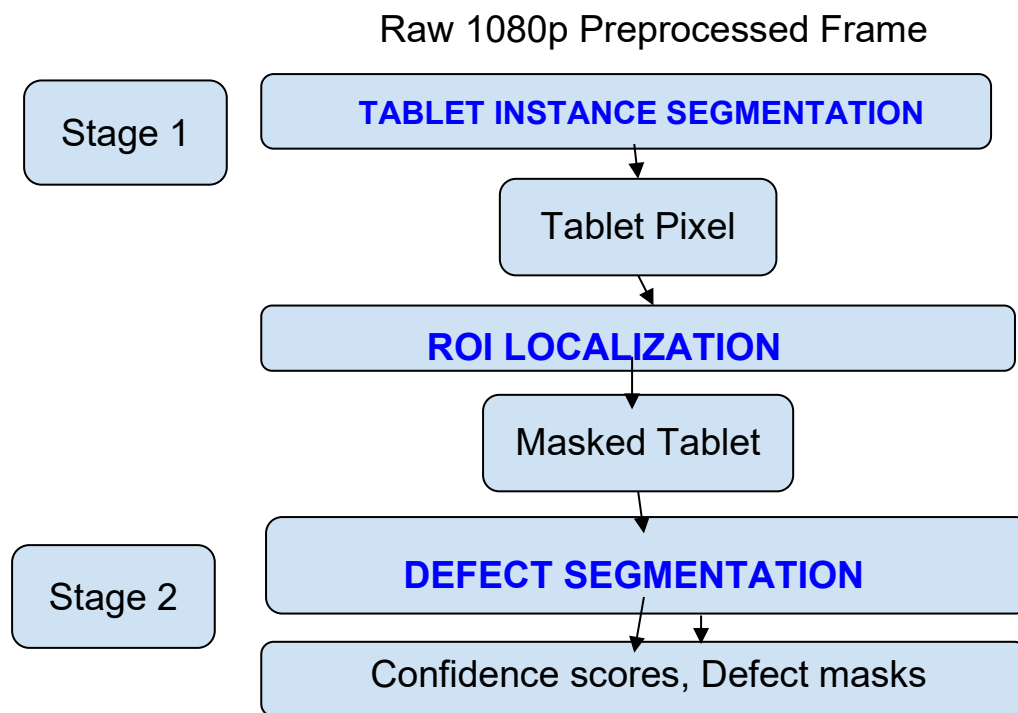


Figure 4.1: Two-Stage pipeline for segmentation

4.2 Deep Learning Based Models

4.2.1 YOLO Architecture

To evaluate the performance, generalization capability, and training stability of the real-time defect segmentation system, three distinct training configurations were engineered using the YOLO11n-seg (Nano Segmentation) architecture. The objective of this multi-run strategy is to analyze the trade-offs between a standard baseline, heavy data augmentation, and targeted architectural regularization.

4.2.1.1 Stage 1 Models

4.2.1.1.1 Run 1: Baseline Standard Resolution Setup

The initial baseline configuration (Configuration A) was engineered to evaluate full-network convergence at a standard spatial resolution of 640×640 pixels. Optimization was driven by stochastic gradient descent (SGD) with a momentum coefficient of 0.937, an initial learning rate (lr_0) of 0.01, and a weight decay threshold of 0.0005 to suppress early parameter divergence.

To expand the variation in defect representation, aggressive pixel-level and spatial augmentations were implemented. This included a high Mosaic augmentation coefficient (1.0), instance Copy-Paste blending (0.7), and extensive Hue-Saturation-Value (HSV) jittering. Training was bounded to 200 epochs with an early stopping patience window of 40 epochs to prevent computational redundancy.

4.2.1.1.2 Run 2: High-Resolution Multi-Scale Transfer Learning

Due to the exponential increase in memory overhead associated with a 1280px input, the batch size was micro-throttled to 2 samples per iteration to avoid hardware out-of-memory (OOM) faults. The augmentation pipeline and optimization coefficients were kept identical to Configuration A to strictly isolate the operational impact of high-resolution spatial scaling on mask intersection-over-union (IoU) metrics.

4.2.1.1.3 Run 3: Rapid-Throughput Downscaled Optimization

The optimization engine was switched from SGD to AdamW with a conservative learning rate of 0.001. Given the reduced spatial resolution, the augmentation policy was heavily restricted reducing Mosaic interaction to 0.1 and minimizing affine translations to prevent

small defect instances from being obliterated during down sampling. Furthermore, Automatic Mixed Precision (AMP) training and RAM data caching were enabled to maximize hardware

Computational Parameter	Baseline	High-Res Shift	Rapid Edge
Input Resolution (imgsz)	640 x 640 px	1280 x 1280 px	416 x 416 px
Optimizer Engine	SGD	SGD	AdamW
Initial Learning Rate (lr0)	0.01	0.01	0.001
Batch Size	8	2	32
Total Target Epochs	200 (Patience = 40)	200 (Patience = 40)	80
Mosaic Activation	1	1	0.1

utilization and achieve rapid epoch completion cycles across a constrained 80-epoch horizon

Table 4.2: Hyperparameter and Augmentation Mapping for Stage-1 Initializations

Model ID	Model	mAP50	mAP75	mIOU	Latency(ms)
1.1 Baseline	YOLOv11-seg Baseline	0.8199	0.5626	0.6096	24.01
1.2 High Res Shift	YOLOv1-seg Full-finetune	0.8302	0.5327	0.6212	73.57
1.3 Rapid Edge	YOLOv1-seg Regularization	0.8555	0.7868	0.7392	5.74

Model 1.3 (Rapid Edge) is unequivocally the best model, achieving the highest segmentation performance across all primary metrics (85.55% mAP50, 0.7392 mIOU) while delivering the lowest operational latency (5.74 ms)

4.2.1.2 Stage 2 Models

4.2.1.2.1 Run 1: Baseline Segmentation Setup (Balanced Training)

Table 4.3: Comparative Analysis of Stage-1 Model Performance and Inference Latency

Run 1 sets the experimental baseline using the complete, unconstrained YOLO11n-seg network design. Using the standard Ultralytics step-decay plan, this setup uses the AdamW optimizer with an initial learning rate (lr0) of 0.0008. A strong batch size of 16 guarantees steady gradient

estimates.

The setup helps to give exact boundary definition top priority during early convergence by letting overlapping masks (overlap mask=True) and a mask down sampling ratio of two. Data augmentation is modest; it uses basic hue-saturation-value (HSV) jitter, micro-rotations ($\pm 5^\circ$), and a small mosaic augmentation coefficient of 0.2.

4.2.1.2.2 Run 2: Augmented Heavy Learning with General Defect Segmentation

Run 2 aims to maximize the model's resilience against natural surroundings and geometric changes found in actual surface flaws (surface chips, structural caps, and subtle edge abnormalities). To fit a bigger spatial resolution dataset (imgsz=640), the architecture is still fully trainable, but the batch size is lowered to 8.

Using the inherent stochastic gradient descent (SGD) auto-scheduling, this setup mostly depends on sophisticated data augmentation instead of hyperparameter tweaking. Aggressive spatial transformations, including a high mosaic probability (0.5), instance copy-paste augmentation (0.3), random horizontal/vertical flips, and multi-scale scaling, drive generalization.

4.2.1.2.3 Run 3: Regularized & Stability-Focused Model (Advanced Stage-2)

Run 3 introduces a highly regularized training pipeline designed to mitigate overfitting and ensure production-level stability. Unlike the previous runs, a partial backbone freezing strategy is implemented; low-level feature extraction layers are locked to retain stable, pre-trained structural primitives, forcing the optimization process to focus entirely on the downstream detection and segmentation heads.

The configuration uses the AdamW optimizer paired with a Cosine learning rate decay schedule. To counter the noisy gradients introduced by a restrictive batch size of 4, a dropout rate of 0.3 is injected into the network heads. This explicitly reduces feature co-adaptation and forces the model to learn redundant, robust feature representations for edge-case defect scenarios.

Table 4.4 : Architectural and Hyperparameter Comparison of Stage-2 YOLO11n-seg

Hyperparameter	Run 1 (Baseline)	Run 2 (Augmented)	Run 3 (Regularized)
Base Architecture	YOLO11n-seg	YOLO11n-seg	YOLO11n-seg
Optimizer	AdamW	SGD	AdamW
Learning Rate Schedule	Step Decay (lr0=0.0008)	Auto Schedule	Cosine Decay
Batch Size	16	8	4
Backbone Freezing	None	None	Partial Backbone Frozen
Dropout Rate	0.0	0.0	0.3
Mosaic Coefficient	0.2	0.5	0.0
Copy-Paste Probability	0.0	0.3	0.0

Table 4.5 : Comparative Analysis of Stage-2 Model Performance and Inference Latency

Model ID	Model	mAP50	AP75	mIOU	Latency(ms)
1.2 Baseline	YOLO11n-seg	0.664	0.6423	0.428	10.57
1.1 Augmented	YOLO11n-seg	0.9628	0.8049	0.7582	19.29
1.3 Regularized	YOLO11n-seg	0.8795	0.7654	0.5989	13.65

Model 1.1 (Augmented) is the top-performing model, delivering the highest overall accuracy and boundary precision (96.28% mAP50, 0.7582 mIOU) while maintaining a highly acceptable real-time operational latency of 19.29 ms.

4.2.1.3 Learning Curves

4.2.1.3.1 Model ID : 1.3 Rapid Edge Learning Curves (1st Stage)

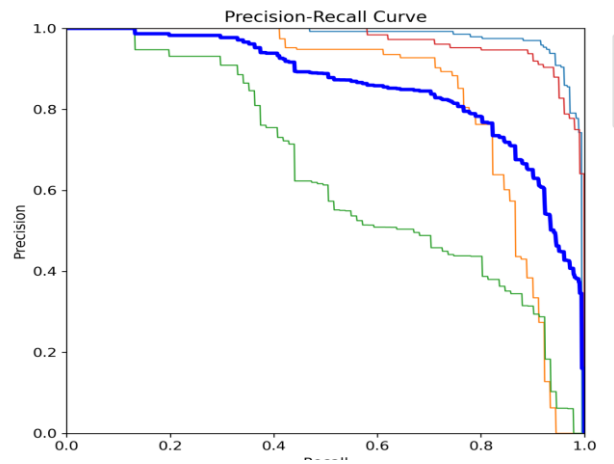


Figure 4.2: Mask PR Curve of Rapid Edge (Yolo-seg 1st Stage)

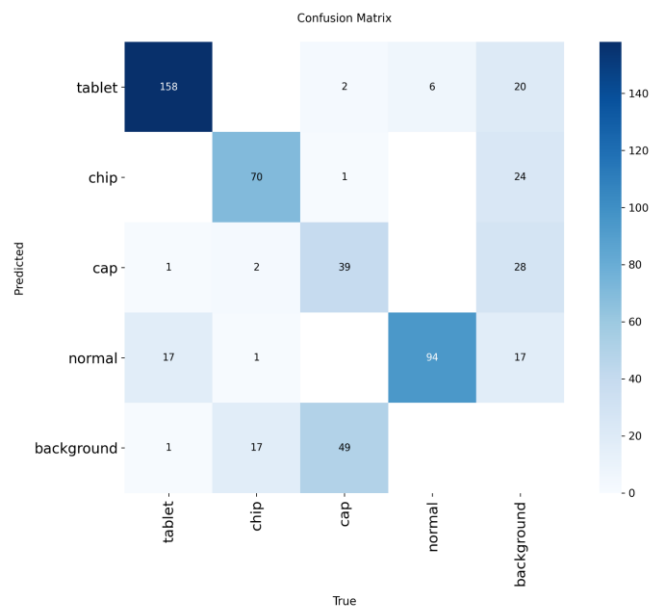


Figure 4.3: Confusion matrix of Rapid Edge (Yolo-seg 1st Stage)

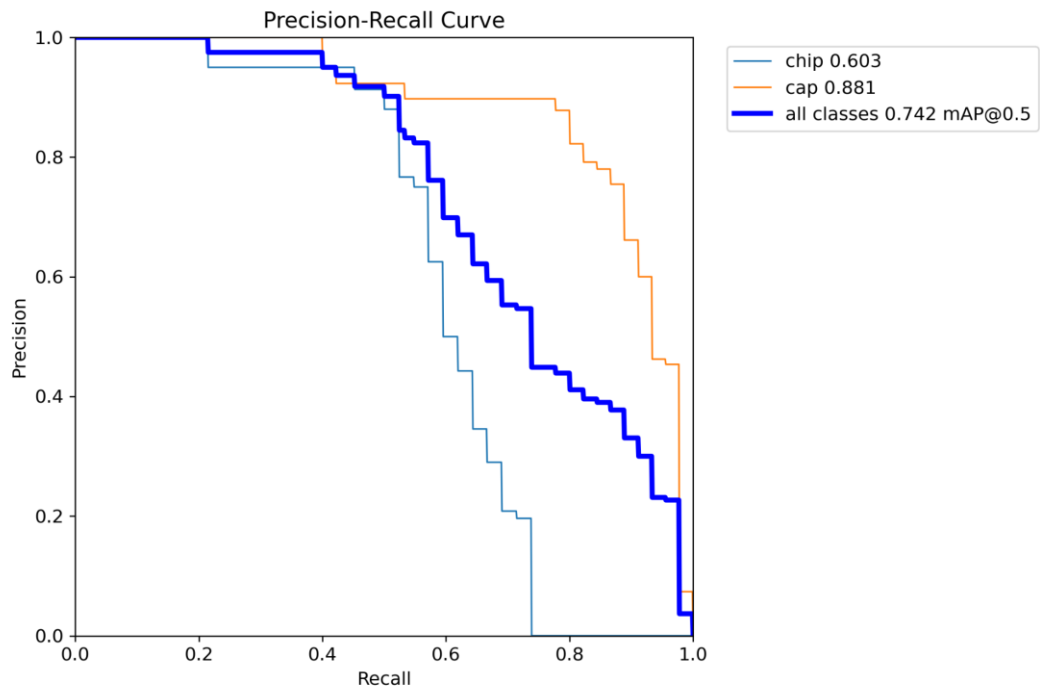


Figure 4.4 : Mask PR Curve of 1.1 Augmented (Yolo-seg 2nd Stage)

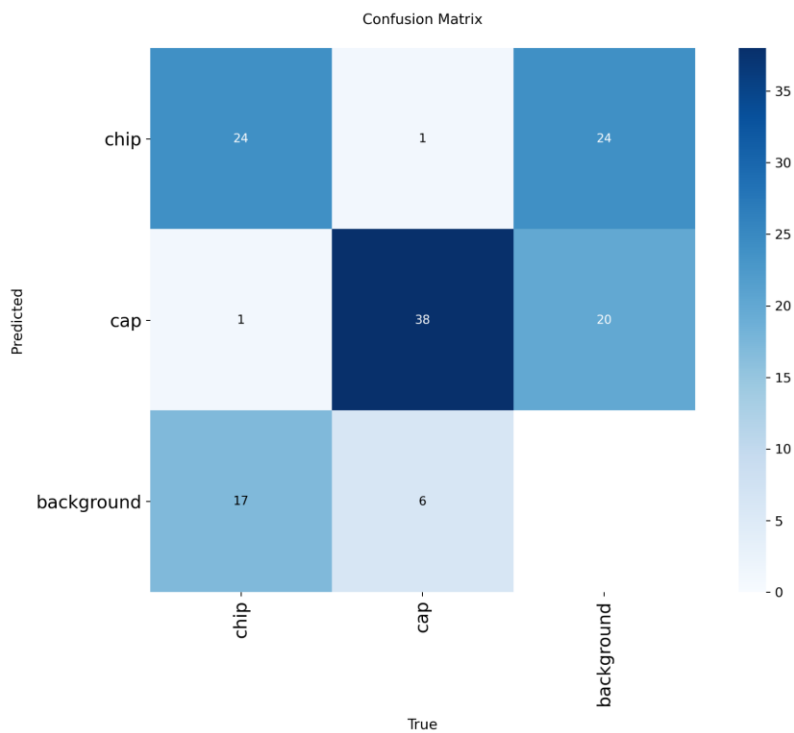


Figure 4.5 : Confusion matrix of 1.1 Augmented (Yolo-seg 2nd Stage)

4.2.1.3 Conclusion

The iterative training process highlights that dataset quality and annotation strategy play a more critical role than dataset size alone. Defect-focused annotations enabled the model to learn meaningful defect features and achieve practical defect localization. The final trained model provides a robust foundation for real-time in-line tablet quality control using computer vision.

4.2.2 Mask RCNN Architecture

To benchmark the single-stage YOLO models against a established topological baseline, a series of experiments were conducted using the two-stage Mask R-CNN framework. Mask R-CNN extends Faster R-CNN by parallelizing a pixel-level mask prediction branch

All experimental runs utilize a shared ResNet-50 paired with a Feature Pyramid Network (FPN) backbone, initialized with pre-trained weights from the MS COCO dataset. 4.2.2.1 Experimental Run Definitions

4.2.2.1.1 Run 1: Stochastic Gradient Descent Baseline (Standard Reference)

Run 1 establishes the structural and optimization baseline for the two-stage pipeline. The model was optimized using standard Stochastic Gradient Descent (SGD) with a conservative learning rate ($lr=0.00025$) and a batch size of 2. The Region of Interest (ROI) heads were configured with a batch size of 128 samples per image to ensure dense region proposal exploration. This unconstrained, full-network fine-tuning setup acts as the primary benchmark to gauge the raw capacity of the network without external regularization or learning rate alterations.

4.2.2.1.2 Run 2: Layer Regularization and Adaptive Optimization (AdamW & Partial Freezing)

Run 2 uses the AdamW (Adam with Decoupled Weight Decay) optimizer at a learning rate of 0.0001 to change the optimizing dynamics by replacing SGD. An architectural restriction was added to help the model to prevent overfitting on particular fault characteristics and to inspire the reuse of low-level structural components: The ResNet-50 backbone was somewhat frozen up to Stage 2. The ROI batch size was also lowered to 64 to ensure more sparse, better-quality region recommendations. This design locks general edge-detecting weights and lets downstream heads specialize to help to generalize.

4.2.2.1.3 Run 3: Non-Linear Learning Rate Decay (SGD Cosine Annealing)

Run 3 explores the impact of non-linear optimization scheduling on model convergence. While maintaining an SGD backbone, this configuration introduces a dynamic Cosine Annealing learning rate schedule initiated by a brief warm-up period. The warm-up phase stabilizes early gradient vectors, while the subsequent cosine decay smoothly downscales the learning rate to prevent late-stage oscillations around local minima. Due to the computational overhead of the schedule tracking, the training batch size was restricted to 1.

Table 4.6: Architectural and Hyperparameter Comparison of Mask RCNN

Model ID	Optimizer	Learning Rate	Batch Size	ROI Batch Size	Scheduler	Freezing Strategy
Run1_SGD	SGD	0.00025	2	128	None	No freezing
Run2_AdamW	AdamW	0.0001	2	64	None	Backbone frozen
Run3_CosineSGD	SGD Momentu m	Variable	1	128	Warm-up & Cosine Annealing	No freezing

Table 4.7: Comparative Analysis of Mask RCNN Performance and Inference Latency

Model ID	Model	mAP50	AP75	mIOU	Latency(ms)
Run1_SGD	SGD Baseline	0.8536	0.5894	0.9665	226.7
Run2_AdamW	AdamW + Partial Freezing	0.845	0.569	0.9663	217.8
Run3_cosine	SGD + Cosine Scheduler	0.8407	0.5818	0.9676	224.5

Run 3 (SGD + Cosine Scheduler) emerges as the most balanced and optimized configuration for production deployment, despite Run 1 holding a marginal lead in detection accuracy.

The Selection Logic: While Run 1 yields the highest bounding box precision (mAP50=0.8536), Run 3 delivers the highest pixel-level segmentation accuracy with a Mean Intersection over Union (mIoU) of 0.9676.

4.2.2.1.3 Learning Curves

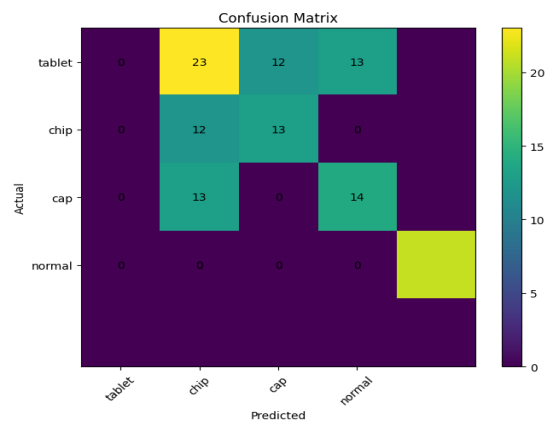


Figure 4.6: Confusion matrix of Run3_SGDCosine (Mask RCNN) best Model

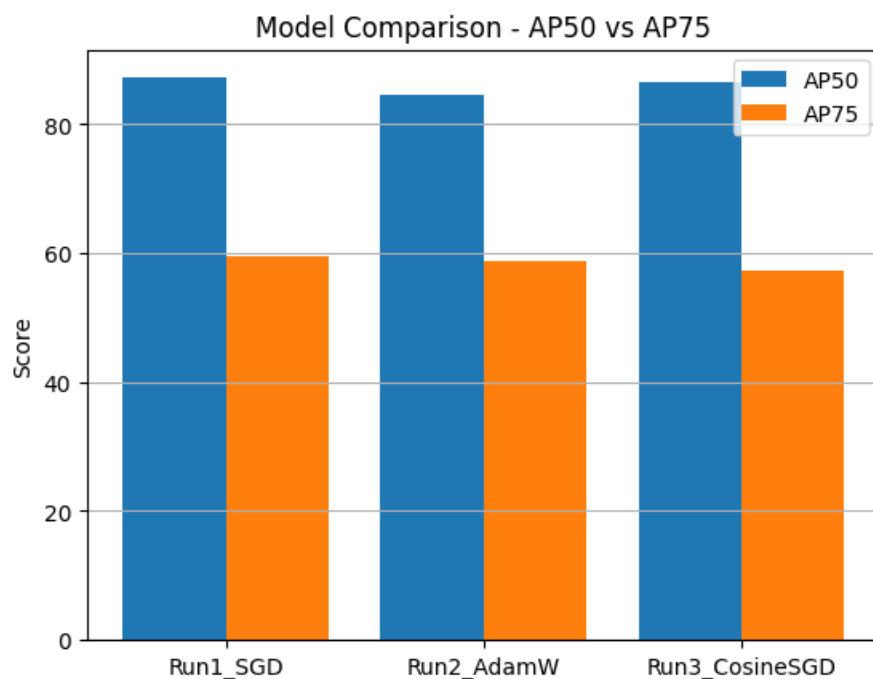


Figure 4.7 : Model Comparison of MaskRCNN architectures

4.2.3 Cross-Architectural Analysis: YOLO11n-seg vs. Mask R-CNN

A critical objective of this study is evaluating the operational trade-offs between modern single-stage networks (**YOLO11n-seg**) and two-stage frameworks (**Mask R-CNN**). This comparison represents a classic architectural paradigm: single-stage direct regression versus two-stage region-proposal alignment.

4.2.3.1 Architectural Paradigm and Feature Mapping

YOLO11n-seg processes the entire input image in a single forward pass, predicting bounding boxes and semantic segmentation masks simultaneously. This ultra-lightweight approach minimizes feature redundancy and optimizes mathematical throughput.

Conversely, Mask R-CNN utilizes a Region Proposal Network (RPN) to first isolate candidate regions of interest (ROIs). A secondary alignment layer (RoIAlign) then extracts localized feature patches, mapping them independently to dedicated classification and mask generation heads.

4.2.3.2 Quantitative Performance Synthesis

To evaluate these structural paradigms on the industrial defect dataset, the top-performing variants from each model class were benchmarked against one another. The cross-architectural performance matrix is detailed in Table 4.5.

Table 4.8: Cross-Architectural Comparison Matrix (YOLO11n-seg vs. Mask R-CNN)

Evaluation Metric	YOLO11n-seg (Stage 2: Run 1.1 Augmented)	Mask R-CNN (Run 3 Cosine)	Performance (Δ)
Detection Accuracy (mAP50)	0.9628	0.8407	+0.1221 (YOLO)
Segmentation Quality (mIoU)	0.7582	0.9676	+0.2094 (Mask R-CNN)
Inference Latency (ms)	23.29 ms	224.50 ms	-201.21 ms (YOLO)
Operational Frame	42.9 FPS (Real-	4.4 FPS (Batch	

Rate	Time)	Only)	
------	-------	-------	--

4.3 Model Development Using Mathematical Vision-Based Defect Detection

To investigate alternatives to deep learning–based inspection for pharmaceutical tablet quality control, three mathematical computer vision approaches were developed and evaluated for detecting tablet defects under real-time constraints.

The experiments were conducted on tablet videos captured in motion, where tablets continuously moved across a conveyor surface.

In pharmaceutical manufacturing, physical integrity anomalies like cracking, chipping, and capping must be intercepted immediately. Automated visual inspection systems frequently rely on Deep Learning. However, these systems demand vast annotated datasets and expensive specialized processing units (GPUs).

Computer vision, built on morphological math and localized gradient profiles, provides a highly predictable, interpretable alternative that can run directly on resource-constrained edge hardware. This paper analyzes three variations of an inspection pipeline designed to track, isolate, and label defective tablets from a video stream.

4.3.1 Shared Core Pipeline Components

While each variation implements distinct architectural differences for isolating anomalies, they all share a common structural foundation:

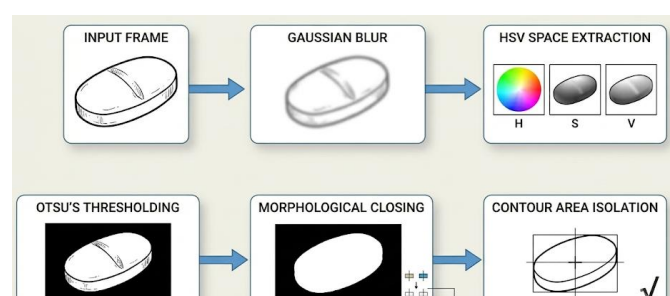


Figure 4.8: Pipeline for classical approach to tablet contour localization

4.3.1.1 Spatial Filtering & Color-Space Invariance

Across all implementations, input RGB frames are initially subjected to spatial smoothing via a 2D Gaussian Kernel to reduce high-frequency imaging noise:

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

Following spatial attenuation, the frame is transformed into the HSV (Hue, Saturation, Value) color space. The illumination component is decoupled from chromatic information by isolating the Value (V) channel (`hsv[:, :, 2]`). This channel maximizes contrast between the background conveyor surface and the tablet foreground, minimizing sensitivity to minor color shifts.

4.3.1.2 Global Adaptive Segmentation and Morphological Refinement

To isolate the tablet matrix, the system applies Otsu's Thresholding to the extracted V-channel histogram. This technique treats the image as a bimodal distribution of foreground and background pixels, automatically calculating an optimal threshold value t^* that minimizes the intra-class variance $\sigma_w^2(t)$:

$$\sigma_w^2(t) = \omega_0(t)\sigma_0^2(t) + \omega_1(t)\sigma_1^2(t)$$

Where ω_0 and ω_1 denote the probabilities of the two classes separated by a threshold t , and σ_0^2 , σ_1^2 represent their respective variances.

To resolve internal shadows, specular reflections, or tiny surface pores, a morphological

Closing operation is
isotropic structuring

performed using an
element (K):

$$A \bullet K = (A \oplus K) \ominus K$$

This mathematical dilation ($\oplus$) fills internal gaps before an erosion ($\ominus$) restores the external boundaries. The largest external contour exceeding a defined minimum spatial area (MIN_TABLET_AREA) is extracted as the primary tablet target matrix, while any background conveyor artifacts are discarded.

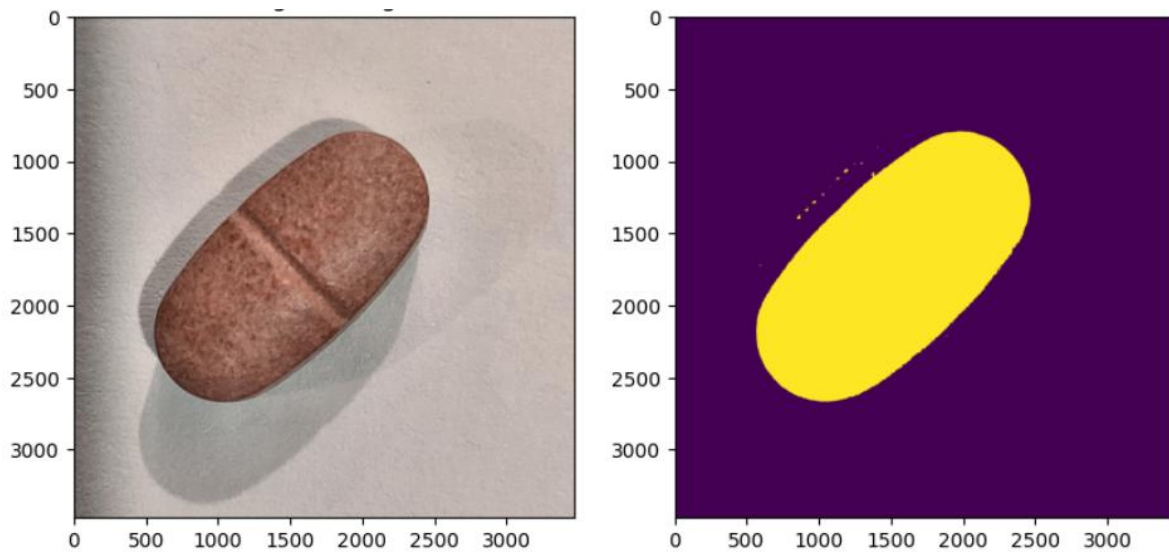


Figure 4.9 : Tablet localization using Adaptive Segmentation and Morphological Refinement

4.3.2 Methodology & Core Algorithmic Variations

4.3.2.1 Variation 1: Geometric Deviation and Global Gradient Profiling

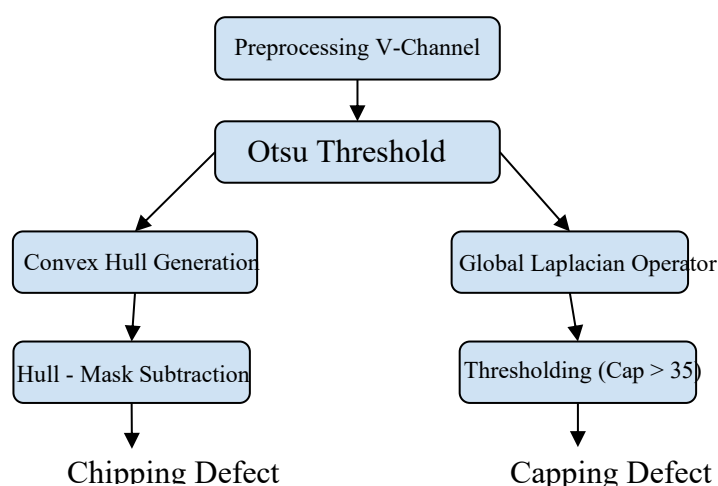


Figure 4.10: Geometric Deviation and Global Gradient Profiling

Variation 1 models tablet integrity through a global geometric framework. It assumes that a

healthy tablet forms a perfectly smooth convex shape, and that capping produces high-frequency surface variations across the entire object.

4.3.2.1.1 Chipping Detection via Convex Hull Subtraction

Let the segmented tablet contour be defined as a set of points X . The **Convex Hull** $H(X)$ is defined as the minimal convex set containing X :

$$\mathcal{H}(X) = \left\{ \sum_{i=1}^k \alpha_i x_i \mid x_i \in X, \alpha_i \geq 0, \sum_{i=1}^k \alpha_i = 1 \right\}$$

The system renders $H(X)$ as a solid binary mask and applies an array subtraction against the empirical tablet mask (M):

$$\text{Chip Mask} = \mathcal{H}(M) \setminus M$$

Any clustered residual pixel area exceeding `CHIP_THRESHOLD` indicates a peripheral structural indentation, classifying the frame as chipped.

4.3.2.1.2 Capping Detection via Unconstrained Laplacian Gradients

Surface fractures change pixel intensities abruptly. Variation 1 calculates these local gradients across the entire masked tablet area using the continuous discrete **Laplacian Operator**:

$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}$$

The magnitude map is converted to an absolute 8-bit integer space. Pixels exhibiting high second-order spatial frequencies (value>35) are summed globally. If this sum exceeds a rigid threshold (`CAP_THRESHOLD=180`), the tablet is flagged as capped.

4.3.2.1.3 Limitations

This configuration is highly fragile in industrial environments. Because it evaluates features globally, normal variations in edge curvature on non-circular tablets (e.g., oval or oblong forms) distort the convex hull calculation, triggering false chipping counts. Furthermore, the global Laplacian capture cannot differentiate between true structural cracks and normal surface textures, text debossing, or minor lighting glares.

4.3.2.1.4 Code: Variation 1

```
import cv2
import numpy as np

def preprocess_v1(frame):
    blur = cv2.GaussianBlur(frame, (5, 5), 0)
    hsv = cv2.cvtColor(blur, cv2.COLOR_BGR2HSV)
    return hsv[:, :, 2]

def segment_tablet_v1(gray, min_tablet_area=1000):
    # Global adaptive binarization via Otsu's method
    _, mask = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY +
cv2.THRESH_OTSU)
    kernel = np.ones((5, 5), np.uint8)
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel,
iterations=2)

    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
    if len(contours) == 0:
        return None, None
```

```

contour = max(contours, key=cv2.contourArea)
if cv2.contourArea(contour) < min_tablet_area:
    return None, None

clean = np.zeros_like(mask)
cv2.drawContours(clean, [contour], -1, 255, -1)
return clean, contour

def detect_chip_v1(mask, contour):
    # Compute the minimal convex envelope
    hull = cv2.convexHull(contour)
    hull_mask = np.zeros_like(mask)
    cv2.drawContours(hull_mask, [hull], -1, 255, -1)

    chip = cv2.subtract(hull_mask, mask)
    chip_area = np.sum(chip > 0)
    return chip, chip_area

def detect_cap_v1(gray, mask):
    # Calculate second-order spatial derivatives using
    standard Laplacian
    lap = cv2.Laplacian(gray, cv2.CV_64F)
    lap = np.uint8(np.abs(lap))

    # Mask the gradient map to restrict evaluation to the
    tablet body
    cap = cv2.bitwise_and(lap, lap, mask=mask)
    cap_area = np.sum(cap > 35)
    return cap, cap_area

```

4.3.2.1.5 Real time results on conveyor Belt

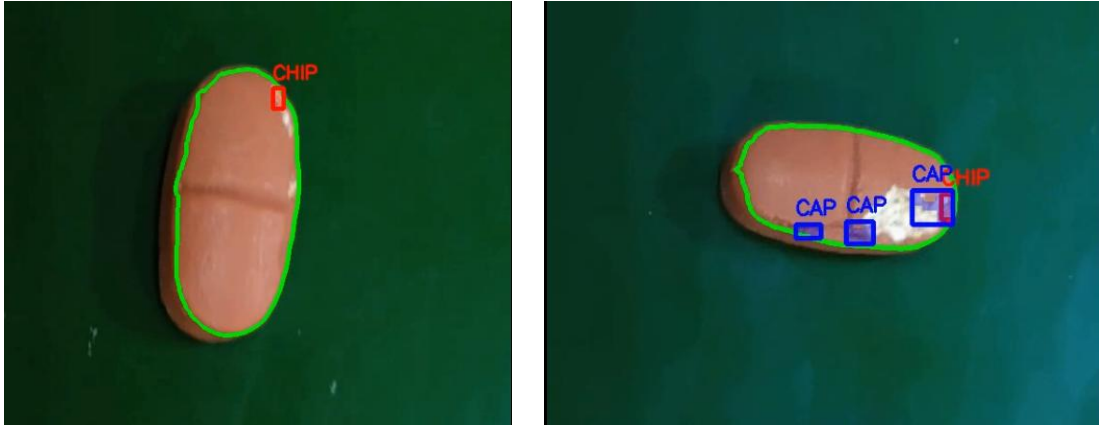


Figure 4.11 : Real time results of variation-1

4.3.2.2 Variation 2: Region-Constrained Spatial Zoning

To address the false positives in Variation 1, Variation 2 introduces spatial zoning. Instead of running global checks, it restricts defect isolation to specific regions where chips and cracks physically occur.

4.3.2.2.1 Boundary-Ring Isolation for Chipping

Because physical chipping occurs along the outer perimeter, Variation 2 isolates a specialized **Boundary Ring**. The primary tablet mask (M) is subjected to morphological erosion using a square kernel (K_{boundary}) with a width of 18 pixels:

$$\text{Boundary Ring} = M \setminus (M \ominus K_{\text{boundary}})$$

This operation isolates a narrow band tracing the perimeter. Concurrently, the grayscale channel is thresholded at a high luminance level ($\text{CHIP_BRIGHTNESS}=175$) to capture the reflective properties of exposed internal excipients. The final chip mask is generated via a localized intersection:

$$\text{Chip Mask} = \text{Boundary Ring} \cap \text{Thresholded Gray}$$

4.3.2.2.2 Filtered Laplacian Capping Analysis

For capping, Variation 2 filters the raw Laplacian gradient map through an explicit binarization step (CAP_TEXTURE=28). To suppress background noise, the binary output undergoes successive Morphological Closing and Opening operations:

$$\text{Final Cap} = (\text{Binarized Laplacian} \bullet K_{\text{close}}) \circ K_{\text{open}}$$

This eliminates isolated single-pixel noise while preserving dense, connected crack networks inside the tablet region.

4.3.2.2.3 Limitations

While restricting analysis to the boundary ring reduces interior false positives, a fixed grayscale threshold (>175) remains highly vulnerable to specular glare on smooth, shiny tablet coatings. Additionally, the standard Laplacian operator lacks the contrast control required to detect fine, low-contrast cracks without amplifying background noise.

4.3.2.2.4 Code: Variation 2

```
import cv2
import numpy as np

def detect_chip_v2(frame, tablet_mask, boundary_width=18,
chip_brightness=175):
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Generate perimeter ring via morphological mask erosion
    subtraction
```

```

    inner = cv2.erode(tablet_mask, np.ones((boundary_width,
boundary_width), np.uint8))
    boundary_ring = cv2.subtract(tablet_mask, inner)

    _, bright = cv2.threshold(gray, chip_brightness, 255,
cv2.THRESH_BINARY)

    chip = cv2.bitwise_and(boundary_ring, bright)
    chip = cv2.morphologyEx(chip, cv2.MORPH_OPEN, np.ones((5,
5), np.uint8))
    chip = cv2.morphologyEx(chip, cv2.MORPH_CLOSE, np.ones((7,
7), np.uint8))
    return chip

def detect_cap_v2(frame, tablet_mask, cap_texture=28):
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Extract structural edges from surface
    lap = cv2.Laplacian(gray, cv2.CV_64F)
    lap = cv2.bitwise_and(np.uint8(np.abs(lap)), tablet_mask)

    # Binarize gradient response and refine via structured
morphology
    _, cap = cv2.threshold(lap, cap_texture, 255,
cv2.THRESH_BINARY)
    cap = cv2.morphologyEx(cap, cv2.MORPH_CLOSE, np.ones((11,
11), np.uint8))
    cap = cv2.morphologyEx(cap, cv2.MORPH_OPEN, np.ones((5,
5), np.uint8))
    return cap

```

4.3.2.2.5 Real time results on conveyor Belt



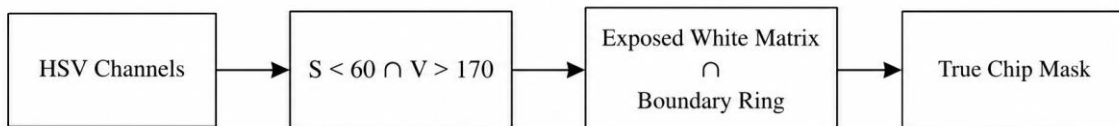
Figure 4.12 : Real time results of variation-2

Blue: Cap, Red:Chip, Green:Tablet

4.3.2.3 Variation 3: Colorimetric Profiling and Morphological Feature Filtering

Variation 3 transitions from simple intensity thresholds to advanced surface modeling. It combines multi-channel colorimetry with localized contrast balancing and geometric aspect-ratio filters.

Chipping:



Capping:

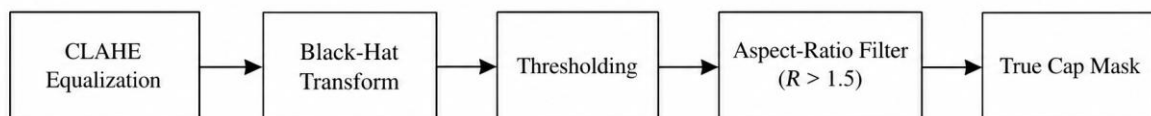


Figure 4.12: Colorimetric Profiling and Morphological Feature Filtering Flow

4.3.2.3.1 Colorimetric Chip Segmentation

When a coated tablet chips, it exposes the uncolored internal chemical core. This exposed core exhibits distinctive color properties: very low saturation (S) and high brightness (V). Variation

3 exploits this by applying a dual-channel logical constraint directly to the HSV components:

$$\text{Exposed Core Mask} = \{p \in \text{Frame} \mid S(p) < 60 \cap V(p) > 170\}$$

4.3.2.3.2 Localized Contrast Enhancement (CLAHE)

Fine capping cracks often have low contrast, making them difficult to isolate with standard gradients. Variation 3 addresses this by applying Contrast Limited Adaptive Histogram Equalization (CLAHE). Traditional global equalization stretches the entire image histogram, which can overamplify background noise. CLAHE divides the frame into distinct, non-overlapping contextual tiles (8×8 pixels).

For each tile, a localized histogram is computed. To prevent noise amplification, the contrast enhancement is limited by clipping the histogram at a predefined value (clipLimit=2.0) before redistributing the clipped pixels evenly across the gray levels. Bilinear interpolation is then applied to blend the tile boundaries and eliminate block artifacts.

4.3.2.3.3 Black-Hat Morphological Cracking Extraction

To isolate narrow, dark fractures from the balanced surface illumination, Variation 3 uses a **Black-Hat Transform**. This mathematical operation computes the difference between an image's morphological closing and the original input image:

$$T_{\text{blackhat}}(I) = (I \bullet K_{\text{blackhat}}) - I$$

Using a custom 15×15 pixel structuring element, this transform filters out large-scale shading variations while capturing dark, linear gaps that match the size of the kernel. The output is then binarized using a strict threshold ($\tau=18$).

4.3.2.3.4 Geometric Feature Filtering via Aspect-Ratio Testing

To separate true, elongated cracks from small, symmetrical artifacts like dust or surface pitting, the system performs a geometric feature pass. Individual contours are extracted from the binary Black-Hat mask, and the aspect ratio (R) of each contour's bounding box is evaluated:

$$R = \frac{\max(w, h)}{\min(w, h) + 1}$$

Contours are preserved only if they meet strict structural constraints: $R > 1.5$ and a minimum defect area > 100 pixels. This linear filter ensures the system selectively retains elongated, crack-like geometries while discarding random surface noise.

4.3.2.3.5 Code: Variation 3

```
import cv2
import numpy as np

def detect_chip_v3(frame, tablet_mask, boundary_width=22):
    # Convert to HSV space and split independent channels
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    _, s, v = cv2.split(hsv)

    inner = cv2.erode(tablet_mask, np.ones((boundary_width,
    boundary_width), np.uint8))
    boundary_ring = cv2.subtract(tablet_mask, inner)

    # Dual-channel logical constraint targeting uncolored,
    highly reflective cores
    white_core = np.logical_and(s < 60, v >
    170).astype(np.uint8) * 255

    # Intersect structural zone with colorimetric profile
    chip = cv2.bitwise_and(white_core, boundary_ring)
    chip = cv2.morphologyEx(chip, cv2.MORPH_CLOSE, np.ones((9,
    9), np.uint8))
    chip = cv2.morphologyEx(chip, cv2.MORPH_OPEN, np.ones((5,
    5), np.uint8))
```

```

    return chip

def detect_cap_v3(frame, tablet_mask, min_defect_area=100):
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Localized contrast normalization via adaptive tiling
    (CLAHE)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,
8))
    gray = clahe.apply(gray)

    # Mathematical Black-Hat transform to isolate dark, linear
    cracks:  $(I \cdot K) - I$ 
    blackhat = cv2.morphologyEx(gray, cv2.MORPH_BLACKHAT,
np.ones((15, 15), np.uint8))
    blackhat = cv2.bitwise_and(blackhat, tablet_mask)

    # Binarize texture anomalies
    _, cap = cv2.threshold(blackhat, 18, 255,
cv2.THRESH_BINARY)
    cap = cv2.morphologyEx(cap, cv2.MORPH_OPEN, np.ones((3,
3), np.uint8))
    cap = cv2.morphologyEx(cap, cv2.MORPH_CLOSE, np.ones((9,
9), np.uint8))

    # Geometric verification using Bounding Box Aspect Ratio
    contours, _ = cv2.findContours(cap, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
    filtered_cap = np.zeros_like(cap)

    for c in contours:

```

```
if cv2.contourArea(c) < min_defect_area:
    continue
x, y, w, h = cv2.boundingRect(c)
ratio = max(w, h) / (min(w, h) + 1)

if ratio > 1.5:
    cv2.drawContours(filtered_cap, [c], -1, 255, -1)

return filtered_cap
```

4.3.2.3.6 Real time results on conveyor Belt

Blue: Cap, Red: Chip, Green: Tablet

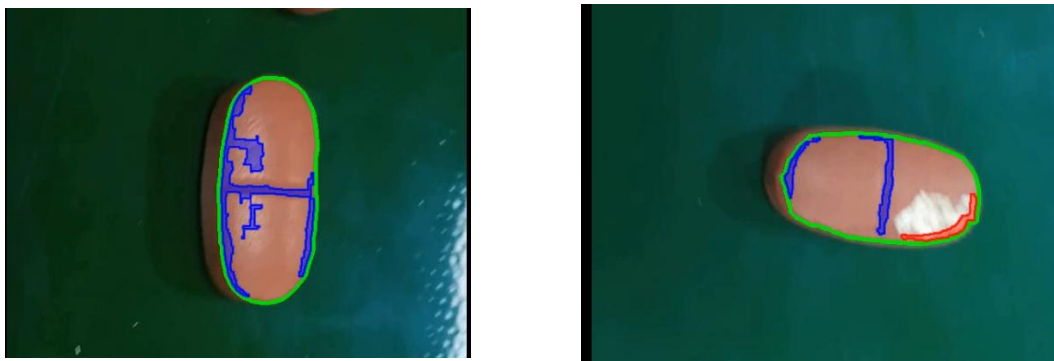


Figure 4.13: Real time results of Variation-3

4.4 Summary

This chapter presented the development and comparative evaluation of three complementary inspection paradigms for automated tablet defect detection: classical mathematical computer vision, deep learning segmentation, and hybrid vision architecture. The study initially investigated mathematical vision-based approaches to achieve interpretable and

computationally efficient defect analysis. Three algorithmic variations were designed using adaptive thresholding, Gaussian filtering, HSV-based illumination normalization, contour extraction, convex hull subtraction, Laplacian gradient profiling, spatial zoning, and morphological operations to localize tablets and quantify structural anomalies such as chipping and capping. Although these approaches provided deterministic decision boundaries and low computational overhead, their performance degraded under industrial variability including motion blur, shape diversity, and illumination changes. To overcome these limitations, the research transitioned toward deep learning-based instance segmentation. Progressive model development was conducted through iterative dataset refinement, annotation strategy optimization, and multi-stage training. Multiple architectures including YOLO11n-seg and Mask R-CNN were trained and benchmarked under different optimization, augmentation, and regularization schemes.

Chapter 5

Hybrid Approach for Tablet Localization & Defect Segmentation

5.1 Overview

The hybrid approach combines classical computer vision techniques with deep learning-based defect recognition instead of performing end-to-end learning directly from raw, unconstrained images, the system introduces deterministic preprocessing stages to standardize the visual input before applying driven inference.

This approach specifically addresses the common operational bottlenecks observed in purely deep learning-based architectures, including high sensitivity to ambient input variations, steep computational overhead, and an intensive dependency on massive, pixel-annotated training datasets. By decoupling the macroscopic task of **object localization** from the microscopic task of **defect feature extraction**, the workload is partitioned optimally: classical computer vision manages spatial constraints, while deep learning models focus entirely on anomaly detection.

5.2 Core Rationales for a Decoupled Architecture

The fundamental architecture of this hybrid system rests on a deliberate separation of duties: **Classical Computer Vision for Tablet Localization** and **Deep Learning for Defect Detection**. This configuration yields distinct operational advantages over end-to-end neural network models:

5.2.1 Eliminating Background Noise and Target Isolation

- By using deterministic, classical operations (such as Otsu's binarization and morphological contouring) to segment the tablet body, the background is stripped away completely.
- The deep learning network is provided with a tightly bounded, cropped, and masked region-of-interest (ROI). This prevents the network from extracting false-positive features from background clutter.

5.2.2 Drastic Reduction in Computational Complexity and Inference Latency

In an end-to-end DL, high-resolution full-frame images must pass through initial convolutional layers to perform spatial downsampling and locate the object of interest. This introduces unnecessary floating-point operations, increasing inference latency and hardware requirements.

- Classical segmentation isolates and crops the tablet bounding box via lightweight array index parsing, running in sub-millisecond execution times on standard CPU threads.
- The deep learning model processes significantly smaller input tensor dimensions (e.g., downscaling from a 1080p scene to a localized 128×128 pixel matrix). This reduces network depth requirements and speeds up inference times to meet strict production line frame rates.

5.2.3 Data Scarcity and Accelerating Model Convergence

Deep learning networks are notorious for requiring extensive datasets to achieve spatial invariance, meaning they must see a defect at every conceivable rotation, translation, and scale to recognize it reliably.

- The classical preprocessing stage applies geometric alignment using localized moments to calculate the tablet's center of mass and principal orientation angle (θ). The image is then mathematically rotated to a standard baseline orientation (0°).
- Because the deep learning model only encounters perfectly centered and oriented tablet matrices, it does not need to learn spatial rotation invariance. This drastically shrinks the volume of defective training samples required for high-accuracy generalization, enabling the network to converge faster during training with fewer parameters.

5.3 Hybrid System Architecture

The hybrid inspection pipeline is executed across three sequential processing tiers, transferring data from deterministic geometric spaces to deep feature maps, and finally to logical decision frameworks.

5.3.1 Real Time result of Hybrid Approach on a conveyor Belt

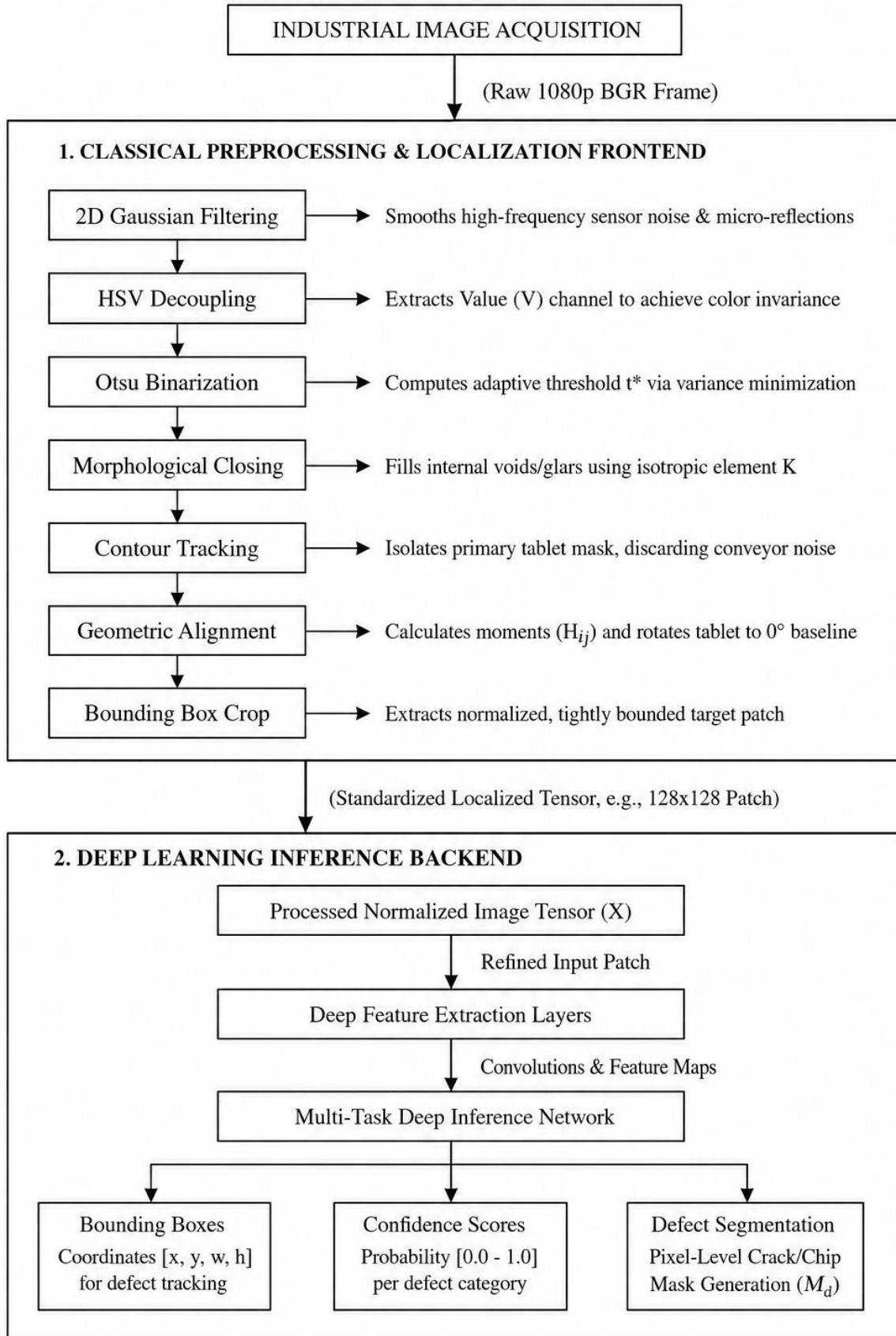


Figure 5.1: Mathematical and Deep Learning hybrid architecture

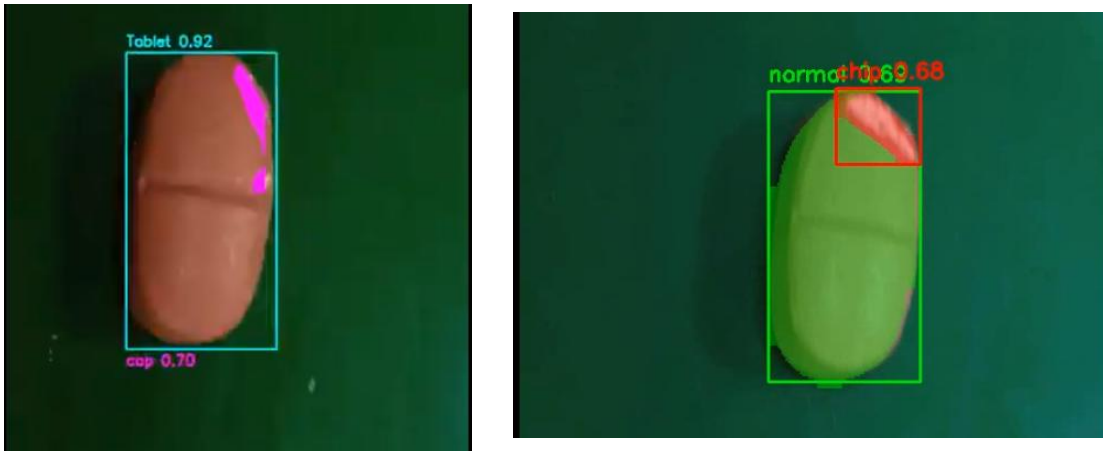


Figure 5.2: Real Time result of Hybrid Approach on a conveyor Belt

5.4 Summary

This chapter introduced the final hybrid architecture, integrating mathematical computer vision with deep learning segmentation to combine interpretability, geometric reasoning, and learning-based robustness. Classical methods were utilized for preprocessing, spatial constraint generation, contour refinement, and defect quantification, while segmentation networks performed high-level semantic understanding and defect boundary prediction. This fusion reduced false positives, improved edge consistency, enhanced explainability, and enabled accurate real-time tablet quality assessment suitable for industrial in-line deployment.

Chapter 6

Hardware Architecture and Deployment

6.1 Introduction

The effectiveness of an In-line tablet quality control system using computer vision is fundamentally dependent on the design and stability of its hardware infrastructure. Unlike purely software-based systems, real-time industrial inspection requires tightly synchronized hardware components that ensure consistent image acquisition, controlled environmental conditions, and reliable data transfer to the processing pipeline.

This chapter presents a detailed description of the complete hardware architecture implemented in the proposed system. The design integrates mechanical, optical, and computational components into a unified inspection framework capable of performing continuous, non-destructive tablet defect detection on a moving conveyor line.

The primary objectives of the hardware design are:

- To ensure consistent and blur-free image capture under motion conditions
- To maintain uniform lighting for robust defect detection
- To enable real-time processing with minimal latency
- To support structured data storage and visualization using modern DevOps tools

The overall hardware architecture is aligned with industrial machine vision standards used in pharmaceutical quality control systems.

6.2 Overall Hardware System Architecture

The proposed In-Line Tablet Quality Control System is composed of several interconnected hardware modules that collectively perform image acquisition, data transmission, defect

detection, and result visualization. Each module is designed to perform a specific function while maintaining synchronization with the remaining components of the inspection pipeline.

The hardware architecture follows a sequential workflow similar to those employed in commercial pharmaceutical inspection systems. Tablets are first transported through the inspection zone using a controlled movement mechanism. As each tablet enters the predefined imaging region, a high-resolution camera captures images under carefully controlled illumination conditions. These images are then transferred to a processing unit where computer vision algorithms perform defect detection.

The architecture incorporates a structured data persistence and visualization pipeline to ensure traceability and real-time monitoring. The resulting inference outputs, along with system logs and metadata, are not directly visualized at this stage. Instead, they are systematically transmitted to a centralized time-series database deployed in a containerized environment.

The system utilizes InfluxDB running inside Docker containers as the primary data storage layer for storing quality inspection results, inference metrics, and system-level logs. In addition to structured database storage, raw operational logs are also persisted in a local log file (`qc_logs.txt`) to ensure redundancy and support debugging, auditability, and fault analysis.

Once stored in the database, the inspection results are visualized through Grafana dashboards, which serve as the primary monitoring interface for real-time and historical analysis. This enables stakeholders to continuously monitor the inspection process, analyze defect patterns, and evaluate overall system performance without requiring a separate standalone application interface.

The complete system architecture consists of the following major subsystems:

- Tablet Feeding and Conveyor Unit
- Imaging and Camera Module
- Controlled Lighting System
- Edge Computing and Processing Unit
- Communication and Data Transfer Layer
- Database & Storage Layer
- User Interface and Monitoring Module

The interaction between these modules forms a continuous inspection pipeline as shown below:

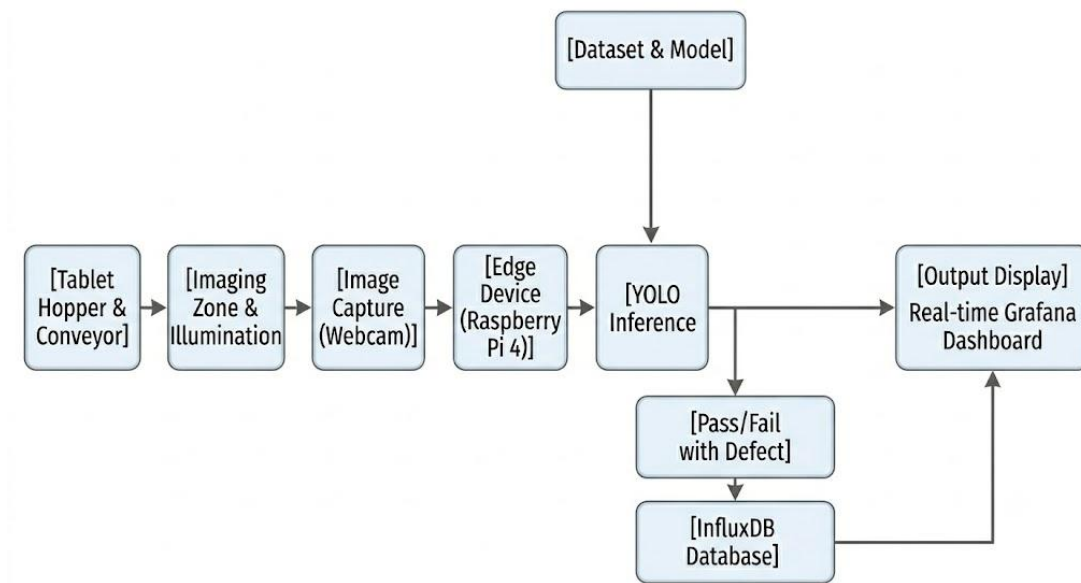


Figure 6.1: Block Diagram of Overall Hardware System Architecture

The architecture ensures that every captured tablet image is processed, analyzed, stored, and visualized in a structured and traceable manner.

6.3 Tablet Transport and Conveyor System

6.3.1 Functional Role

The tablet feeding and conveyor subsystem serves as the primary transportation mechanism within the proposed inspection framework. Its main purpose is to move tablets through the inspection zone where image acquisition is performed. The conveyor ensures that tablets remain within the camera's field of view for sufficient time, allowing high-quality images to be captured for defect analysis.

Unlike fully automated industrial production lines, the prototype developed for this project utilizes a manually controlled conveyor system. The operator initiates conveyor movement through a dedicated push-button switch, allowing controlled transportation of tablets during inspection experiments. This approach was selected to simplify implementation while maintaining the fundamental operational principles of an industrial in-line inspection system.

The conveyor subsystem plays an important role in ensuring repeatable image acquisition conditions and maintaining a consistent inspection workflow. By providing controlled tablet movement, the system minimizes positioning variations that could negatively affect defect detection accuracy.

6.3.2 Mechanical Design Considerations

The conveyor system was designed to satisfy the operational requirements of the proposed computer vision inspection platform. Several design considerations were identified during the development phase to ensure reliable tablet transportation and image acquisition.

The key design requirements were as follows:

- Uniform and stable tablet movement throughout the inspection process.
- Minimal mechanical vibration to prevent image distortion.
- Adequate tablet visibility within the camera field of view.
- Simple operator-controlled functionality.
- Compact dimensions suitable for laboratory-scale implementation.
- Low-cost construction using readily available materials.
- Sufficient durability for repeated testing and experimentation.

Additionally, the conveyor dimensions were selected to provide an adequate inspection region while maintaining a compact system footprint. The final conveyor design utilized a belt width of approximately 4 inches and a total conveyor length of approximately 2 feet.

6.3.3 Physical Construction

The conveyor prototype consists of a leather-based conveyor belt mounted on a supporting frame and driven by a small electric motor. Leather was selected as the belt material due to its availability, sufficient grip characteristics, and ability to provide relatively smooth motion during operation.

The conveyor structure includes:

- Leather conveyor belt.
- Drive assembly.

- Mechanical frame structure.
- Push-button control switch.
- Power supply unit.

The push-button control mechanism allows the operator to start and stop the conveyor as required during image acquisition experiments. This configuration enabled controlled testing and data collection without requiring complex automation hardware.

Table 6.1 Development and Refinement Process of the Conveyor System

Phase	Activity Performed	Objective	Observations	Outcome
Phase 1	Initial Conveyor Construction	To develop the first prototype of the conveyor system for experiments.	The conveyor successfully transported tablets; however, manual power control was limited and operational flexibility was low.	A functional prototype was established for preliminary testing and hardware validation.
Phase 2	Integration of Push-Button Control Mechanism	To improve operational control by allowing the operator to start and stop the conveyor on demand.	The addition of a push-button switch simplified testing procedures and improved user control during image acquisition experiments.	Conveyor usability and testing convenience were significantly improved.
Phase 3	Structural Repair and Alignment Adjustment	To restore mechanical stability after structural misalignment caused by handling and	Minor frame deformation affected belt alignment and smooth motion.	Conveyor stability was restored, resulting in smoother tablet movement and improved inspection consistency.

		transportation during experimentation.	Mechanical adjustments and reinforcement were performed.	
--	--	--	--	--



Figure 6.2: Conveyor Belt after Phase I

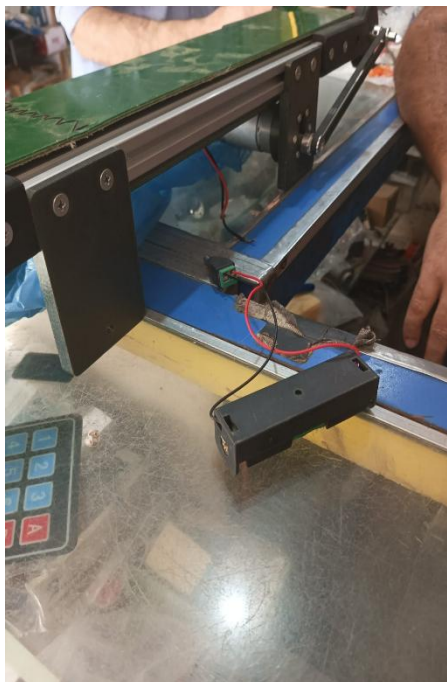


Figure 6.3: Conveyor Belt during repairing



Figure 6.4: Conveyor Belt in use

6.3.4 Speed Regulation

To evaluate the operational characteristics of the conveyor system, experimental measurements were conducted to determine the average transportation time.

The results indicated that a tablet requires approximately 13 seconds to travel from the feeding end of the conveyor to the discharge end.

Mathematically, synchronization can be represented as:

CONVEYOR VELOCITY CALCULATION	
Given Parameters:	Conveyor Length (d) = 2 ft = 0.6096 m Travel Time (t) = 13 s
Governing Formula:	$v = d / t$
Substitution & Calculation:	$v = 0.6096 \text{ m} / 13 \text{ s}$ $v = 0.046892... \text{ m/s}$
Final Value:	$v \approx 0.047 \text{ m/s}$

The relatively low conveyor speed is advantageous for the prototype system because it reduces motion blur, increases image acquisition stability, and provides sufficient processing time for real-time defect detection.

6.3.5 Engineering Considerations

Several engineering considerations were incorporated into the conveyor design to improve system stability and inspection performance.

Vibration Reduction

The leather conveyor belt provides a relatively smooth transportation surface, reducing abrupt movements and mechanical disturbances that could affect image quality.

Tablet Position Stability

The conveyor width was selected to accommodate tablet movement while limiting excessive lateral displacement. This helps maintain tablets within the predefined inspection region.

Ease of Operation

The push-button control mechanism simplifies operation and enables repeatable testing without requiring advanced industrial controllers.

Scalability

Although the current implementation uses manual control, the conveyor architecture can be extended in future work through the integration of sensors, programmable logic controllers (PLCs), encoders, and automatic feeding mechanisms to support fully automated industrial inspection workflows.

6.3.6 Limitations of the Prototype Conveyor

While the developed conveyor system successfully supports experimental validation of the proposed computer vision framework, several limitations remain.

- Tablet feeding is performed manually.
- Conveyor speed cannot be dynamically adjusted through software.
- No encoder-based synchronization exists between conveyor motion and image acquisition.
- Automatic reject mechanisms are not implemented.
- Industrial-grade throughput cannot be achieved with the current prototype.

Despite these limitations, the conveyor system provides a practical and cost-effective platform for evaluating the feasibility of real-time tablet quality control using computer vision.

6.4 Imaging System (Camera Module)

6.4.1 Role of the Imaging System

The imaging subsystem serves as the primary data acquisition component of the proposed in-line tablet quality control system. Since the performance of the defect detection model depends

heavily on the quality of the acquired images, the design of the imaging system directly influences the overall inspection accuracy, reliability, and robustness of the solution.

In a computer vision-based inspection environment, the camera acts as the interface between the physical manufacturing process and the digital analysis pipeline. Any degradation in image quality, such as motion blur, poor focus, uneven illumination, or geometric distortion, can significantly affect the ability of the machine learning model to correctly identify tablet defects. Therefore, the imaging module was designed to capture clear, high-resolution, and consistent images of tablets moving through the inspection region.

The imaging system operates continuously during inspection. As tablets move through the predefined inspection zone, images are captured and immediately forwarded to the processing unit for inference. This approach enables non-destructive and real-time quality assessment while maintaining compatibility with industrial production environments.

6.4.2 Camera Selection & Specifications

The imaging subsystem utilizes a commercially available USB webcam for image acquisition. The selected camera is an **AutoFocus HD Video Webcam** equipped with a CMOS image sensor. The camera supports Full HD image capture with a maximum resolution of 1920×1080 pixels at 30 frames per second (FPS). The autofocus functionality assists in maintaining image sharpness when minor variations occur in the distance between the camera and the inspection surface.

The webcam was selected due to its affordability, ease of integration with the processing unit, and ability to provide image quality sufficient for detecting visible tablet defects.

The specifications of the camera are summarized in Table 5.2.

Table 6.2 Specification of a Camera

Parameter	Specification
Camera Type	AutoFocus HD Video Webcam
Image Sensor	CMOS HD Image Sensor
Resolution	1920×1080 Pixels

Frame Rate	Up to 30 FPS
Focus Mechanism	Auto Focus
Connection Interface	USB
Output Format	Digital Video Stream



Figure 6.5: Auto Focus HD Webcam

6.4.3 Camera Placement Strategy & Lighting Setup

The camera was positioned on the side of the conveyor system and directed towards the tablet inspection area. This placement was selected according to the physical structure of the prototype and the available mounting space.

The camera was adjusted manually to ensure that the tablets remained visible while moving on the conveyor belt. Several trial runs were performed to determine a suitable distance and angle that allowed clear image capture without interfering with the movement of tablets.

Proper illumination is an important requirement for image-based inspection systems because image quality directly affects the performance of the defect detection model.

Importance of Lighting

Inconsistent lighting leads to:

- False positives in defect detection

- Poor edge definition
- Reduced model generalization

To ensure clear visibility of tablet surfaces, a ring light was installed around the camera lens. The ring light provides uniform illumination around the inspection area and helps reduce shadows that may occur due to external lighting conditions.

The lighting setup was selected because of its simple installation, low cost, and ability to provide sufficient brightness for the prototype inspection system. During experimentation, the ring light remained fixed with the camera to maintain consistent illumination conditions for all captured images.

The combination of the side-mounted camera and ring light helped produce clear images suitable for computer vision-based defect detection.



Figure 6.6: Setup of a Camera

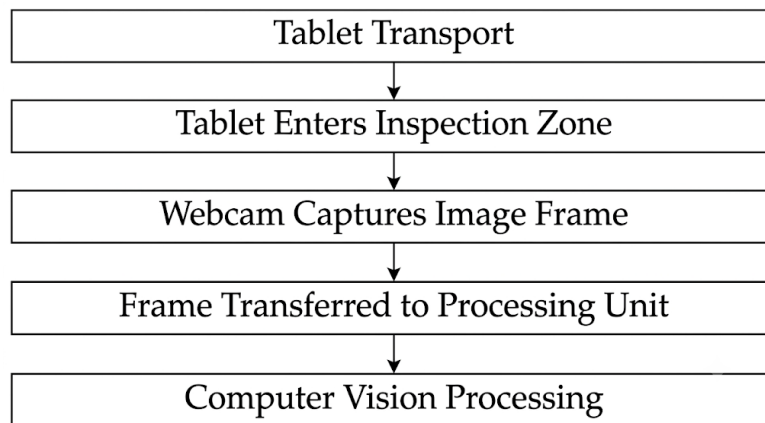


Figure 6.7: Setup of a Ring-light

6.4.4 Image Acquisition Process

During operation, tablets move through the inspection region using the conveyor mechanism. As tablets enter the camera's field of view, image frames are continuously captured and transmitted to the processing unit.

The image acquisition workflow can be summarized as follows:



High-quality image acquisition enabled the YOLOv8 model to learn defect-specific features more effectively, contributing to improved detection performance across multiple defect categories.

The imaging module therefore serves as a critical factor in the overall effectiveness of the computer vision inspection pipeline.

6.5 Edge Inference and Processing Hardware

6.5.1 Raspberry Pi 4 Hardware Platform.

The edge processing subsystem utilizes a Raspberry Pi 4 Model B equipped with 8 GB RAM. The device was selected due to its compact form factor, affordability, low power consumption, and ability to execute lightweight deep learning workloads.



Figure 6.8: Raspberry Pi 4 Model B

Its primary responsibility is to execute the real-time computer vision pipeline and generate defect detection results from incoming image streams. After inference, the generated inspection results are transmitted through the network to the monitoring infrastructure hosted on a separate workstation, where long-term storage and visualization are performed.

Table 6.3 Raspberry Pi Specifications

Parameter	Specification
Device	Raspberry Pi 4 Model B
RAM	8 GB
Processor	Broadcom BCM2711
CPU Architecture	Quad-Core ARM Cortex-A72
Clock Frequency	1.5 GHz
Operating System	Raspberry Pi OS
Storage Medium	SD Card
Connectivity	Ethernet
Deployment Mode	Headless

6.5.2 Separation of Edge and Monitoring Infrastructure

The proposed inspection system follows a distributed architecture in which real-time inspection tasks are separated from monitoring and data management services. This design was adopted to reduce the computational burden on the edge device and improve overall system organization.

The Raspberry Pi 4 serves as the edge computing unit and is responsible for image acquisition, tablet detection, object tracking, and defect identification using the trained YOLO models. Since these operations require immediate processing, they are executed locally at the inspection station to ensure real-time performance.

Monitoring and data storage services are hosted on a separate workstation connected through a TCP/IP network. Inspection statistics generated by the Raspberry Pi are transmitted over the network and stored in the database, where they can be accessed for analysis and reporting.

By separating inference tasks from monitoring services, the system avoids placing additional workload on the Raspberry Pi and enables more efficient utilization of available hardware resources. This approach also simplifies system maintenance, as updates to the database or dashboard services can be performed independently without affecting the inspection pipeline.

The architecture provides the following benefits:

- Reduced computational load on the Raspberry Pi.
- Centralized storage of inspection and production data.
- Simplified dashboard configuration and management.
- Independent scaling of monitoring and visualization services.
- Easier integration of additional monitoring components in future deployments.

6.6 Deployment Strategy and Operational Environment of the inspection system

6.6.1 Objectives of System Deployment

The deployment phase represents the transition of the proposed computer vision-based inspection system from a development environment to an operational environment capable of performing real-time tablet inspection. While model development and validation establish the effectiveness of the defect detection algorithm, deployment focuses on integrating the trained model with hardware resources, communication infrastructure, monitoring tools, and storage services to create a complete inspection solution.

The primary objective of deployment was to establish a fully functional in-line quality control system capable of detecting tablet defects in real time while simultaneously recording inspection data for monitoring and analysis. The deployed system was required to support continuous image acquisition, real-time inference, centralized data storage, and graphical visualization of production statistics.

Unlike traditional quality control approaches where inspection data is manually recorded and analyzed after production, the proposed system provides immediate feedback regarding production quality. This enables faster identification of manufacturing issues and improves traceability throughout the inspection process.

The specific objectives of deployment can be summarized as follows:

- Deployment of the trained YOLO defect detection model on a low-cost edge computing device.
- Real-time acquisition and processing of tablet images.
- Automatic logging of inspection events and defect statistics.
- Centralized storage of inspection records for historical analysis.
- Development of real-time monitoring dashboards for production oversight.
- Establishment of a scalable architecture that separates inference tasks from monitoring services.

The achievement of these objectives was essential for demonstrating the practical applicability of the proposed system within pharmaceutical manufacturing environments.

6.6.2 Transition from Development Environment to Operational Environment

Once model development was completed, attention shifted towards deploying the trained model on an edge computing platform. This transition introduced several engineering challenges including limited computational resources, network communication requirements, storage management, and real-time performance constraints.

The deployment environment was designed to closely resemble a realistic industrial inspection setup. A USB camera continuously captured images of tablets, while a Raspberry Pi 4 executed the computer vision inference pipeline locally. Inspection results generated by the Raspberry Pi were transmitted through a local network to a monitoring workstation where database and visualization services were hosted.

The transition from development to deployment also required additional considerations related to system stability, software dependency management, remote administration, and long-term operational reliability. Consequently, the deployment phase served not only as a validation of the trained model but also as an evaluation of the complete hardware-software ecosystem.

6.6.3 Overview of the Final Deployment Architecture

The final deployment architecture follows a distributed edge-computing design. The architecture consists of four major layers:

1. Image Acquisition Layer
2. Edge Processing Layer
3. Monitoring and Storage Layer
4. Visualization Layer

The Image Acquisition Layer is responsible for capturing tablet images using a USB camera positioned above the inspection area. Continuous image streams are generated and transmitted directly to the edge processing device.

The Edge Processing Layer consists of a Raspberry Pi 4 configured as the primary inference device. This layer executes the YOLO model and performs all image processing operations required for defect detection. After processing each frame, the Raspberry Pi generates inspection results including defect classifications, confidence scores, pass-fail status, and processing timestamps.

The Monitoring and Storage Layer is hosted on a dedicated workstation. This workstation runs Docker-based services including InfluxDB and Grafana. Inspection records received from the Raspberry Pi are stored in InfluxDB as time-series data, allowing efficient storage and retrieval of production information.

The Visualization Layer consists of Grafana dashboards that transform inspection records into meaningful visual insights. Operators can observe production counts, defect frequencies, pass-fail statistics, and system performance metrics through an intuitive graphical interface.

This layered architecture improves modularity, simplifies maintenance activities, and enables independent scaling of monitoring infrastructure without impacting edge inference performance.

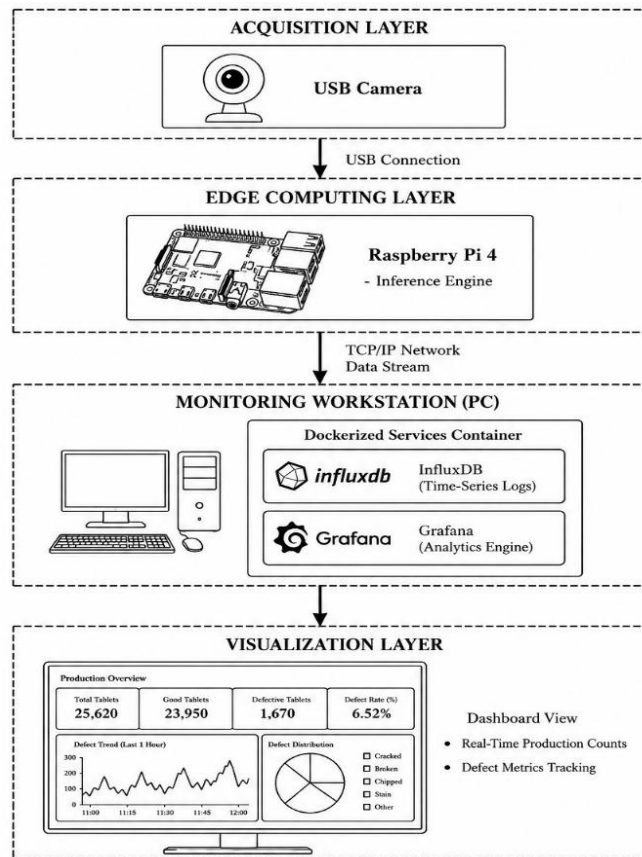


Figure 6.9: Hardware-to-Software Data flow Pipeline



Figure 6.10: Hardware to Software Setup

6.6.4 Data Communication and System Integration Workflow

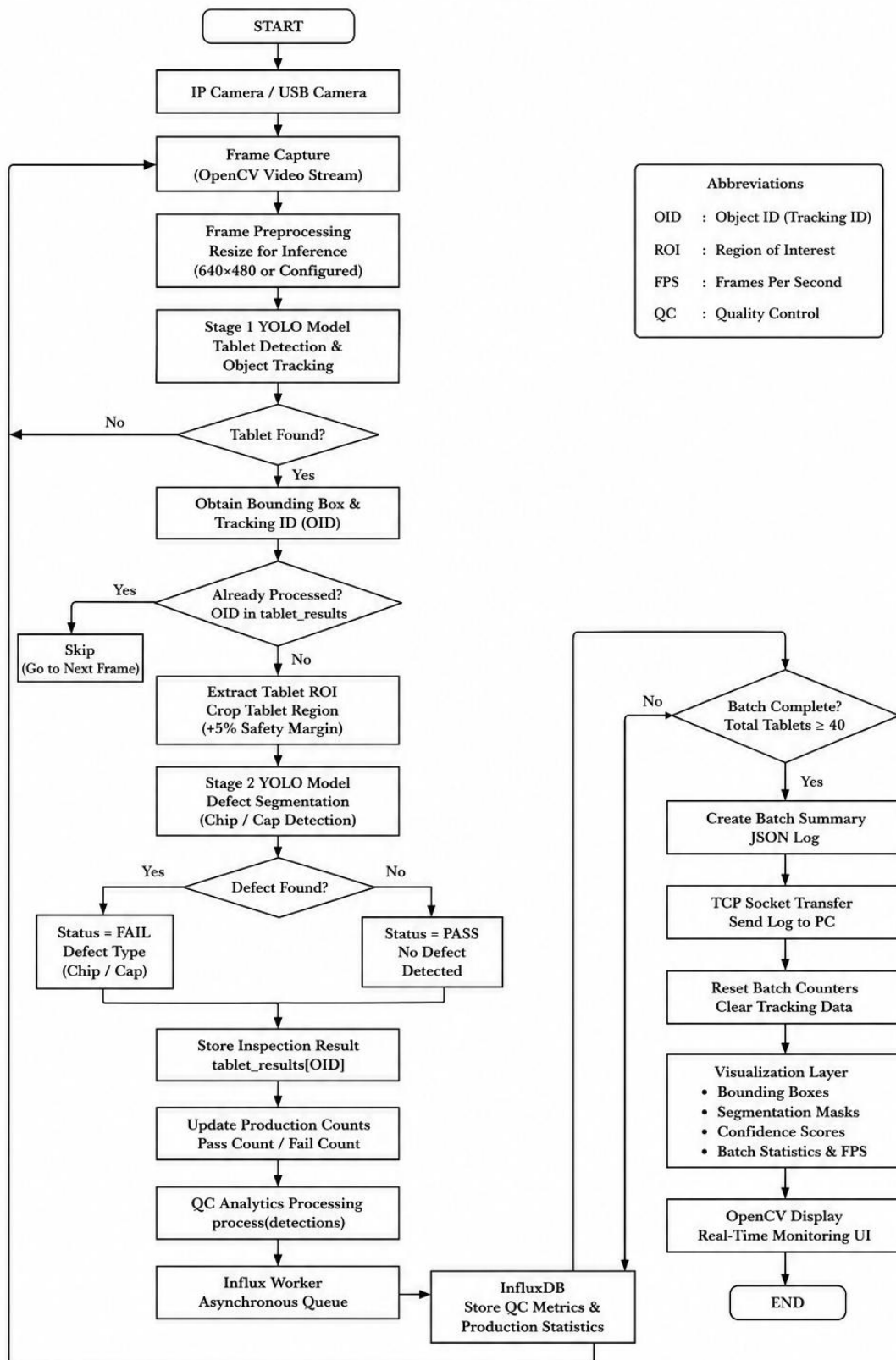


Figure 6.11: System Integration Workflow

6.6.5 System Constraints and Limitations

Although the proposed system achieves functional real-time tablet inspection under the defined experimental setup, certain hardware and system-level constraints were observed during implementation. These limitations are primarily related to computational capacity, imaging hardware quality, and network communication latency. Such constraints are common in low-cost edge-based inspection systems and represent key trade-offs between affordability and performance.

Limited Computational Performance of Raspberry Pi

The Raspberry Pi 4, while suitable for lightweight edge computing tasks, has limited processing capability compared to dedicated GPU-based systems commonly used in industrial machine vision applications.

During real-time execution of the inference pipeline, the device exhibits increased processing latency under continuous frame input conditions. This limits the maximum achievable frame processing rate and may affect performance in high-speed conveyor scenarios.

Consumer-Grade Camera Limitations

The imaging subsystem uses a standard Auto Focus HD USB webcam, which is a consumer-grade device rather than an industrial vision camera.

While the camera is capable of capturing Full HD resolution (1920×1080), its limitations include:

- Reduced sensitivity to fine surface textures
- Limited control over optical parameters such as exposure precision and shutter control
- Lower consistency under varying lighting conditions compared to industrial cameras

These factors can impact the visibility of very subtle tablet defects, particularly those requiring high micro-texture resolution. Despite these limitations, the camera provides just acceptable performance for detecting major surface and shape-related defects within the scope of this project.

Network-Induced Latency

Since the system follows a distributed architecture, where inference is performed on the Raspberry Pi and data storage and visualization are handled on a separate workstation, network communication introduces measurable latency.

Inspection results are transmitted over a local network to the PC hosting InfluxDB and Grafana services. Under variable network conditions, minor delays may occur between:

- Defect detection (edge device)
- Data storage (InfluxDB)
- Dashboard update (Grafana visualization)

Although this delay does not affect inference results on the edge device, it introduces slight lag in real-time dashboard visualization.

6.7 Summary

This chapter presented the complete hardware architecture of the proposed in-line tablet quality control system. The system integrates a conveyor-based transport mechanism, a webcam-based imaging module, controlled lighting conditions, and a Raspberry Pi 4-based edge inference unit.

The Raspberry Pi performs real-time defect detection using a YOLO based computer vision pipeline, while inspection results are transmitted to a separate workstation hosting InfluxDB and Grafana for storage and visualization.

The proposed hardware design emphasizes low-cost implementation, modular system separation, and real-time operational capability. Despite certain limitations related to computational power, imaging hardware quality, and network latency, the system demonstrates a functional and scalable approach to automated tablet inspection suitable for cost-sensitive industrial environments.

Chapter 7

System Testing, Deployment Constraints and Performance Evaluation

7.1 Introduction

The successful deployment of a computer vision system requires more than the development of an accurate machine learning model. In practical industrial environments, the model must be integrated with hardware devices, communication networks, storage systems, and monitoring tools to achieve reliable real-time operation. Therefore, after completing model development and validation, the next phase of this project focused on system deployment and experimental evaluation.

This chapter evaluates the operational performance of the system under practical conditions. Various aspects including image acquisition, real-time inference, data communication, logging, dashboard visualization, and overall system responsiveness were assessed. The objective was to verify that all software and hardware components functioned together as an integrated quality control solution.

The chapter also discusses challenges encountered during deployment, limitations observed during testing, and the measures adopted to improve system stability. Unlike laboratory-based model evaluation, deployment introduced several practical challenges related to hardware limitations, network communication, storage management, remote administration, and system integration. Multiple deployment strategies were investigated and tested before arriving at the final architecture. Throughout this process, several iterations were performed to improve system responsiveness, reduce latency, and enhance operational stability. Through this evaluation, the feasibility of implementing a computer vision-based tablet inspection system in a manufacturing environment is demonstrated.

7.2 Configuration and Remote Development Environment

7.2.1 Raspberry Pi OS Configuration

The implementation phase began with the configuration of the Raspberry Pi 4 as the primary edge computing platform for executing the defect detection model. Since the inspection system

was intended to operate continuously without requiring a dedicated monitor, keyboard, or mouse, a headless deployment strategy was adopted. This approach allowed the Raspberry Pi to be remotely managed from a workstation while reducing hardware requirements and simplifying deployment within laboratory and industrial environments.

The initial configuration process involved installing Raspberry Pi OS and enabling remote administration services.

Several challenges were encountered during the operating system installation stage. Initially, the 8 GB version of Raspberry Pi OS was installed on a single USB storage device. Although the operating system functioned correctly, storage utilization increased rapidly after installing Python dependencies, computer vision libraries, model weights, and additional software packages. As a result, available storage became insufficient for long-term operation.

To overcome this issue, the operating system was replaced with a lighter 4 GB installation. While this reduced storage consumption, the available memory proved inadequate for stable execution of the YOLO inference pipeline. The limited resources resulted in reduced system responsiveness and occasional instability during model execution.

Consequently, the operating system was reinstalled using the 8 GB configuration while introducing a dual-storage arrangement consisting of both a USB drive and an SD card. This configuration provided sufficient storage capacity for operating system files, trained models, datasets, and logging data while maintaining adequate system performance. The final storage configuration remained stable throughout subsequent testing and deployment activities.

Table 7.1 summarizes the operating system deployment iterations performed during implementation.

Table 7.1 Operating System Deployment Iterations

Configuration	Result
8 GB + USB	Storage Full
4 GB OS	Low Performance
8 GB + USB + SD	Final Setup

The final Raspberry Pi configuration established a reliable platform for executing the computer vision application and supporting continuous inspection operations.

7.2.2 SSH-Based Remote Administration

Secure Shell (SSH) served as the primary mechanism for remote administration of the Raspberry Pi throughout the implementation phase. SSH is a network protocol that enables secure command-line access to remote systems over a network connection. The protocol encrypts communication between devices, ensuring secure transmission of commands and configuration data.

Following the installation of Raspberry Pi OS, SSH functionality was enabled through the Raspberry Pi configuration utility. Once activated, the Raspberry Pi became accessible through the local network using its assigned IP address. PuTTY was selected as the SSH client due to its lightweight design and widespread adoption for remote Linux administration.

The SSH environment was utilized extensively during system development. Through SSH access, software packages were installed, Python environments were configured, system updates were performed, and debugging operations were conducted. The ability to remotely execute commands significantly accelerated the development process and eliminated the need for direct physical interaction with the Raspberry Pi hardware.

The SSH implementation proved highly reliable and remained an essential component throughout all subsequent deployment stages. However, SSH only provides terminal-based access and does not support graphical visualization of applications. Therefore, additional mechanisms were required for viewing detection results and monitoring system operation.

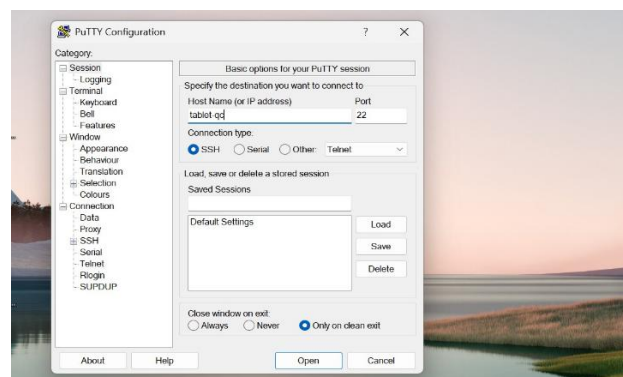


Figure 7.1 PuTTY Interface

7.3 Evolution of the Deployment Architecture

One of the most challenging aspects of system implementation involved establishing a reliable mechanism for transmitting video streams between the Raspberry Pi and the monitoring workstation. Since the object detection model was executed on the Raspberry Pi while visualization and monitoring activities were often performed on a separate workstation, multiple transmission strategies were investigated and evaluated.

One of the most challenging aspects of system implementation involved establishing a reliable mechanism for transmitting video streams between the Raspberry Pi and the monitoring workstation. Since the object detection model was executed on the Raspberry Pi while visualization and monitoring activities were often performed on a separate workstation, multiple transmission strategies were investigated and evaluated.

7.3.1 OpenCV Socket-Based Streaming

The first implementation utilized a custom socket-based communication mechanism developed using Python. In this configuration, a dedicated streaming script was executed on the workstation. Upon execution, the script entered a listening state and waited for incoming image frames transmitted by the Raspberry Pi.

After the streaming application was initialized on the workstation, the inference application was launched on the Raspberry Pi. Communication between the two systems was established through IP-based socket connections. Captured frames were processed by the inference pipeline and subsequently transmitted to the workstation for visualization.

The second application was executed on the Raspberry Pi and was responsible for:

- Capturing frames from the USB camera
- Running YOLO inference
- Sending processed frames through a network socket
- Displaying results on the workstation

The communication process is illustrated in Figure 6.1.



Figure 7.2 Socket Based Streaming

7.3.2 Challenges Encountered During Socket-Based Streaming

Although the system functioned correctly, significant latency issues were observed during practical testing. Visual changes occurring in front of the camera required several minutes before appearing on the workstation display. In certain situations, delays approaching five minutes were recorded, making the system unsuitable for real-time monitoring applications.

An additional challenge was associated with network configuration. The IP addresses assigned to devices differed depending on the operating environment. For example, deployment within the university laboratory utilized a different network configuration than deployment within a home environment. Consequently, source code modifications were frequently required to update IP addresses before establishing successful communication.

These limitations motivated the investigation of alternative video transmission methods.

7.3.3 Transition towards RTSP-Based Streaming Using MediaMTX and VLC

To overcome the latency limitations of the socket-based implementation, a Real Time Streaming Protocol (RTSP) solution was investigated. RTSP is widely used in surveillance and industrial monitoring systems because it provides efficient low-latency video transmission capabilities.

The RTSP implementation utilized MediaMTX as the streaming server. The Raspberry Pi acted as the video source while VLC Media Player was employed as the client application for stream visualization.

The implementation process involved configuring MediaMTX, establishing RTSP endpoints, and modifying the inference pipeline to support network-based video streaming. While the

RTSP approach significantly improved video transmission performance, the configuration process introduced substantial complexity.

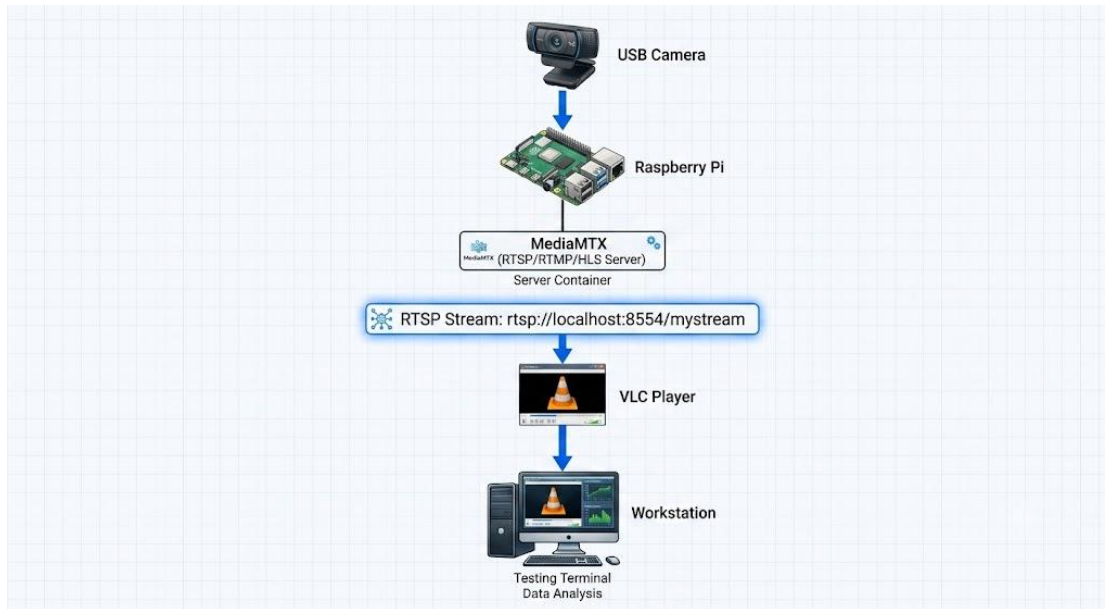


Figure 7.3 RTSP Based Streaming

7.3.4 Challenges Encountered During RTSP-Based Streaming

Several issues were encountered during integration. In particular, the inference application was unable to directly initialize and access the camera stream when executed through the Python environment. However, when camera initialization commands were manually executed through the terminal, the video stream operated correctly. This inconsistency complicated debugging activities and increased deployment complexity.

Although RTSP reduced transmission latency compared with the previous approach, the configuration process required extensive troubleshooting and maintenance. The complexity of managing MediaMTX configurations, stream endpoints, camera initialization procedures, and VLC integration reduced the practicality of the solution for routine deployment.

As a result, an alternative strategy emphasizing simplicity and maintainability was investigated.

7.3.5 Adoption of RealVNC for Remote Edge Device Management

Following extensive experimentation with both custom streaming and RTSP transmission, a more practical solution was adopted through the use of Virtual Network Computing (VNC).

VNC is a graphical desktop-sharing technology that enables complete remote control of a computer over a network connection.

Unlike SSH, which provides only terminal access, VNC transmits the graphical desktop environment of the Raspberry Pi to a remote workstation. This allows applications running on the Raspberry Pi to be viewed and controlled exactly as if a monitor were directly connected to the device.

The VNC implementation utilized RealVNC Server on the Raspberry Pi and RealVNC Viewer on the Windows workstation. After entering the Raspberry Pi IP address into the VNC Viewer application, a secure connection was established and the complete Raspberry Pi desktop became accessible remotely.

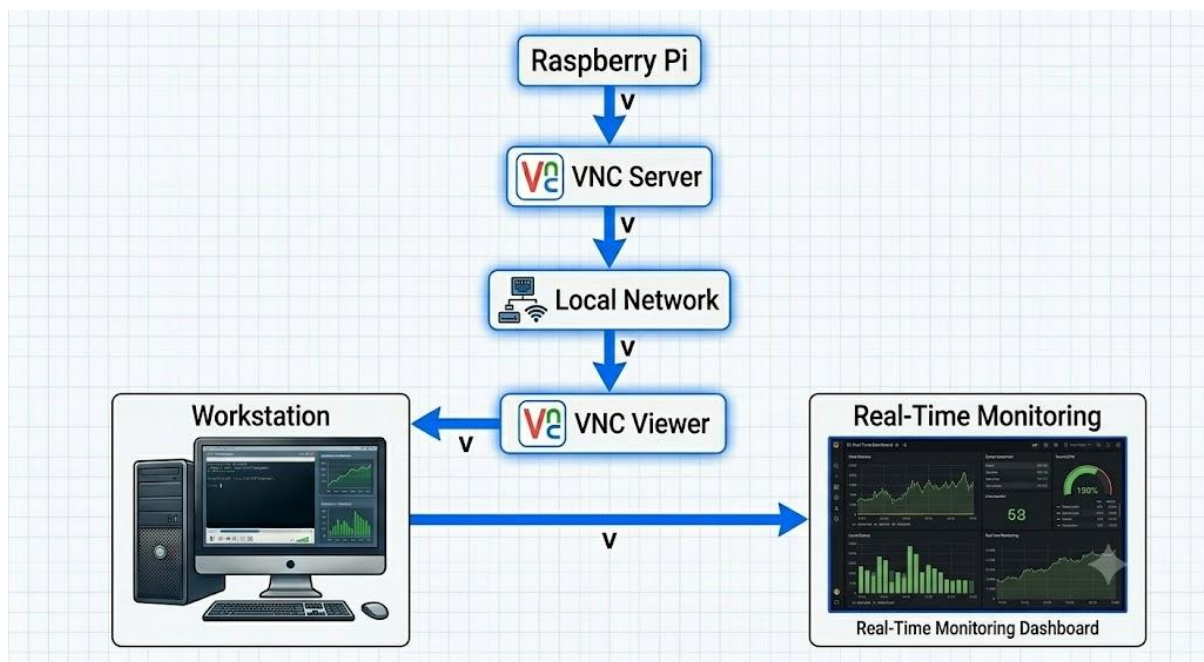


Figure 7.4 RealVNC Communication

The VNC-based approach eliminated the need for separate streaming applications and substantially simplified deployment. The inference application executed directly on the Raspberry Pi while all monitoring activities were performed through the remote desktop session.

This approach provided several advantages including simplified configuration, improved reliability, reduced software complexity, and easier troubleshooting. Furthermore, because the

complete desktop environment was available remotely, file management, software updates, and debugging operations could be performed from a single interface.

Consequently, VNC became the primary monitoring solution used throughout the final testing and validation stages of the project.

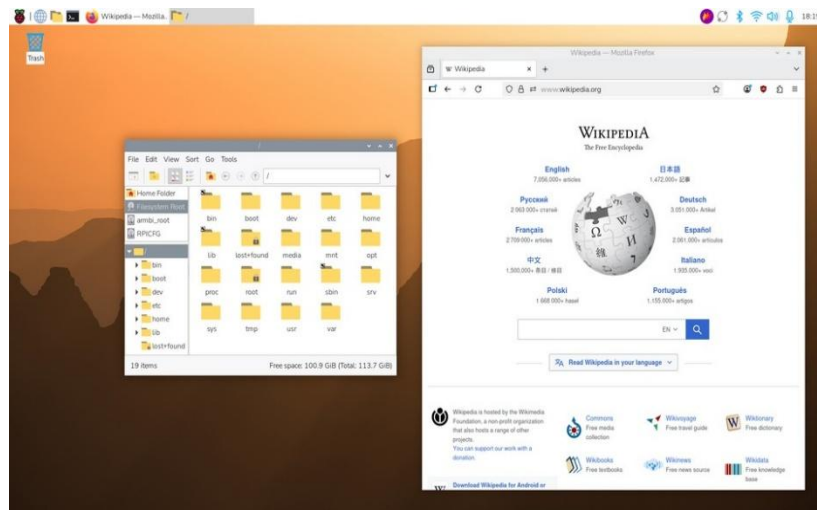


Figure 7.5 VNC Terminal

7.3.6 Comparative Evaluation of Monitoring Approaches

Throughout the implementation phase, three primary monitoring approaches were evaluated. Each solution was assessed with respect to latency, deployment complexity, maintainability, and suitability for real-time industrial inspection.

Table 7.2 Comparative Evaluation of Monitoring Approaches

Method	Advantages	Limitations
OpenCV Streaming	Simple implementation	Extremely high latency
RTSP + MediaMTX	Low latency streaming	Complex configuration
VNC	Complete desktop access, easy maintenance	Requires stable network

Based on experimental observations, the VNC solution offered the most balanced combination of functionality, reliability, and ease of deployment. While RTSP achieved lower latency, its implementation complexity outweighed the benefits for the intended deployment environment. As a result, VNC was selected as the final monitoring and visualization solution for the deployed inspection system.

7.4 InfluxDB Integration and Data Persistence

The successful deployment of the computer vision inference pipeline enabled real-time tablet defect detection on the Raspberry Pi. However, detection results alone were insufficient for practical industrial operation. A complete inspection system must also provide mechanisms for storing inspection records, maintaining production history, supporting traceability, and facilitating long-term performance analysis.

For this reason, a dedicated database infrastructure was incorporated into the proposed system. The selected database platform was InfluxDB, a high-performance time-series database specifically designed for storing chronological data generated by monitoring and industrial systems. The integration of InfluxDB transformed the inspection platform from a standalone detection system into a comprehensive monitoring solution capable of maintaining historical production records and supporting real-time visualization through Grafana dashboards.

The following sections describe the implementation, configuration, integration, and validation of the database infrastructure.

7.4.1 Motivation for Database Integration

During early testing phases, defect detection results were displayed directly within the inference window and stored locally in log files. While this approach was sufficient for initial verification, it presented several limitations when considered for continuous production monitoring.

First, local log files become increasingly difficult to manage as production duration increases. Searching through thousands of inspection entries manually is inefficient and unsuitable for industrial environments.

Second, local storage mechanisms provide limited support for real-time analytics and visualization. Production supervisors require immediate access to inspection statistics, defect distributions, and system performance indicators without manually examining text-based records.

Third, historical production analysis requires efficient querying capabilities. Information such as daily production counts, defect frequencies, and inspection trends must be retrievable quickly to support quality assurance activities.

To address these requirements, a centralized database solution was introduced. The database infrastructure was designed to perform the following functions:

- Store inspection events generated during inference.
- Maintain accurate timestamps for each inspection record.
- Support long-term storage of production statistics.
- Enable real-time dashboard visualization.
- Facilitate historical quality analysis.
- Improve traceability throughout the inspection process.

The introduction of database storage significantly enhanced the practical applicability of the proposed inspection system.

7.4.2 Selection of InfluxDB

Several database technologies were evaluated before selecting the final platform. Traditional relational databases such as MySQL and PostgreSQL were considered; however, these systems are primarily optimized for structured transactional data rather than continuously generated monitoring information. The inspection system generates timestamped inspection events continuously throughout operation. Each detection event includes a timestamp, classification result, confidence score, processing time, and inspection status. Such information naturally forms a time-series dataset.

InfluxDB was selected because it is specifically designed for storing and processing time-series information. The database provides efficient write operations, rapid query execution, and seamless integration with monitoring platforms such as Grafana.

Additional advantages of InfluxDB include:

- High-speed data ingestion.
- Efficient storage of timestamped records.
- Native support for industrial monitoring applications.
- Scalability for future expansion.
- Compatibility with Grafana visualization tools.
- Simplified dashboard integration.

These characteristics made InfluxDB an appropriate choice for the requirements of the proposed quality inspection platform.

7.4.3 Installation and Configuration of InfluxDB

The database infrastructure was deployed on the monitoring workstation rather than the Raspberry Pi. This architectural decision reduced computational overhead on the edge device and allowed the Raspberry Pi to focus exclusively on image acquisition and inference processing.

InfluxDB was deployed within a Docker container environment. Containerization simplified installation, dependency management, and future maintenance activities. The installation process involved downloading the InfluxDB container image, creating a persistent storage volume, and launching the database service through Docker.

The deployment workflow is illustrated below.

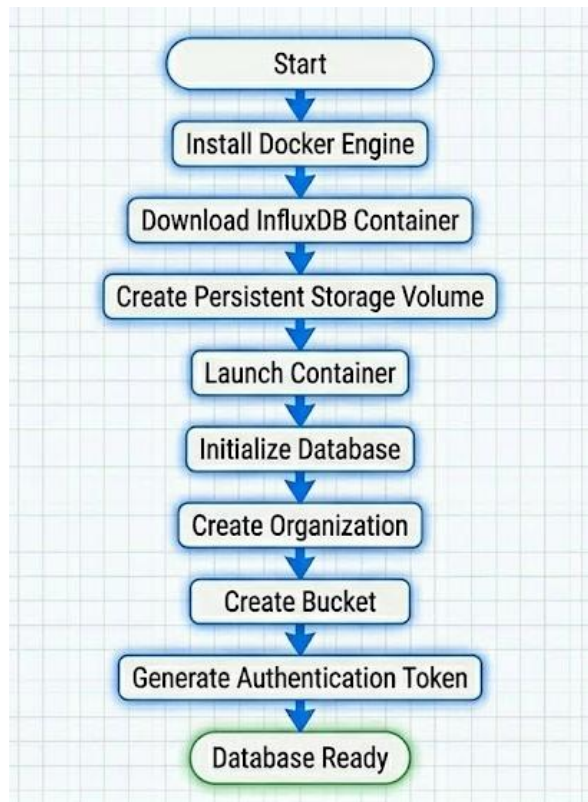


Figure 7.6 InfluxDB Deployment Workflow

After deployment, an organization and bucket were created within InfluxDB. The bucket served as the primary repository for inspection records generated by the computer vision system. Authentication tokens were generated to secure communication between the Raspberry Pi and the database server. These tokens ensured that only authorized applications could submit inspection records.

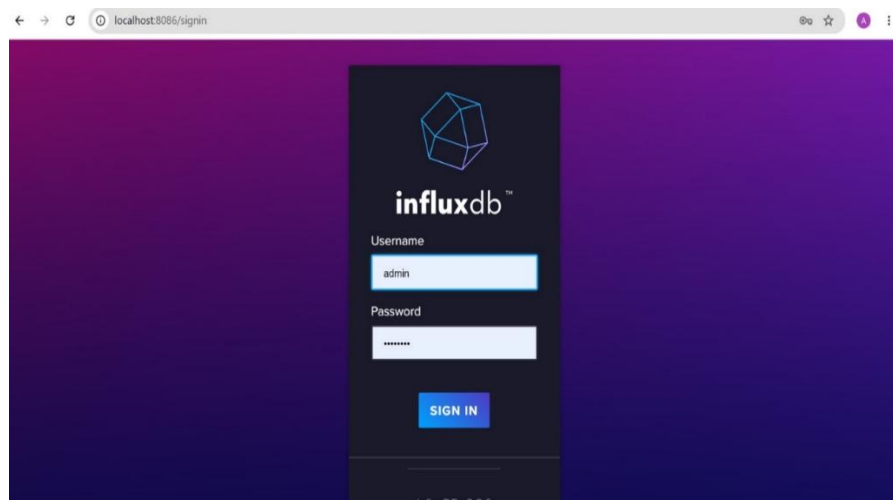


Figure 7.7 InfluxDB Login Interface

7.4.4 Inspection Data Structure

A structured schema was developed to ensure consistency across all inspection records. Each database entry contains information necessary for production monitoring and defect analysis.

Table 7.3 presents the data fields stored for each inspection event.

Table 7.3 Inspection Data Fields Stored in InfluxDB

Field Name	Description
Timestamp	Date and time at which the inspection record was generated
Pass Count	Total number of tablets classified as acceptable during a monitoring interval
Fail Count	Total number of tablets classified as defective during a monitoring interval
Total Count	Total number of inspected tablets (Pass + Fail)
FPS	Processing frame rate achieved during real-time inference
Inference Time	Time required to process a frame and generate a prediction
Batch ID	Identifier associated with the current inspection batch
System Source	Raspberry Pi edge device generating the inspection data

The use of structured fields simplified dashboard development and enabled efficient querying of inspection statistics.

7.4.6 Database Communication Validation

Following implementation of the database communication module, a series of validation tests were performed to verify correct system operation. Testing focused on ensuring reliable transmission of inspection records under various operating conditions.

The following validation procedures were performed:

- Verification of database connectivity.
- Validation of authentication credentials.
- Inspection record insertion testing.

- Timestamp accuracy verification.
- Continuous data logging evaluation.
- Data retrieval verification.

Table 7.4 Database Validation Results

Test Case	Result
Database Connection	Pass
Authentication Verification	Pass
Record Insertion	Pass
Timestamp Accuracy	Pass
Continuous Logging	Pass
Data Retrieval	Pass

The successful completion of these tests confirmed that the database infrastructure was

```
Data successfully written to InfluxDB
QC RESULT: {'time': '2026-04-30T17:17:01.909996', 'batch_id': 'B063', 'total': 3, 'ta
Data successfully written to InfluxDB
QC RESULT: {'time': '2026-04-30T17:17:03.499993', 'batch_id': 'B063', 'total': 4, 'ta
Data successfully written to InfluxDB
QC RESULT: {'time': '2026-04-30T17:17:05.004859', 'batch_id': 'B063', 'total': 2, 'ta
Data successfully written to InfluxDB
QC RESULT: {'time': '2026-04-30T17:17:06.501033', 'batch_id': 'B063', 'total': 3, 'ta
```

capable of supporting long-duration inspection activities.

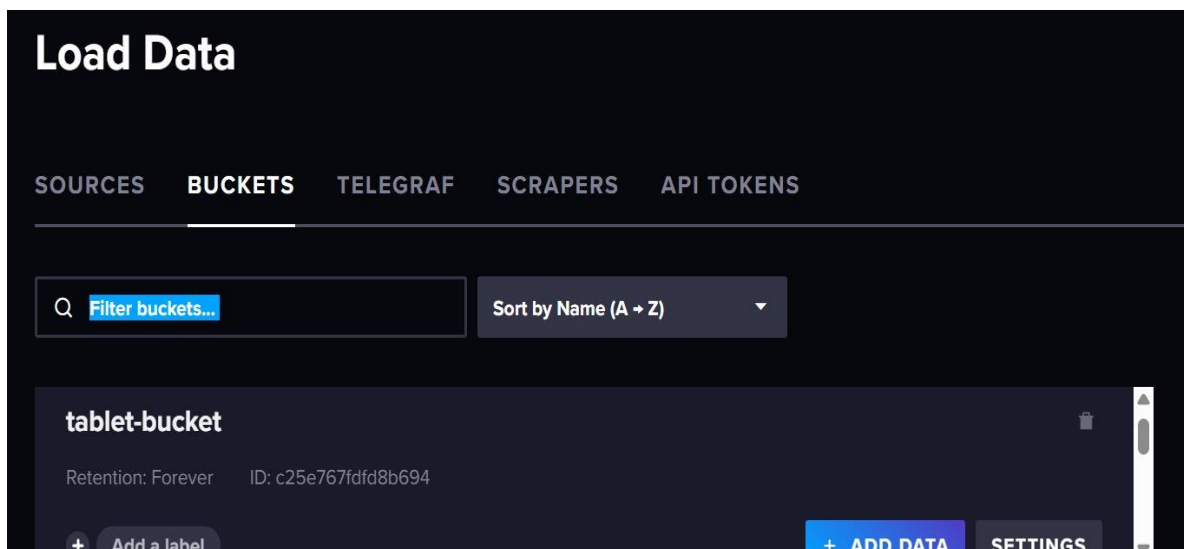


Figure 7.8 Data Bucket of a Tablet

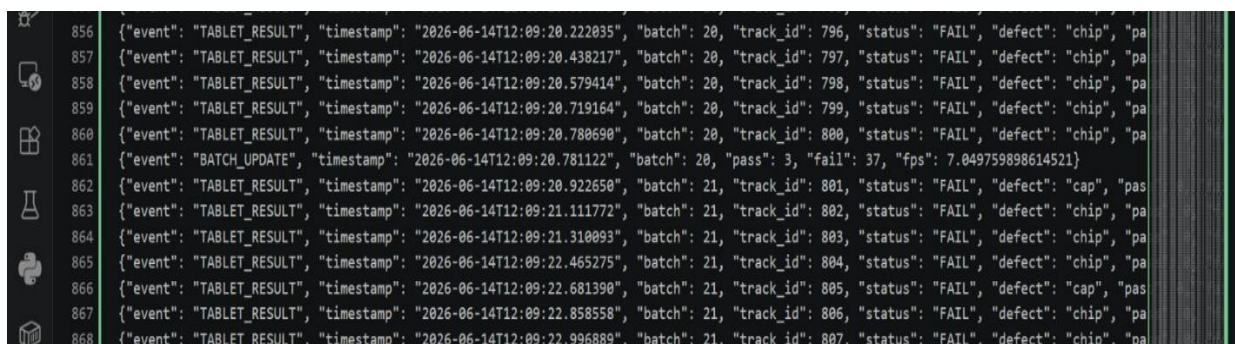
7.4.7 Benefits of Continuous Inspection Logging

One of the most significant advantages of the database integration was the ability to perform continuous inspection logging. Every tablet processed by the system automatically generated a corresponding database record. This functionality enabled complete traceability of production activities and provided a historical archive of inspection results. Because all inspection events were preserved within the database, production history could be analyzed retrospectively to identify trends, recurring defects, and process abnormalities. The logging infrastructure also provided a reliable foundation for Grafana dashboard integration, which is discussed in the subsequent section.

The integration of InfluxDB introduced several practical advantages that significantly enhanced the functionality of the proposed inspection platform.

- First, centralized storage eliminated dependence on local text-based logs and provided a scalable solution for long-term data retention.
- Second, the database infrastructure enabled real-time monitoring through direct integration with Grafana dashboards.
- Third, timestamped inspection records improved traceability and supported detailed production analysis.

Finally, the modular architecture ensured that future enhancements such as predictive maintenance, automated reporting, and cloud synchronization could be incorporated without modifying the core defect detection pipeline. The database infrastructure became a critical component of the overall inspection system and played a central role in enabling advanced monitoring and analytics capabilities.



```
856 {"event": "TABLET_RESULT", "timestamp": "2026-06-14T12:09:20.222035", "batch": 20, "track_id": 796, "status": "FAIL", "defect": "chip", "pa
857 {"event": "TABLET_RESULT", "timestamp": "2026-06-14T12:09:20.438217", "batch": 20, "track_id": 797, "status": "FAIL", "defect": "chip", "pa
858 {"event": "TABLET_RESULT", "timestamp": "2026-06-14T12:09:20.579414", "batch": 20, "track_id": 798, "status": "FAIL", "defect": "chip", "pa
859 {"event": "TABLET_RESULT", "timestamp": "2026-06-14T12:09:20.719164", "batch": 20, "track_id": 799, "status": "FAIL", "defect": "chip", "pa
860 {"event": "TABLET_RESULT", "timestamp": "2026-06-14T12:09:20.780690", "batch": 20, "track_id": 800, "status": "FAIL", "defect": "chip", "pa
861 {"event": "BATCH_UPDATE", "timestamp": "2026-06-14T12:09:20.781122", "batch": 20, "pass": 3, "fail": 37, "fps": 7.049759898614521}
862 {"event": "TABLET_RESULT", "timestamp": "2026-06-14T12:09:20.922650", "batch": 21, "track_id": 801, "status": "FAIL", "defect": "cap", "pas
863 {"event": "TABLET_RESULT", "timestamp": "2026-06-14T12:09:21.111772", "batch": 21, "track_id": 802, "status": "FAIL", "defect": "chip", "pa
864 {"event": "TABLET_RESULT", "timestamp": "2026-06-14T12:09:21.310093", "batch": 21, "track_id": 803, "status": "FAIL", "defect": "chip", "pa
865 {"event": "TABLET_RESULT", "timestamp": "2026-06-14T12:09:22.465275", "batch": 21, "track_id": 804, "status": "FAIL", "defect": "chip", "pa
866 {"event": "TABLET_RESULT", "timestamp": "2026-06-14T12:09:22.681390", "batch": 21, "track_id": 805, "status": "FAIL", "defect": "cap", "pas
867 {"event": "TABLET_RESULT", "timestamp": "2026-06-14T12:09:22.858558", "batch": 21, "track_id": 806, "status": "FAIL", "defect": "chip", "pa
868 {"event": "TABLET_RESULT", "timestamp": "2026-06-14T12:09:22.996889", "batch": 21, "track_id": 807, "status": "FAIL", "defect": "chip", "pa
```

Figure 7.9 Real-time Data Loggings

7.4.8 InfluxDB Dashobard

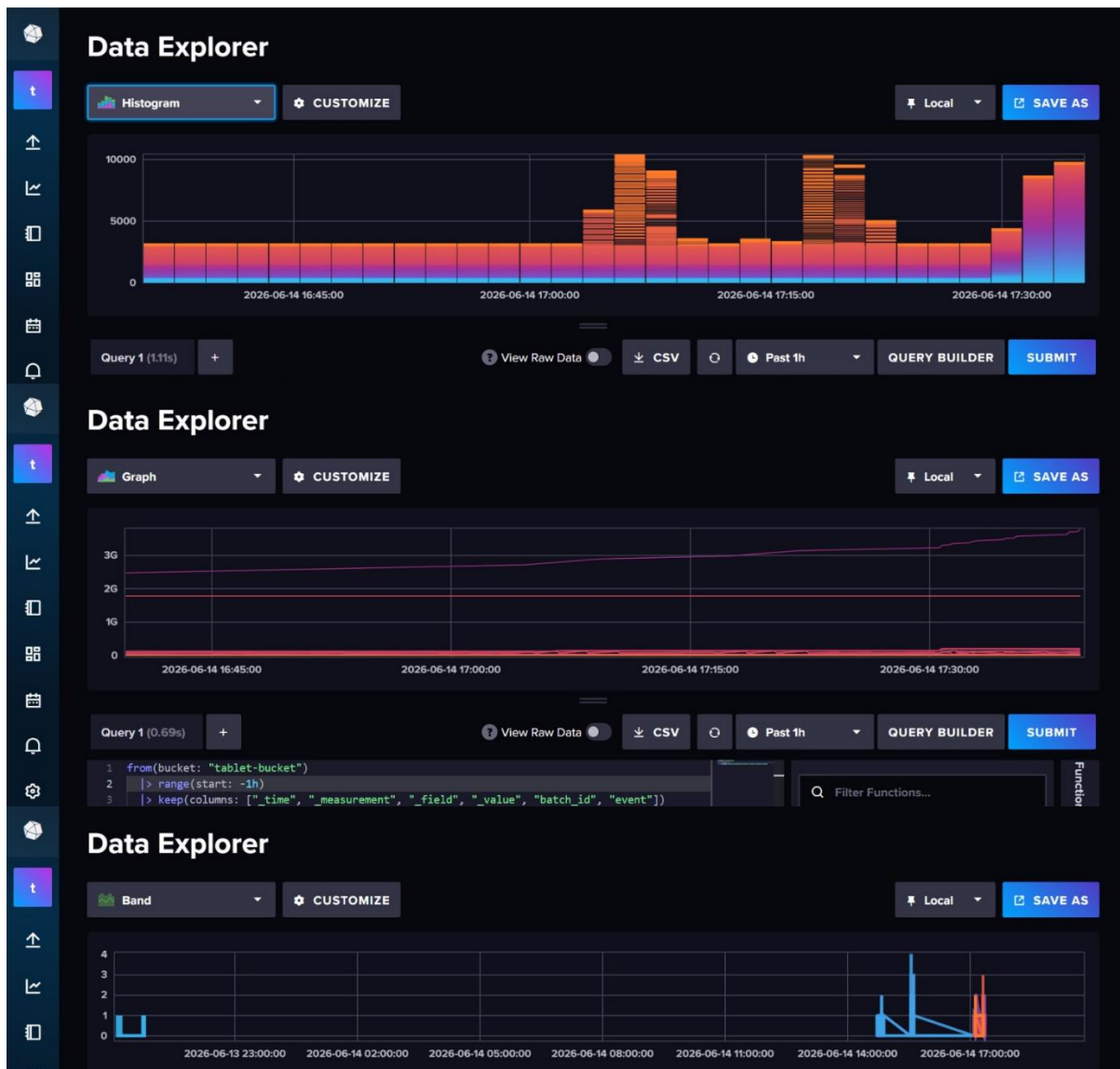


Figure 7.10 InfluxDB Dashboard

7.5 Dashboard Development and Visualization

The successful implementation of the InfluxDB database infrastructure enabled the storage of inspection records generated by the computer vision system. However, although database storage provides reliable persistence of production information, raw database entries are not easily interpretable by production operators or quality assurance personnel. Industrial environments require information to be presented in a clear, intuitive, and visually accessible manner that supports rapid decision-making. Consequently, a dedicated visualization platform was required to transform inspection records into meaningful production insights. To satisfy

this requirement, Grafana was selected as the primary dashboard and monitoring platform for the proposed tablet defect detection system.

Grafana is an open-source analytics and visualization platform widely used in industrial monitoring, manufacturing systems, network administration, and Internet of Things (IoT) applications. The platform provides extensive support for real-time data visualization through configurable dashboards and integrates seamlessly with time-series databases such as InfluxDB. The adoption of Grafana enabled the development of a centralized monitoring environment capable of presenting inspection results, production statistics, defect distributions, and system performance metrics through a unified graphical interface.

The primary objective of dashboard implementation was to provide operators with immediate visibility into production activities without requiring direct access to the database infrastructure. Through graphical visualization, operators could monitor system performance, identify abnormal defect rates, observe production trends, and evaluate inspection outcomes in real time. The dashboard therefore served as the primary human-machine interface for the deployed inspection system.

7.5.1 Grafana Installation and Initial Configuration

The Grafana platform was deployed on the monitoring workstation alongside the InfluxDB infrastructure. Similar to the database deployment strategy, Grafana was executed within a Docker container environment. Containerization simplified installation, minimized dependency conflicts, and improved portability of the monitoring infrastructure. Furthermore, the use of Docker ensured that dashboard services could be restarted, upgraded, or migrated without affecting the operation of the inspection system running on the Raspberry Pi.

The installation process began with downloading the Grafana container image and creating the required network configuration for communication with the InfluxDB database. Once the container was launched successfully, the Grafana web interface became accessible through a web browser. Initial configuration activities included administrator account creation, security configuration, and verification of container health status.

The complete deployment process is illustrated below.

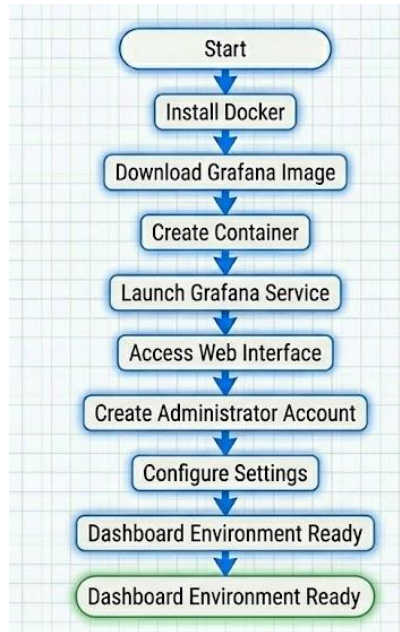


Figure 7.11 Grafana Deployment Workflow

Following successful installation, accessibility testing was performed to ensure that the Grafana interface remained available after system reboots and service restarts. Several deployment iterations were conducted before a stable monitoring environment was achieved. The final configuration provided reliable operation throughout subsequent testing activities.

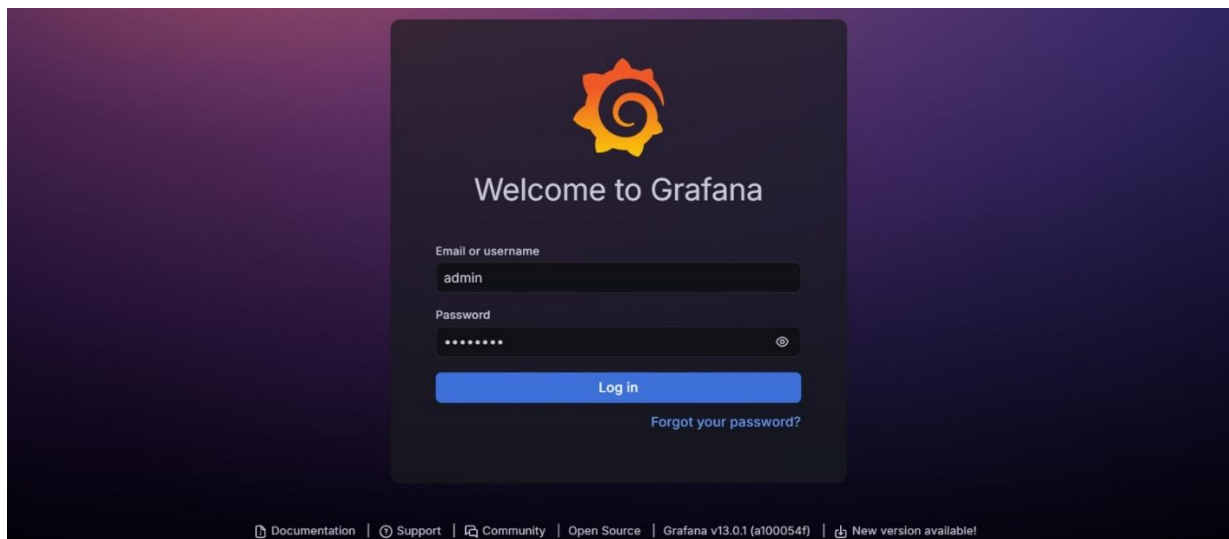


Figure 7.12 Grafana Login Interface

7.5.2 Configuration of Data Source

After completing the Grafana installation process, the next implementation phase involved establishing communication between Grafana and the InfluxDB database. This connection was essential because Grafana obtains all visualization data directly from the database. Without a properly configured data source, dashboard panels would be unable to retrieve inspection records generated by the defect detection system. The configuration procedure required several parameters including the database server address, authentication token, organization identifier, and bucket information. These parameters were obtained from the InfluxDB configuration interface and entered into the Grafana data source settings panel.

The process of configuring database connectivity involved multiple validation stages to ensure that communication between both platforms was functioning correctly. Test queries were executed to verify authentication credentials and confirm successful retrieval of stored inspection records.

The connectivity configuration process is illustrated below.

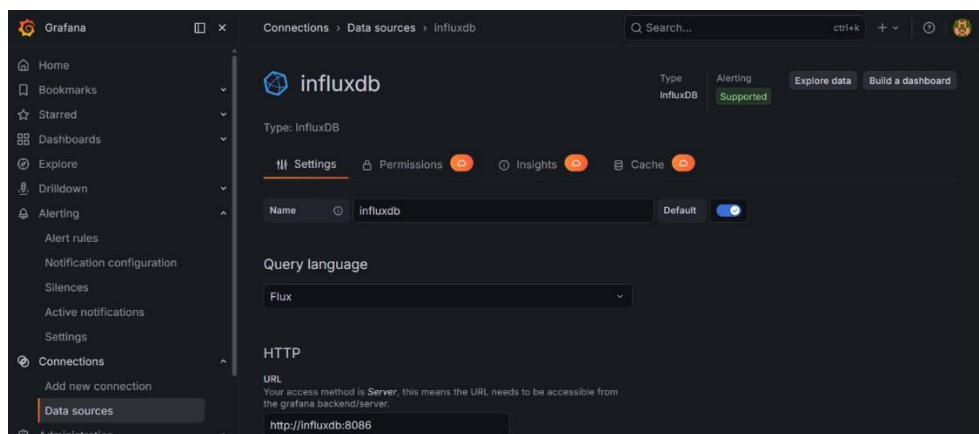


Figure 7.13 Successful configuration of the InfluxDB data source within Grafana.

Once connectivity was established, Grafana gained direct access to inspection data generated by the Raspberry Pi inference pipeline. This achievement represented a major milestone in the implementation process because it enabled real-time visualization of production information.

InfluxDB Details

Organization	tablet-org	
Token	configured	Reset
Default Bucket	tablet-bucket	
Min time interval ⓘ	10s	
Max series ⓘ	1000	

Delete **Save & test**

Figure 7.14 Data Source login within Grafana

7.5.3 Dashboard Design Objectives

Before creating individual dashboard panels, a systematic design strategy was developed to ensure that the final interface satisfied operational requirements. The dashboard was intended to function as the primary monitoring environment for production operators and quality control personnel. Therefore, information presentation needed to be simple, intuitive, and capable of supporting rapid interpretation under real-world operating conditions.

Several design objectives guided dashboard development. First, critical production statistics such as total inspection count, accepted tablets, and defective tablets needed to be visible immediately upon opening the dashboard. These indicators provide a high-level overview of production status and enable operators to assess manufacturing performance quickly.

Second, defect-related information needed to be presented visually to facilitate identification of recurring quality issues. The ability to monitor defect frequency is particularly important because sudden increases in specific defect categories may indicate equipment malfunction, process instability, or raw material problems.

Third, the dashboard needed to support historical analysis through trend visualization. Time-series graphs allow production personnel to observe changes in inspection activity over time and identify abnormal production behavior.

Finally, the dashboard needed to update automatically as new inspection records became available. Real-time updating ensures that production decisions can be based on current manufacturing conditions rather than delayed information.

These design considerations significantly influenced the structure and layout of the final dashboard implementation.

7.5.4 Development of Production Statistics Panels

The first group of dashboard components focused on presenting overall production statistics. These panels were designed to provide operators with an immediate understanding of production performance and inspection activity. Several numerical panels were created to display key production indicators. These included total inspected tablets, accepted tablets, rejected tablets, and overall inspection count. The values displayed within these panels were calculated dynamically using Flux queries executed against the InfluxDB database. The development of these panels required careful query construction to ensure accurate aggregation of inspection records. Extensive testing was performed to validate query results against database contents and confirm consistency between displayed values and stored records.

The panel creation workflow is illustrated below.

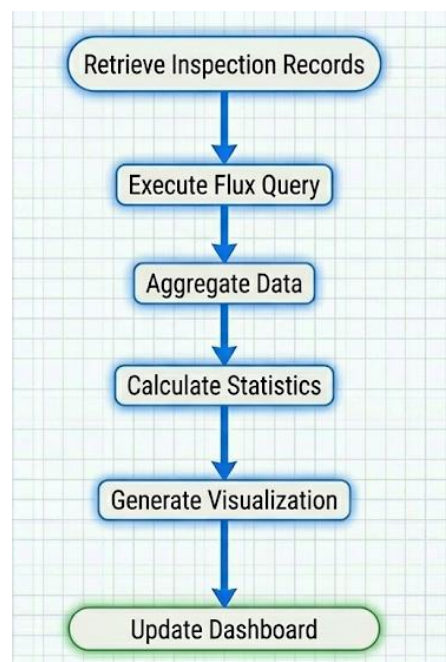


Figure 7.15 Panel Development Workflow

One of the most powerful capabilities offered by Grafana is support for time-series visualization. Since all inspection records stored within InfluxDB contained timestamps, it was possible to analyze production activity across different time intervals.

Time-series panels were developed to display inspection trends, production rates, and defect occurrences over time. These graphs provided valuable insight into system behavior and enabled observation of changes occurring throughout production runs.

The implementation of time-series visualization required careful selection of aggregation windows and query intervals. Excessively small intervals produced noisy graphs, while excessively large intervals reduced visibility of short-term fluctuations. Through repeated experimentation, appropriate visualization parameters were identified.

Time-series monitoring provided several practical benefits. Production operators could identify periods of abnormal defect generation, observe production throughput, and evaluate system consistency over extended operating durations.

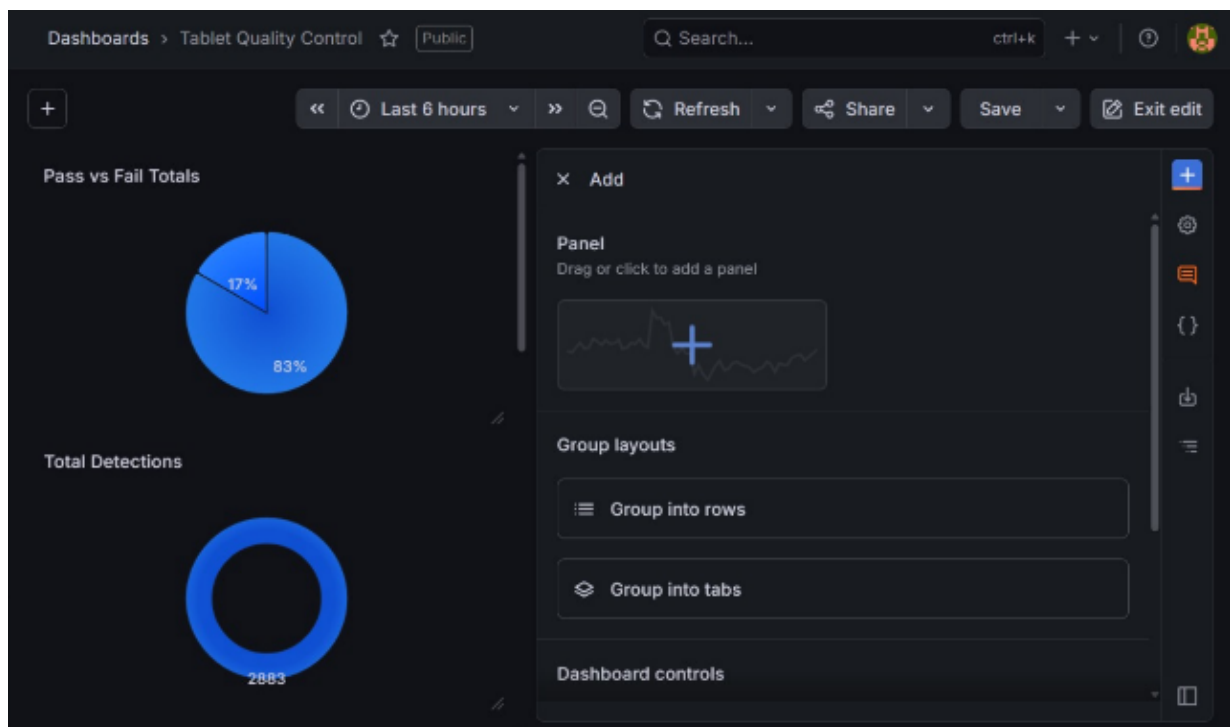


Figure 7.16 Panel Selection & Designing Process

7.5.8 Real-time Tablet Inspection Dashboard



Figure 7.17 Tablet Quality Control Cumulative Dashboard

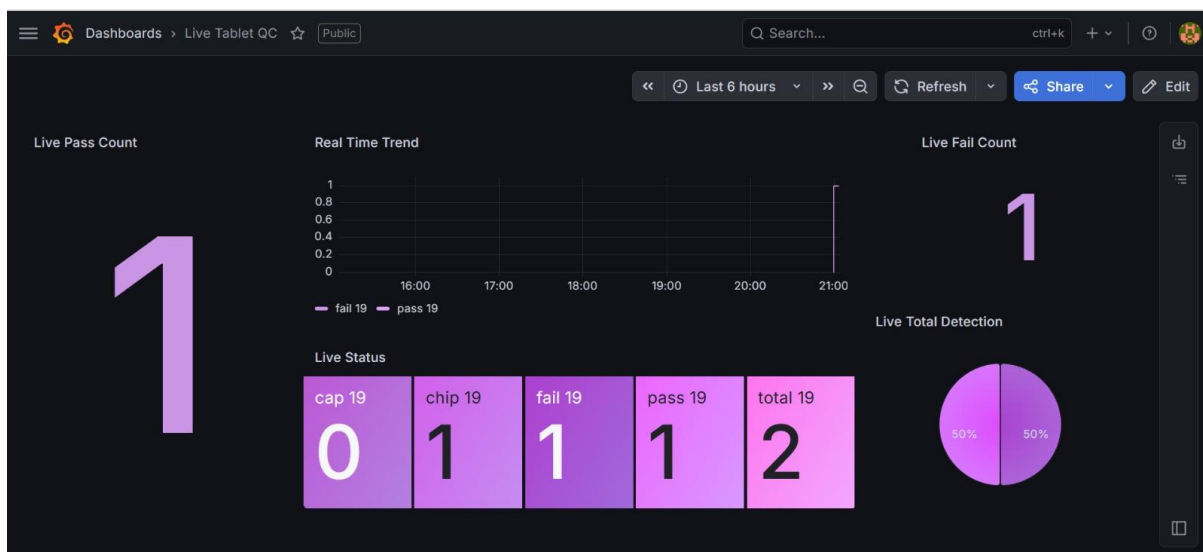


Figure 7.18 Tablet Quality Control Real-time Dashboard

7.6 System Testing and Experimental Validation

The successful implementation and integration of the proposed tablet inspection system was followed by a comprehensive testing and validation phase aimed at evaluating the real-world operational reliability of the complete pipeline. The primary objective of this phase was to ensure that all hardware and software components functioned cohesively under practical deployment conditions, while also verifying that the system was capable of performing real-time defect detection with acceptable stability and consistency. Unlike the implementation

stage, which focused on building individual modules such as the inference engine, database communication layer, and visualization dashboard, the testing phase emphasized end-to-end system behavior, including communication flow, processing latency, and continuous operation under dynamic conditions.

Testing activities were conducted across multiple environments in order to evaluate system robustness under varying operational constraints. These included controlled laboratory experiments using a conveyor-based inspection setup, as well as additional validation in a non-laboratory environment to assess adaptability under different network configurations and environmental conditions. In both cases, the system was evaluated for real-time detection capability, stability of data transmission to InfluxDB, correctness of dashboard visualization through Grafana, and reliability of remote monitoring via VNC. The outcomes of these experiments provided practical evidence that the proposed edge-based inspection architecture is feasible for small-scale pharmaceutical quality control applications, particularly where low-cost deployment and real-time monitoring are essential.

6.6.1 Testing Methodology

A structured and modular testing methodology was adopted to systematically evaluate the performance of the deployed system. The testing strategy was designed to first validate each component independently before progressing toward full system integration. Individual modules such as the camera capture pipeline, YOLO-based inference engine, InfluxDB data logging mechanism, Grafana visualization layer, and remote access configuration were initially tested in isolation to ensure correct functionality and eliminate configuration-level issues. This approach allowed for early identification of errors and ensured that each subsystem was stable before integration into the full pipeline.

After successful component-level validation, integrated system testing was performed where all modules operated simultaneously in a continuous workflow. The system was executed in a pipeline format starting from image acquisition on the Raspberry Pi camera, followed by real-time object detection using the YOLO model, generation of inference results, transmission of structured data to the database layer, and final visualization through the Grafana dashboard. This end-to-end testing approach ensured that the complete workflow could operate without interruption while maintaining synchronization between hardware processing, software inference, and data visualization components.

7.6.2 Testing in Laboratory Environment

The initial phase of experimental validation was conducted in a controlled laboratory environment using a conveyor-based inspection system developed at the university. This setup provided a stable and repeatable testing environment where variables such as lighting conditions, conveyor speed, and camera positioning could be controlled with precision. The conveyor transported tablets through a defined inspection zone positioned directly beneath the camera, allowing continuous image capture and real-time processing through the deployed YOLO inference pipeline running on the Raspberry Pi.

During laboratory testing, the system was evaluated under multiple operational conditions, including varying tablet positions, different defect categories, and continuous movement scenarios. The controlled environment allowed for systematic observation of detection accuracy, frame processing stability, and communication reliability between system components. The system successfully detected defective tablets while maintaining continuous operation, and detection outputs were simultaneously logged into the InfluxDB database and visualized through the Grafana dashboard in real time. Remote monitoring via VNC further enabled live observation of inference behavior, confirming the correct execution of the deployed pipeline.



Figure 7.19 Testing in Laboratory Environment

7.6.3 Testing in Home Environment

Following successful validation in the laboratory setup, additional testing was conducted in a home environment to evaluate system adaptability under non-controlled conditions. This phase of testing was particularly important for assessing how the system behaves under variations in network configuration, ambient lighting, and hardware setup differences. Unlike the laboratory environment, where infrastructure is optimized for experimentation, the home setup introduced real-world variability that helped in identifying deployment-level constraints.

One of the primary challenges encountered during this phase was related to network configuration differences, particularly changes in IP addressing and connectivity parameters required for communication between the Raspberry Pi, database server, and dashboard interface. However, after reconfiguring the network settings and utilizing VNC for remote access, the system was successfully operated without major interruptions. Despite environmental variations, the inference pipeline continued to function correctly, and detection results were consistently transmitted to the database and reflected in the dashboard. This phase demonstrated that the system is not strictly limited to a single deployment environment and can be reconfigured for operation across different setups with minimal adjustments.

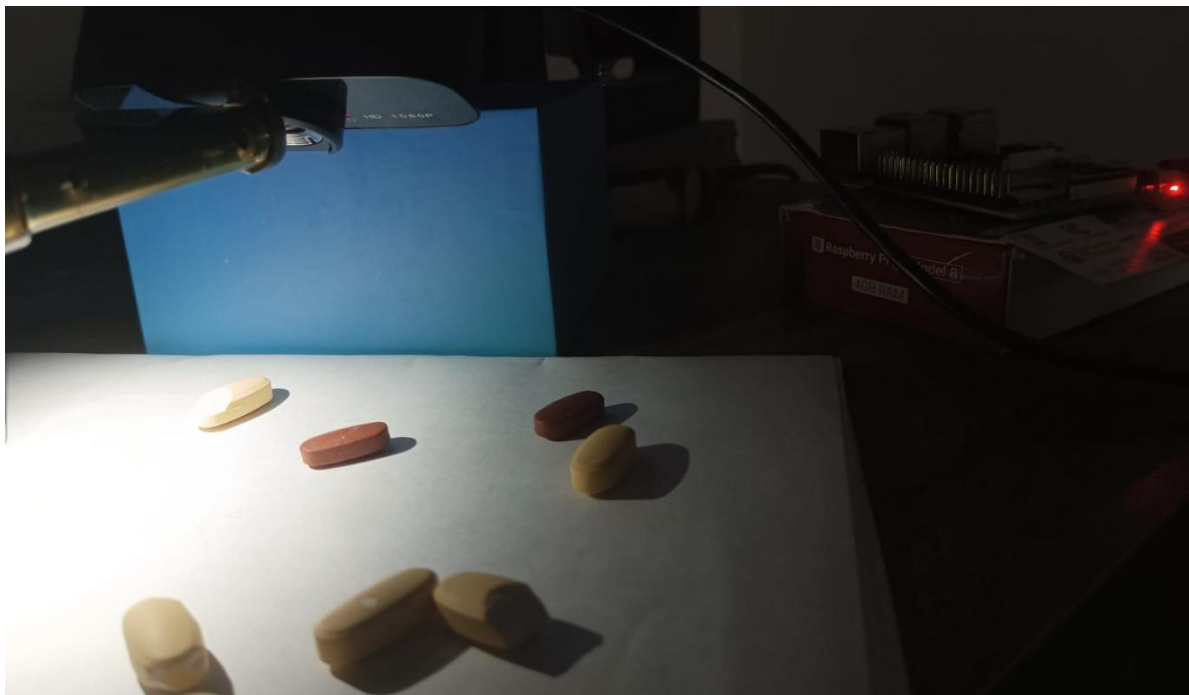


Figure 7.20 Testing in Home Environment

7.6.4 Real-Time Defect Detection Validation

A critical objective of the testing phase was to validate the real-time performance of the YOLO-based defect detection model when deployed on edge hardware. While training-phase evaluation had already provided insights into model accuracy, deployment testing focused on runtime performance, inference speed, and stability under continuous operation. Tablets with both normal and defective characteristics were introduced into the conveyor stream, and image frames were continuously processed by the inference engine running on the Raspberry Pi.

The system successfully performed real-time detection, generating bounding boxes and confidence scores for detected defects without interrupting the conveyor flow. Each detection result was immediately structured and transmitted to the backend database, ensuring synchronized logging of inspection data. The ability to perform continuous inference while maintaining system responsiveness demonstrated that the deployed model is suitable for edge-based quality control applications where low latency and uninterrupted operation are required.

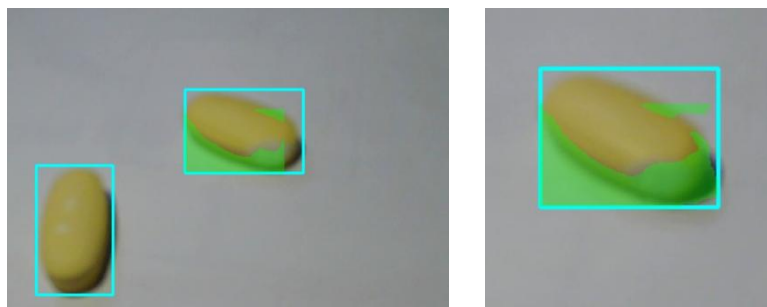


Figure 7.21 Real-time Defect Detection

7.6.5 Conveyor-Based Inspection Testing

An important aspect of system validation involved evaluating performance under continuous conveyor movement, as industrial inspection systems are required to operate under uninterrupted production conditions. During this testing phase, tablets were continuously transported through the inspection region, and the system was required to perform detection without manual intervention or process interruption. The camera was carefully positioned to ensure stable coverage of the inspection zone, while the inference pipeline processed each frame in real time.

Across multiple test runs, the system maintained stable operation without any critical failures or breakdowns in communication between components. Detection outputs were consistently

generated, and data transmission to the InfluxDB database remained stable throughout the testing period. These results confirmed that the system is capable of supporting continuous inspection workflows, although performance optimization would be required for high-speed industrial-scale deployment scenarios.



Figure 7.22 Conveyor-Based Inspection Testing

7.6.6 Throughput Estimation and Production Capacity Analysis

An important metric for evaluating the practical applicability of the proposed system is its throughput capacity under experimental conditions. Throughput estimation was performed based on the observed conveyor speed and the rate at which tablets passed through the inspection region during testing. It was observed that the system was capable of processing approximately 1.5 to 2 tablets per second under stable operating conditions, depending on conveyor speed and tablet spacing.

Based on these observations, the estimated throughput range of the system is approximately 90 to 120 tablets per minute. This value was derived as a practical approximation rather than a fixed benchmark, as real-world performance can vary depending on lighting conditions, hardware load, and system optimization levels. Although this throughput is lower than that of industrial-grade pharmaceutical inspection systems, it demonstrates the viability of the proposed architecture for prototype development, research environments, and small-scale production scenarios where cost efficiency and flexibility are prioritized.

In terms of overall system performance, the complete end-to-end inspection pipeline achieved an estimated **80% to 85% accuracy under experimental conditions**, considering detection

reliability across continuous conveyor operation, stable communication between modules, and consistency of real-time defect identification. This performance reflects the combined effectiveness of the YOLO-based inference model along with the supporting hardware and software infrastructure. Variations in accuracy were primarily influenced by environmental factors such as lighting variation, motion blur during fast conveyor movement, and Raspberry Pi processing constraints under sustained load. Despite these limitations, the system maintained consistent detection behavior and demonstrated reliable operation for prototype-level deployment.

7.6.7 End-to-End System Validation

The final stage of testing involved complete end-to-end validation of the entire inspection pipeline to ensure seamless interaction between all system components. This included real-time image acquisition from the camera module, YOLO-based inference execution on the Raspberry Pi, defect classification and result generation, transmission of structured data to the InfluxDB database, and visualization of results through the Grafana dashboard. Additionally, remote monitoring via VNC provided operational visibility into system performance during live execution.

The experimental validation results demonstrate that the proposed tablet defect detection system has been successfully deployed and tested under both controlled and semi-controlled environments. The system consistently achieved real-time defect detection using a YOLO-based inference model deployed on edge hardware, while maintaining stable communication between the camera module, processing unit, database layer, and visualization dashboard. The integration of InfluxDB and Grafana provided reliable real-time monitoring capabilities, enabling effective visualization of inspection data and system performance metrics.

Although the system operated successfully across different environments, certain limitations were observed during extended testing on Raspberry Pi hardware, particularly related to processing load and thermal constraints under continuous inference execution. These constraints highlight the importance of hardware optimization and efficient model deployment strategies for sustained industrial use. Despite these limitations, the system demonstrated consistent performance, stable data logging, and reliable real-time visualization, confirming its suitability for prototype-level and small-scale automated inspection applications.

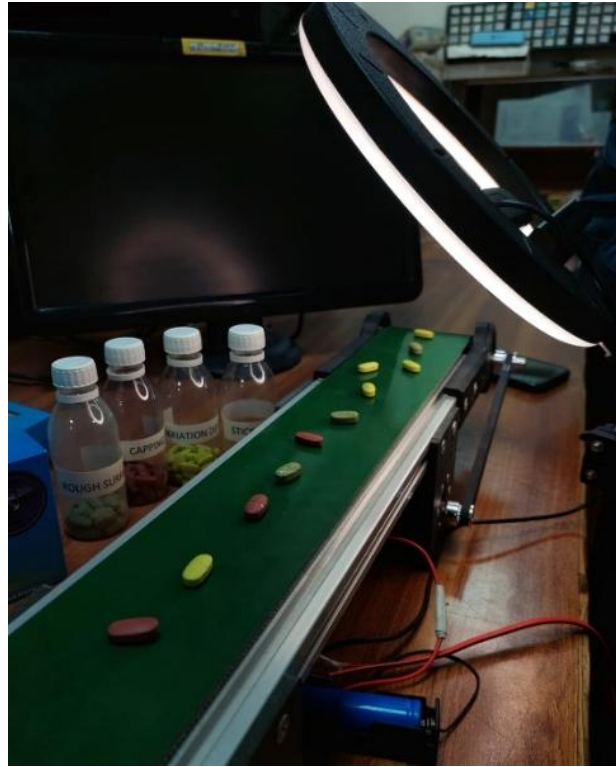


Figure 7.23 End-to-End System

7.7 Summary

This chapter presented the comprehensive implementation, deployment, and experimental validation of the proposed tablet defect detection system. The chapter detailed the configuration of the Raspberry Pi-based edge computing platform, integration of YOLO-based real-time inference, establishment of InfluxDB for structured data logging, and deployment of Grafana dashboards for real-time monitoring. The system was further enhanced with remote access capabilities using VNC, enabling effective monitoring of inference operations in real time.

Extensive testing was conducted in both laboratory and non-laboratory environments to evaluate system functionality, robustness, and adaptability. The results confirmed that the system is capable of performing real-time defect detection while maintaining stable communication between all modules. Conveyor-based testing demonstrated continuous inspection capability, and throughput analysis estimated a processing capacity of approximately 90 to 120 tablets per minute under experimental conditions. Overall, the results validate the feasibility of the proposed system as a cost-effective and scalable solution for automated pharmaceutical tablet inspection in edge-computing environments.

Chapter 8

Conclusion

8.1 Summary of the Whole Project

This project presented the design and development of an in-line tablet quality control system based on computer vision for automated defect detection. The primary objective was to replace conventional off-line quality inspection practices with a non-contact, real-time inspection approach capable of analyzing every tablet on a production line. The system focused on identifying common surface defects, including edge chipping, surface damage, smearing, spots, and scratches, to improve quality assurance efficiency and product safety.

We designed a complete end-to-end solution consisting of a controlled imaging setup, a data-driven model development pipeline, and a real-time software interface for visualization and reporting. A conveyor-based imaging rig with controlled illumination was used to capture high-quality images of tablet cores under consistent conditions. A structured data collection and preparation workflow was established to ensure balanced datasets and reliable annotations.

The computer vision model development process was carried out in multiple phases to systematically address challenges related to dataset size, class imbalance, preprocessing, and annotation strategy. Through iterative experimentation, we identified that defect-only annotations provided the most meaningful learning signal for defect localization. Increasing dataset diversity while maintaining annotation consistency improved model robustness and generalization. The final trained model achieved reliable defect detection performance suitable for practical deployment in an in-line inspection environment.

Alongside deep learning experimentation, the project also investigated classical mathematical vision models to provide interpretable and computationally lightweight alternatives. These approaches employed adaptive thresholding, contour analysis, convex hull subtraction, Laplacian gradient profiling, morphological operations, geometric deviation analysis, and colorimetric feature extraction to identify tablet defects under constrained hardware environments. The final system adopted a hybrid architecture combining mathematical vision and deep learning segmentation, leveraging classical computer vision for efficient localization

and geometric filtering while utilizing segmentation networks for high-precision defect characterization.

Deployment was achieved by incorporating the entire pipeline into a system consisting of conveyor belt image capturing equipment, proper lighting conditions, high-resolution cameras, edge detection via Raspberry Pi, and a real-time monitoring system comprising InfluxDB and Grafana dashboard tools. The final system is capable of handling processes such as image capturing, pre-processing, inference of segmentation, defect analysis, logging, and manufacturing visualization under one umbrella. Results from the experiment show that the architecture developed allows for efficient real-time tablet inspection with reduced manual intervention in the process.

Implemented a real-time inference pipeline and a human-machine interface to visualize detection results, generate quality control reports, and maintain traceability through logging.

The system architecture was designed with scalability in mind, allowing future expansion to higher throughput and multi-lane conveyor configurations.

Overall, this project demonstrates the feasibility and effectiveness of using computer vision for automated tablet quality inspection. The proposed system has the potential to reduce manual inspection effort, minimize production waste, and enable early detection of defects, thereby supporting improved manufacturing quality and operational efficiency.

References

- [1] A. Diószegi, M. Ficzer, L. A. Mészáros, O. Péterfi, A. Farkas, D. L. Galata, and Z. K. Nagy, “Automated tablet defect detection and the prediction of disintegration time and crushing strength with deep learning based on tablet surface images,” *International Journal of Pharmaceutics*, vol. 667, p. 124896, 2024. Available: <https://doi.org/10.1016/j.ijpharm.2024.124896>
- [2] M. Ficzer, L. A. Mészáros, N. Kállai-Szabó, A. Kovács, I. Antal, Z. K. Nagy, and D. L. Galata, “Real-time coating thickness measurement and defect recognition of film coated tablets with machine vision and deep learning,” *International Journal of Pharmaceutics*, vol. 623, p. 121957, 2022. Available: <https://www.sciencedirect.com/science/article/pii/S0378517322005129>
- [3] A. K. K. Pillai, A. Ellis, H. H. Rossato, and E. Fallon, “Real-time tablet inspection using computer vision for blister packing machines,” in *Proc. 2023 5th Int. Conf. on Computer and Applications (ICCA)*, Cairo, Egypt, 2023, pp. 39–44. Available: <https://ieeexplore.ieee.org/document/10401685>
- [4] A. A. M. S. Ibrahim and J.-R. Tapamo, “A survey of vision-based methods for surface defects’ detection and classification in steel products,” *Informatics*, vol. 11, no. 2, p. 25, 2024. Available: https://www.researchgate.net/publication/380030034_A_Survey_of_Vision-Based_Methods_for_Surface_Defects%27_Detection_and_Classification_in_Steel_Products
- [5] M. H. F. Wilkinson and J. B. T. M. Roerdink, “Fast morphological attribute operations using Tarjan's union-find algorithm,” *Computers & Mathematics with Applications*, vol. 62, no. 3, pp. 1188–1197, 2011. Available: <https://www.sciencedirect.com/science/article/pii/S0898122111009850>
- [6] X. Zhang, Y. Wang, and J. Liu, “Research on surface defect detection method based on machine vision,” *Journal of Physics: Conference Series*, vol. 2365, no. 1, p. 012045, 2022. Available: <https://doi.org/10.1088/1742-6596/2365/1/012045>

[7] M. A. Shafique and M. A. Khan, “Automatic product quality inspection using computer vision systems,” 2015. Available:

https://www.researchgate.net/publication/286111321_Automatic_Product_Quality_Inspection_Using_Computer_Vision_Systems